UNIVERSITÉ DE DOUALA

THE UNIVERSITY OF DOUALA

* * *

ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE DE DOUALA

(ENSPD)

* * *

DÉPARTEMENT DE GÉNIE INFORMATIQUE

MÉMOIRE DE MASTER

Master of Science en Data Science et Intelligence Artificielle (SDIA)

COMPARAISON ENTRE L'ALGORITHME EM CLASSIQUE ET L'EM VARIATIONNEL

(Avantages, inconvénients et applications)

Dressé et présenté par :

KANA TIODONG ICARE BRICHEL

Mémoire co-encadré par :

Prof. Dr. habil. P. NJIONOU SADJANG

Professeur Associé, École Nationale Supérieure Polytechnique de Douala

Prof. Patrice TAKAM SOH

Professeur Associé, Université de Yaoundé 1

Année Académique 2025 – 2026

Table des matières

Liste des abréviations et sigles **1**

1 FONDEMENTS THÉORIQUES ET ALGORITHMES EM **2**

1.1 Modèles à variables latentes 2

1.1.1 Définition formelle 2

1.1.2 Taxonomie des variables latentes 3

1.1.3 Exemples fondamentaux 3

1.2 Vraisemblance et estimation 4

1.2.1 Vraisemblance complète et vraisemblance incomplète 4

1.2.2 L'inégalité de Jensen et le principe de la borne inférieure 5

1.2.3 Identifiabilité et unicité 5

1.3 Modèles de mélanges gaussiens (GMM) 5

1.3.1 La distribution gaussienne univariée 5

1.3.2 La distribution gaussienne multivariée 6

1.3.3 Le modèle de mélange gaussien 6

1.3.4 Relation entre GMM et k-means 7

1.3.5 Exemple numérique : évaluation de la densité d'un GMM 7

1.4 Introduction au cadre bayésien 8

1.4.1 La philosophie bayésienne 8

1.4.2 Le théorème de Bayes et ses composantes 8

1.4.3 Distributions conjuguées 9

1.4.4 Exemple numérique illustratif : conjugaison Dirichlet–Catégorielle 9

1.5 La divergence de Kullback-Leibler 10

1.5.1 Définition et propriétés 10

1.5.2 Interprétation informationnelle 10

1.5.3 KL et ELBO : la décomposition fondamentale 10

1.5.4 KL directe et KL inverse : le phénomène de zero-forcing 11

1.6 L'algorithme EM classique 12

1.6.1 Historique et contexte 12

1.6.2 Formulation mathématique complète 12

1.6.3 Propriétés théoriques 15

1.6.4 Application au mélange gaussien : exemple numérique détaillé . 17

1.6.5 Variantes de l'EM classique 18

1.6.6 Extension aux données séquentielles : l'algorithme de Baum-Welch 19

1.6.7 Avantages de l'EM classique 21
1.6.8 Inconvénients et limites de l'EM classique 21
1.7 L'algorithme EM variationnel 22
1.7.1 Motivation et cadre général 22
1.7.2 Inférence variationnelle 23
1.7.3 Formulation de l'EM variationnel 24
1.7.4 Sélection automatique du nombre de composantes 27
1.7.5 Exemple numérique illustratif 28
1.7.6 Calcul de l'ELBO et critère de convergence 29
1.7.7 Variantes avancées de l'EM variationnel 29
1.7.8 Avantages de l'EM variationnel 30
1.7.9 Inconvénients et limites de l'EM variationnel 31
1.8 Conclusion du chapitre 1 32

2 COMPARAISON ENTRE L'EM CLASSIQUE ET L'EM VARIATIONNEL 34
2.1 Comparaison théorique 34
2.1.1 Dérivation de l'équivalence de Neal et Hinton 36
2.1.2 Trois niveaux d'inférence : MLE, MAP et Bayes complet 37
2.2 Comparaison algorithmique 38
2.2.1 Complexité computationnelle 38
2.2.2 Comportement numérique et stabilité 39
2.2.3 Critères de sélection de modèle 39
2.3 Étude expérimentale 41
2.3.1 Protocole expérimental 41
2.3.2 Résultats de l'EM classique (K = 6 imposé) 43
2.3.3 Résultats de l'EM variationnel (Kmax = 6) 44
2.3.4 Comparaison visuelle et qualité de partition 45
2.3.5 Sensibilité au choix du prior de Dirichlet (α_0) 46
2.3.6 Étude de la scalabilité computationnelle 48
2.3.7 Cas de référence : nombre de composantes correctement spécifié (K = 3) 49
2.3.8 Convergence de l'ELBO 50
2.3.9 Robustesse à l'initialisation 51
2.3.10 Sensibilité au type de covariance 52
2.3.11 Validation sur un jeu de données réel (Iris) 53
2.3.12 Sensibilité aux valeurs aberrantes 55
2.3.13 Réplication sur des jeux de données indépendants 56
2.4 Analyse des résultats et synthèse 57

2.5 Discussion critique du protocole expérimental 58

3 APPLICATIONS ET IMPLÉMENTATION 60

3.1 Applications pratiques 60
3.1.1 Traitement automatique du langage naturel 60
3.1.2 Vision par ordinateur 63
3.1.3 Bio-informatique 63
3.1.4 Finance 64
3.1.5 Systèmes de recommandation 70
3.2 Implémentation de l'EM classique 71
3.2.1 Étapes E et M 72
3.2.2 Initialisation, relances et critère d'arrêt 72
3.3 Implémentation de l'EM variationnel 73
3.3.1 Étapes E et M variationnelles 73
3.3.2 Effet de l'initialisation et des priors 74
3.4 Résultats expérimentaux et discussion 75
3.4.1 Synthèse consolidée de l'ensemble des résultats expérimentaux . 76
3.5 Recommandations pratiques 77
3.6 Conclusion du chapitre 3 78

Conclusion générale 79

Synthèse des contributions 80
Limites du travail 81
Perspectives 81

Bibliographie 83

Annexes 86

Annexe A : Code source 86
Annexe B : Détail du calcul de l'ELBO pour le GMM bayésien 98
Annexe C : Table de correspondance des symboles mathématiques 100
Annexe D : Environnement logiciel et reproductibilité 100
Annexe E : Rappels mathématiques utilisés dans les dérivations 101

Liste des figures

1.1 Évolution de la log-vraisemblance au fil des itérations (exemple numérique de la section 1.6.4). La courbe est croissante puis stationnaire dès la troisième itération, ce qui illustre la garantie de monotonie établie à la section 1.6.2.f. 18

2.1 Critères BIC et AIC de l'EM classique en fonction du nombre de composantes K, sur le jeu de données à trois groupes réels. 40
2.2 Jeu de données synthétique utilisé pour l'étude expérimentale : trois groupes gaussiens réels de 100 observations chacun (N = 300 au total). 43
2.3 Partition obtenue avec six composantes : une couleur par composante, ellipses de covariance à deux écarts-types, croix aux centres. À gauche, l'EM classique conserve ses six composantes ; à droite, l'EM variationnel n'en retient que trois (les composantes élaguées figurent en gris dans la légende). 45
2.4 Poids de mélange π_k finaux pour les six composantes : EM classique et EM variationnel. Le contraste traduit le mécanisme d'élagage bayésien décrit à la section 1.7.4. 45
2.5 Nombre de composantes actives en fonction de la concentration α_0 du prior de Dirichlet (échelle logarithmique), pour les deux jeux de priors. Le mécanisme d'élagage disparaît lorsque α_0 devient grand. 47
2.6 Temps d'exécution des deux algorithmes en fonction du nombre d'observations N (à gauche, échelles logarithmiques) et de la dimension D (à droite). Les étiquettes donnent le rapport du temps variationnel au temps classique. 48
2.7 Évolution de l'ELBO (EM variationnel, à gauche) et de la log-vraisemblance (EM classique, à droite) en fonction du nombre d'itérations. Les deux courbes sont croissantes, conformément aux garanties théoriques des sections 1.6.2.f et 1.7.6. 50
2.8 Répartition du nombre de composantes actives selon l'algorithme et la stratégie d'initialisation (20 exécutions chacune, une initialisation par exécution). 51
2.9 Indice de Rand ajusté selon la contrainte imposée sur la covariance (K = 6 imposé). 53
2.10 Indice de Rand ajusté et silhouette sur le jeu Iris, selon l'algorithme et le nombre de composantes. 54

2.11 Distribution de l'indice de Rand ajusté (à gauche, un point par jeu de données) et du nombre de composantes actives (à droite) sur 30 jeux de données indépendants. 57

3.1 Distribution des rendements quotidiens du S&P 500 sur la période d'apprentissage et densités ajustées (échelle logarithmique de la densité). La gaussienne unique sous-estime fortement les queues et le pic central ; les deux mélanges suivent les deux. 66

3.2 Choix du nombre de composantes : BIC et AIC de l'EM classique selon K (à gauche) et nombre effectif d'observations N_k de chaque composante de l'EM variationnel (à droite, échelle logarithmique). 67

3.3 Incertitude sur la VaR à 1 % : tirages a posteriori sous la loi variationnelle (histogramme bleu) et bootstrap non paramétrique de l'EM classique (histogramme orange). Les traits verticaux indiquent les VaR ponctuelles de chaque modèle et le quantile empirique de la période d'évaluation. . 69

3.4 Rendements quotidiens (haut) et probabilité a posteriori de l'état agité (bas) d'un modèle de Markov caché à deux états gaussiens, estimé sur la période d'apprentissage et appliqué à toute la période. 70

Liste des tableaux

1.1 Exemples fondamentaux de modèles à variables latentes 4
1.2 Convergence de l'algorithme EM sur l'exemple numérique à K = 2 composantes 17
1.3 Principales variantes de l'algorithme EM classique 18
1.4 Calcul détaillé des quantités forward, backward et a posteriori pour O = (NPa, Pa, Pa) 20
1.5 Résultat de l'EM variationnel sur l'exemple numérique à 8 observations (Kmax = 3) 28

2.1 Comparaison théorique entre l'EM classique et l'EM variationnel 35
2.2 Les trois niveaux d'inférence statistique pour les modèles à variables latentes 38
2.3 Complexité algorithmique comparée (par itération) 38
2.4 Paramètres du protocole expérimental 42
2.5 Poids de mélange obtenus par l'EM classique (K = 6) 43
2.6 Poids de mélange obtenus par l'EM variationnel (Kmax = 6) 44
2.7 Métriques de qualité de partition (K = 6 imposé) 46
2.8 Nombre de composantes actives ($N_k \geq 1$) et ARI en fonction de la concentration α_0 du prior de Dirichlet 47
2.9 Temps d'exécution (en secondes) en fonction de N et de D 48
2.10 Résultats de référence avec le nombre de composantes correctement spécifié (K = 3) 49
2.11 Nombre de composantes actives obtenu sur 20 initialisations indépendantes (une initialisation par exécution) 51
2.12 Indice de Rand ajusté et nombre de composantes actives selon le type de covariance (K = 6 imposé) 52
2.13 Résultats sur le jeu de données réel Iris (N = 150, D = 4) 54
2.14 Comportement des deux algorithmes en présence de 10 valeurs aberrantes ajoutées (N = 310, K = 6) 55
2.15 Résultats moyens sur 30 jeux de données indépendants (K = 6 imposé) 56
2.16 Différences appariées d'indice de Rand ajusté entre EM variationnel et EM classique 56
2.17 Synthèse comparative globale entre l'EM classique et l'EM variationnel 58

3.1 Topics extraits par LDA variationnel sur le corpus jouet (trois topics demandés) 62
3.2 Statistiques descriptives des rendements quotidiens du S&P 500 65
3.3 Log-vraisemblance moyenne par observation, en apprentissage et hors échantillon (n.d. : non défini) 67
3.4 VaR, Expected Shortfall et test de Kupiec sur la période d'évaluation (1 193 jours ; n.d. : non défini) 68
3.5 Synthèse des domaines d'application et de l'algorithme le plus adapté . 71
3.6 Effet de l'initialisation et des priors (jeu de référence, K = 6) 75
3.7 Synthèse consolidée de l'ensemble des expériences réalisées 76
3.8 Guide de décision pratique pour le choix entre EM classique et EM variationnel 77
3.9 Correspondance entre fonctions du code, sections et équations 86
3.10 Distribution topique complète des douze documents du corpus jouet . . 98

Liste des abréviations et sigles

EM Espérance-Maximisation (Expectation-Maximization)

GMM Modèle de Mélange Gaussien (Gaussian Mixture Model)

ELBO Borne Inférieure de l'Évidence (Evidence Lower Bound)

KL Divergence de Kullback-Leibler

VI Inférence Variationnelle (Variational Inference)

SVI Inférence Variationnelle Stochastique (Stochastic Variational Inference)

VAE Auto-encodeur Variationnel (Variational Autoencoder)

MAP Maximum A Posteriori

MLE Estimateur du Maximum de Vraisemblance (Maximum Likelihood Estimator)

HMM Modèle de Markov Caché (Hidden Markov Model)

PPCA Analyse en Composantes Principales Probabiliste

LDA Allocation de Dirichlet Latente (Latent Dirichlet Allocation)

BIC Critère d'Information Bayésien (Bayesian Information Criterion)

AIC Critère d'Information d'Akaike (Akaike Information Criterion)

ICL Critère de Vraisemblance Classifiante Intégrée

SEM EM Stochastique (Stochastic EM)

GEM EM Généralisé (Generalized EM)

MCAR Données Manquantes Complètement Aléatoires (Missing Completely At Random)

MAR Données Manquantes Aléatoires (Missing At Random)

NLP Traitement Automatique du Langage Naturel (Natural Language Processing)

Chapitre 1

FONDEMENTS THÉORIQUES ET ALGORITHMES EM

Ce premier chapitre pose l'ensemble des bases théoriques nécessaires à la compréhension du reste de ce mémoire, puis présente en détail les deux algorithmes qui en constituent l'objet central : l'algorithme Espérance-Maximisation (EM) classique et sa généralisation bayésienne, l'EM variationnel. Nous commençons par définir la notion de modèle à variables latentes, avant d'introduire les outils mathématiques indispensables la vraisemblance et sa maximisation, les distributions gaussiennes et le modèle de mélange gaussien, le cadre bayésien, l'inégalité de Jensen et la divergence de Kullback-Leibler. Ces éléments théoriques permettent ensuite de dérouler, de façon rigoureuse et progressive, la formulation mathématique complète de l'EM classique puis de l'EM variationnel, en insistant à chaque étape sur les propriétés de convergence, les variantes existantes, ainsi que les avantages et les limites de chacune des deux approches.

1.1 Modèles à variables latentes

1.1.1 Définition formelle

Un modèle probabiliste à variables latentes est défini par la donnée d'un espace d'observations et d'un espace de variables cachées, reliés par une loi jointe. Intuitivement, on peut se représenter une variable latente comme une cause non observée qui explique la structure des données observées : par exemple, la note obtenue par un étudiant à un examen (observée) dépend de son niveau réel de maîtrise de la matière (non observé directement). Formellement, pour une observation x_i, on écrit :

$$p(x_i|\theta) = \int_{\mathcal{Z}} p(x_i,z_i|\theta)dz_i \tag{1.1}$$

L'objectif de l'inférence est d'estimer le paramètre θ à partir des seules observations $\{x_i\}$, les variables latentes z_i n'étant jamais directement observées. La loi jointe se factorise selon le processus génératif du modèle :

$$p(X,Z|\theta) = p(Z|\theta) \times p(X|Z,\theta) = \prod_i p(z_i|\theta)p(x_i|z_i,\theta) \tag{1.2}$$

Cette factorisation traduit le processus génératif du modèle : on tire d'abord les variables latentes selon leur loi a priori, puis les observations conditionnellement à ces variables latentes.

1.1.2 Taxonomie des variables latentes

Les variables latentes se présentent sous différentes formes selon la nature du modèle considéré. On distingue en particulier :

- les variables latentes discrètes, comme les indicateurs d'appartenance à un groupe dans les modèles de mélange ;
- les variables latentes continues, comme les coordonnées dans un espace de représentation réduit, dans les modèles factoriels ;
- les variables latentes structurées, comme les séquences d'états cachés dans les modèles de Markov cachés.

La nature des variables latentes influence directement la traçabilité du calcul de la vraisemblance marginale et, par conséquent, la forme de l'algorithme EM applicable au modèle considéré.

1.1.3 Exemples fondamentaux

Les modèles à variables latentes sont omniprésents dans les applications de l'apprentissage automatique. Le tableau 1.1 présente les exemples les plus représentatifs, qui seront pour certains repris au chapitre 3 dans le cadre des applications pratiques.

Modèle	Variables latentes	Paramètres	Application
Mélange gaussien (GMM)	Appartenance à une composante $z_i \in \{1..K\}$	π, μ_k, Σ_k	Clustering, estimation de densité
Modèle de Markov caché (HMM)	Séquence d'états discrets	A, B, π_0	Reconnaissance vocale
ACP probabiliste (PPCA)	Coordonnées latentes $z_i \in \mathbb{R}^L$	W, σ^2	Réduction de dimension
Analyse factorielle	Facteurs latents	Λ, Ψ	Psychologie, économie
LDA	Topics par document, mots par topic	α, β	Modélisation de sujets textuels
VAE	Code latent $z_i \in \mathbb{R}^L$	θenc, θdec	Génération d'images
IRT	Niveau de compétence θ_i	a_j, b_j	Tests psychométriques

Table 1.1: Exemples fondamentaux de modèles à variables latentes

1.2 Vraisemblance et estimation

1.2.1 Vraisemblance complète et vraisemblance incomplète

La distinction entre vraisemblance complète et vraisemblance incomplète est au cœur de la compréhension de l'algorithme EM. La log-vraisemblance complète est définie sur l'espace des données complètes, c'est-à-dire en incluant les variables latentes :

$$\ell c(\theta) = \log p(X,Z|\theta) = \Sigma_i \log p(x_i,z_i|\theta) \tag{1.3}$$

Cette quantité est généralement facile à optimiser, car p(X, Z | θ) se factorise de manière simple, conduisant souvent à des expressions analytiques fermées pour les estimateurs du maximum de vraisemblance. En revanche, la log-vraisemblance incomplète, définie sur les seules observations, s'écrit :

$$\ell(\theta) = \log p(X|\theta) = \Sigma_i \log \int p(x_i,z_i|\theta) dz_i = \Sigma_i \log \Sigma_k p(x_i,z_i = k|\theta) \tag{1.4}$$

L'opération de logarithme d'une somme (ou d'une intégrale) détruit la structure factorielle favorable de la vraisemblance complète, rendant l'optimisation directe de $\ell(\theta)$ généralement intractable sur le plan analytique. C'est précisément cette difficulté que l'algorithme EM contourne de manière élégante, en s'appuyant sur une borne inférieure de la vraisemblance incomplète l'ELBO que nous introduisons dès à présent.

1.2.2 L'inégalité de Jensen et le principe de la borne inférieure

L'inégalité de Jensen est un résultat fondamental de l'analyse convexe qui sous-tend directement la construction de l'algorithme EM. Elle énonce que pour toute fonction concave f et toute variable aléatoire X, on a :

$$f(\mathbb{E}[X]) \geq \mathbb{E}[f(X)] \tag{1.5}$$

avec égalité si et seulement si X est constante presque sûrement, ou si f est linéaire. Appliquée à la fonction logarithme, qui est concave, avec q(Z) comme distribution de pondération, cette inégalité fournit, pour tout θ et toute distribution q(Z) :

$$\ell(\theta) = \log p(X|\theta) \geq \mathbb{E}_q[\log p(X,Z|\theta)] - \mathbb{E}_q[\log q(Z)] =: ELBO(q,\theta) \tag{1.6}$$

Cette borne inférieure, appelée ELBO (Evidence Lower BOund), est l'idée clé qui fonde l'algorithme EM sous ses deux variantes : au lieu d'optimiser directement la log-vraisemblance incomplète $\ell(\theta)$, on optimise alternativement cette borne inférieure par rapport à q (étape E) et par rapport à θ (étape M), ce qui garantit que la log-vraisemblance marginale ne diminue jamais au cours des itérations. L'égalité dans l'inégalité de Jensen est atteinte lorsque q(Z) = p(Z | X, θ), c'est-à-dire lorsque la distribution variationnelle coïncide exactement avec la distribution a posteriori des variables latentes : c'est exactement ce que réalise l'étape E de l'algorithme EM classique.

1.2.3 Identifiabilité et unicité

Une question importante en théorie de l'estimation concerne l'identifiabilité du modèle : peut-on distinguer deux valeurs de paramètres différentes $\theta \neq \theta'$ uniquement sur la base de la distribution des observations ? Un modèle est identifiable si p(X | θ) = p(X | θ') implique $\theta = \theta'$. Les modèles de mélanges souffrent typiquement d'une non-identifiabilité partielle : les K! permutations des étiquettes de composantes donnent toutes la même distribution marginale, créant ainsi K! optima globaux équivalents. Cette propriété, appelée label switching, n'affecte pas la qualité de la solution trouvée par l'EM, mais complique l'interprétation et la comparaison des résultats entre plusieurs lancements de l'algorithme.

1.3 Modèles de mélanges gaussiens (GMM)

1.3.1 La distribution gaussienne univariée

La distribution gaussienne (ou normale) $\mathcal{N}(\mu, \sigma^2)$ est une distribution de probabilité continue définie sur $\mathbb{R}$, entièrement caractérisée par sa moyenne $\mu \in \mathbb{R}$ et sa variance

$\sigma^2 > 0$. Sa densité de probabilité est donnée par :

$$p(x|\mu,\sigma^2) = 1/\sqrt{2\pi\sigma^2} \cdot \exp[-(x-\mu)^2/(2\sigma^2)] \tag{1.7}$$

La gaussienne jouit de plusieurs propriétés remarquables qui expliquent sa place centrale en statistiques : elle est stable par combinaison linéaire (si X $\sim \mathcal{N}(\mu, \sigma^2)$, alors aX + b $\sim \mathcal{N}(a\mu+$b, $a^2\sigma^2)$) ; elle est la loi limite des sommes normalisées de variables indépendantes, conformément au théorème central limite ; elle maximise l'entropie parmi toutes les distributions de variance fixée ; enfin, elle est sa propre distribution a priori conjuguée pour l'estimation de la moyenne.

1.3.2 La distribution gaussienne multivariée

En dimension D, la distribution gaussienne multivariée $\mathcal{N}(\mu, \Sigma)$ est paramétrée par un vecteur moyenne $\mu \in \mathbb{R}^D$ et une matrice de covariance $\Sigma \in \mathbb{R}^{DxD}$, symétrique définie positive. Sa densité s'écrit :

$$p(x|\mu,\Sigma) = (2\pi)^{-D/2}|\Sigma|^{-1/2} \exp[-\tfrac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)] \tag{1.8}$$

La matrice de covariance Σ encode les variances individuelles sur sa diagonale et les corrélations entre dimensions hors de la diagonale. Plusieurs cas particuliers sont couramment utilisés : la gaussienne sphérique ($\Sigma = \sigma^2 I$), la gaussienne diagonale ($\Sigma =$ diag($\sigma_1^2, \ldots, \sigma^{D2}$)), et la gaussienne générale (Σ quelconque), ces trois cas correspondant à des niveaux de complexité croissants dans les modèles de mélanges gaussiens.

1.3.3 Le modèle de mélange gaussien

Un mélange de K gaussiennes est le modèle probabiliste central de ce mémoire. Il suppose que chaque observation x_i est générée selon le processus suivant : on tire d'abord un indicateur $z_i \in \{1, \ldots, \text{K}\}$ selon une distribution catégorielle de paramètres ($\pi_1, \ldots, \pi K$), puis on tire x_i selon la gaussienne de la composante sélectionnée. La densité marginale du mélange s'écrit :

$$p(x|\theta) = \Sigma_{k=1}^{K} \pi_k \mathcal{N}(x|\mu_k,\Sigma_k) \tag{1.9}$$

où les poids de mélange π_k satisfont $\pi_k \geq 0$ et $\Sigma_k\ \pi_k = 1$, et où $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$ désigne l'ensemble des paramètres du modèle. Le GMM est un approximateur universel de densités continues : pour K suffisamment grand, un GMM peut approcher arbitrairement bien n'importe quelle densité de probabilité continue, ce qui explique son utilisation massive comme modèle de densité générique. En tant que méthode de clustering, le GMM offre des partitions souples (soft clustering), où chaque point appartient

simultanément à plusieurs clusters avec des probabilités différentes, contrairement au clustering dur (hard clustering) réalisé par l'algorithme k-means.

1.3.4 Relation entre GMM et k-means

Le célèbre algorithme k-means peut être vu comme un cas limite dégénéré du GMM. Lorsque toutes les matrices de covariance sont contraintes à $\Sigma_k = \sigma^2 I$ et que $\sigma^2 \to 0$, les responsabilités de l'EM convergent vers des affectations dures : chaque point est assigné entièrement à la composante dont le centre est le plus proche, et les mises à jour des centres correspondent exactement aux mises à jour des centroïdes de k-means. Cette connexion illustre que l'EM pour GMM est une généralisation probabiliste de k-means, offrant davantage de flexibilité et de richesse modélisatrice, au prix d'un coût computationnel légèrement supérieur.

1.3.5 Exemple numérique : évaluation de la densité d'un GMM

Afin de rendre pleinement concrète l'équation (1.8) avant son utilisation intensive dans la suite de ce chapitre, évaluons-la numériquement en reprenant le modèle à deux composantes convergé à la section 1.6.4 (π_1=π_2=0,5 ; μ_1=1,125, σ_1=0,466 ; μ_2=7,725, σ_2=0,460), et calculons la densité prédite p(x|θ) pour une nouvelle observation x = 2,0, non présente dans l'échantillon original.

$$p(2{,}0|\theta) = 0{,}5 \times \mathcal{N}(2{,}0|1{,}125; 0{,}466^2) + 0{,}5 \times \mathcal{N}(2{,}0|7{,}725; 0{,}460^2) \tag{1.10}$$

Le premier terme, associé à la composante centrée en 1,125, vaut $0{,}5 \times 0{,}1468 \approx 0{,}0734$, la valeur x = 2,0 n'étant qu'à 1,9 écart-type environ du centre de cette composante. Le second terme, associé à la composante centrée en 7,725, vaut quant à lui environ 10^{-34} : la valeur x = 2,0 se trouve à plus de douze écarts-types du centre de cette seconde composante, ce qui rend sa contribution totalement négligeable en comparaison de la première, un exemple numérique direct du phénomène d'underflow évoqué à la section 1.6.2.e, qui justifie le recours systématique au calcul en log-espace. La densité totale vaut donc p(2,0|θ) $\approx$ 0,0734, et la responsabilité de la première composante pour cette observation, calculée par l'équation (1.15), vaut $r_1 = 0{,}0734/0{,}0734 \approx 1{,}000$: le modèle attribue, sans la moindre ambiguïté, l'observation x = 2,0 à la composante centrée en 1,125, ce qui correspond parfaitement à l'intuition géométrique.

1.4 Introduction au cadre bayésien

1.4.1 La philosophie bayésienne

Le cadre bayésien repose sur une vision fondamentalement différente de celle des statistiques fréquentistes classiques. Là où le fréquentiste considère les paramètres comme des constantes inconnues et définit la probabilité comme une fréquence limite d'événements répétables, le bayésien traite les paramètres comme des variables aléatoires et interprète la probabilité comme un degré de croyance subjectif, pouvant s'appliquer à tout événement incertain, qu'il soit répétable ou non.

Cette différence philosophique a des conséquences pratiques importantes : un bayésien peut exprimer ses connaissances préalables sur les paramètres sous forme d'une distribution a priori, peut quantifier son incertitude résiduelle après observation des données sous forme d'une distribution a posteriori, et peut naturellement intégrer de nouvelles données en utilisant la distribution a posteriori courante comme nouveau prior. C'est cette dernière propriété qui rend le cadre bayésien particulièrement adapté à l'EM variationnel, présenté à la section 1.7.

1.4.2 Le théorème de Bayes et ses composantes

Le théorème de Bayes est le pivot de toute l'inférence bayésienne. Pour un modèle avec paramètres θ et observations X, il s'écrit :

$$p(\theta|X) = p(X|\theta)p(\theta)/p(X) \tag{1.11}$$

Chaque terme de cette équation joue un rôle précis : la distribution a priori p(θ) encode les connaissances préalables sur θ, avant toute observation ; la vraisemblance p(X | θ) mesure la compatibilité des données observées avec chaque valeur possible de θ ; la distribution a posteriori p(θ | X) représente les croyances mises à jour sur θ après observation de X ; enfin, la vraisemblance marginale, ou évidence, p(X) = $\int$ p(X | θ) p(θ) $d\theta$, est une constante de normalisation indépendante de θ, mais qui joue un rôle central dans la sélection de modèles bayésiens.

En pratique, pour un GMM bayésien, on place des distributions a priori conjuguées sur les paramètres : les poids de mélange reçoivent un prior de Dirichlet, $\pi \sim \mathrm{Dir}(\alpha_0, \ldots, \alpha_0)$, et les paramètres des composantes reçoivent un prior Normal-Wishart, $(\mu_k, \Lambda_k) \sim \mathcal{NW}(m_0, \beta_0, W_0, \nu_0)$, où $\Lambda_k = \Sigma_k^{-1}$ désigne la matrice de précision. Ces priors conjugués garantissent que les distributions a posteriori appartiennent à la même famille que les priors, ce qui permet des mises à jour analytiques lors des étapes variationnelles présentées à la section 1.7.

1.4.3 Distributions conjuguées

Le concept de conjugaison est fondamental en inférence bayésienne. On dit qu'un prior $p(\theta)$ est conjugué à la vraisemblance $p(X \mid \theta)$ si la distribution a posteriori $p(\theta \mid X)$ appartient à la même famille paramétrique que le prior. La conjugaison garantit des formes analytiques fermées pour les mises à jour bayésiennes, ce qui est particulièrement précieux pour l'EM variationnel. Les couples conjugués les plus importants pour notre étude sont : Gaussienne–Gaussienne (pour la moyenne d'une gaussienne de variance connue), Dirichlet–Catégorielle (pour les paramètres d'une distribution catégorielle), et Normal-Wishart–Gaussienne multivariée (pour les paramètres complets d'une gaussienne multivariée).

1.4.4 Exemple numérique illustratif : conjugaison Dirichlet–Catégorielle

Afin d'ancrer concrètement la notion de conjugaison avant son utilisation intensive à la section 1.7, illustrons-la sur le cas le plus simple rencontré dans ce mémoire : la mise à jour d'un prior de Dirichlet sur les poids d'un mélange à partir d'observations catégorielles. Considérons un mélange à $K = 3$ composantes, muni d'un prior a priori peu informatif $\mathrm{Dir}(\alpha_0, \alpha_0, \alpha_0)$ avec $\alpha_0 = 1$ (ce qui correspond à une distribution uniforme sur le simplexe des poids π). Supposons que l'étape E d'une itération donnée attribue, sur un petit jeu de $N = 12$ observations, les responsabilités suivantes, cumulées par composante : $N_1 = 7$, $N_2 = 4$, $N_3 = 1$ (soit $\Sigma_k N_k = 12$, conformément à la contrainte $\Sigma_k r_{ik} = 1$ pour chaque observation).

La règle de mise à jour bayésienne pour le couple conjugué Dirichlet–Catégorielle, appliquée à la section 1.7.3.c, donne directement le paramètre a posteriori $\alpha_k = \alpha_0 + N_k$, soit ici :

$$\alpha_1 = 1 + 7 = 8, \alpha_2 = 1 + 4 = 5, \alpha_3 = 1 + 1 = 2 \tag{1.12}$$

La distribution a posteriori sur les poids de mélange est donc $\mathrm{Dir}(8, 5, 2)$, dont la moyenne l'estimation ponctuelle bayésienne des poids vaut $\mathbb{E}[\pi_k] = \alpha_k \,/\, \Sigma_j \alpha_j$, soit $\mathbb{E}[\pi_1] = 8/15 \approx 0{,}533$, $\mathbb{E}[\pi_2] = 5/15 \approx 0{,}333$ et $\mathbb{E}[\pi_3] = 2/15 \approx 0{,}133$. On notera que ces valeurs moyennes sont très proches, mais non identiques, des fréquences empiriques $N_k/N = 7/12 \approx 0{,}583$; $4/12 \approx 0{,}333$; $1/12 \approx 0{,}083$ qu'aurait directement utilisées l'EM classique à l'équation (1.16) : l'écart entre les deux, plus marqué pour la composante la moins représentée (0,133 contre 0,083 pour π_3), illustre précisément l'effet de régularisation du prior décrit à la section 1.7.8 le prior « tire » les estimations vers l'uniformité a priori (1/3 pour chaque composante), d'autant plus fortement que le nombre d'observations disponibles pour cette composante est faible. Cet exemple,

volontairement simplifié à une seule mise à jour, préfigure directement le mécanisme itératif complet de l'EM variationnel détaillé à la section 1.7.3, dans lequel cette même règle de mise à jour est appliquée de façon répétée, conjointement à la mise à jour des paramètres Normal-Wishart, jusqu'à convergence de l'ELBO.

1.5 La divergence de Kullback-Leibler

1.5.1 Définition et propriétés

La divergence de Kullback-Leibler (KL) entre deux distributions de probabilité q et p est définie par :

$$KL[q\|p] = \mathbb{E}_q[\log q(z) - \log p(z)] = \int q(z) \log[q(z)/p(z)]dz \tag{1.13}$$

Cette divergence possède plusieurs propriétés importantes : elle est toujours non négative, KL[q ‖ p] ≥ 0, avec égalité si et seulement si q = p presque partout, ce résultat étant connu sous le nom d'inégalité de Gibbs ; elle est asymétrique, c'est-à-dire que KL[q ‖ p] ≠ KL[p ‖ q] en général ; elle n'est pas une distance au sens mathématique du terme, car elle ne satisfait pas l'inégalité triangulaire ; enfin, elle est étroitement liée à l'information de Fisher et joue un rôle central en théorie de l'information.

1.5.2 Interprétation informationnelle

La divergence KL[q ‖ p] peut être interprétée comme le coût moyen, exprimé en nats (ou en bits si le logarithme est en base 2), d'encoder des données générées par q en utilisant un code optimal conçu pour p. Plus q et p sont similaires, plus ce coût est faible. Dans le contexte de l'inférence variationnelle, minimiser KL[q ‖ p], où p = p(θ, Z | X) désigne la vraie distribution a posteriori, revient à rechercher, au sein d'une famille variationnelle donnée, la distribution q la plus proche possible de la véritable distribution a posteriori.

1.5.3 KL et ELBO : la décomposition fondamentale

La relation entre la divergence KL, l'ELBO et la log-vraisemblance marginale est établie par la décomposition fondamentale suivante :

$$\log p(X) = ELBO(q) + KL[q(Z,\theta)\|p(Z,\theta|X)] \tag{1.14}$$

Cette décomposition est fondamentale, car elle montre que maximiser l'ELBO par rapport à q est équivalent à minimiser KL[q ‖ p(· | X)], puisque log p(X) est une constante par rapport à q. Elle montre également que l'ELBO sous-estime toujours la

log-vraisemblance marginale, puisque la divergence KL est non négative, et que l'écart entre les deux est exactement la divergence KL entre l'approximation variationnelle et la vraie distribution a posteriori. Cette relation constitue le socle mathématique commun sur lequel reposent à la fois l'EM classique, présenté à la section 1.6, et l'EM variationnel, présenté à la section 1.7.

1.5.4 KL directe et KL inverse : le phénomène de zero-forcing

Un point souvent source de confusion, mais dont la compréhension éclaire directement l'une des limites de l'EM variationnel signalées à la section 1.7.9, concerne le choix particulier de la divergence KL minimisée par l'inférence variationnelle. La décomposition de la section précédente minimise KL[q ‖ p], c'est-à-dire la divergence dite « inverse » (reverse KL), et non KL[p ‖ q], la divergence « directe » (forward KL) qui serait utilisée, par exemple, dans l'estimation du maximum de vraisemblance d'un modèle paramétrique ajusté à des données. Ce choix n'est pas arbitraire : il est dicté par la tractabilité, puisque KL[q ‖ p] = $\mathbb{E}$_q[log q - log p] ne requiert que des espérances sous q, la distribution que l'on contrôle, alors que KL[p ‖ q] nécessiterait des espérances sous la véritable distribution a posteriori p, précisément celle que l'on cherche à approximer et que l'on ne sait pas calculer.

Ce choix a toutefois une conséquence importante sur la forme de l'approximation obtenue. La divergence KL inverse pénalise très fortement les régions où $q(z) > 0$ mais $p(z) \approx 0$ (le terme $\log[q(z)/p(z)]$ y devient très grand), alors qu'elle tolère au contraire les régions où $q(z) \approx 0$ mais $p(z) > 0$. En conséquence, lorsque la véritable distribution a posteriori p est multimodale, la distribution variationnelle q typiquement unimodale sous l'hypothèse du champ moyen tend à se concentrer sur un seul des modes de p plutôt que d'étaler sa masse pour couvrir l'ensemble des modes, phénomène connu sous le nom de zero-forcing ou de mode-seeking behaviour. C'est ce même phénomène qui explique la sous-estimation systématique de la variance a posteriori évoquée à la section 1.7.9 comme premier inconvénient de l'EM variationnel : en se concentrant sur un mode plutôt que d'épouser fidèlement l'étalement complet de la véritable distribution a posteriori, l'approximation variationnelle produit mécaniquement des intervalles de crédibilité plus étroits que ceux qu'une caractérisation exacte de $p(\theta|X)$ aurait fournis un biais optimiste sur la confiance accordée aux paramètres estimés, dont le praticien doit avoir conscience lorsqu'il interprète les distributions a posteriori produites par l'EM variationnel.

1.6 L'algorithme EM classique

1.6.1 Historique et contexte

L'algorithme Espérance-Maximisation a une histoire riche qui précède sa formalisation par Dempster, Laird et Rubin. Des idées similaires apparaissent dans les travaux de Hartley (1958) sur les tableaux de contingence, de Baum et al. (1970) dans le contexte des modèles de Markov cachés l'algorithme de Baum-Welch , et d'Orchard et Woodbury (1972) sur le traitement des données manquantes. C'est toutefois l'article de 1977 de Dempster, Laird et Rubin qui a unifié ces approches disparates sous un cadre théorique commun, démontré les propriétés générales de convergence, et fourni le langage étapes E et M devenu standard.

L'article original de Dempster, Laird et Rubin est remarquable par sa généralité : il ne se restreint pas à une classe particulière de modèles, mais formule l'algorithme dans le cadre général de la famille exponentielle, couvrant ainsi une large classe de modèles statistiques courants. Depuis 1977, l'EM a fait l'objet de milliers de publications, donnant lieu à de nombreuses extensions EM stochastique, EM online, EM généralisé, entre autres et à des applications dans presque tous les domaines des sciences des données.

Il est intéressant de noter que la démonstration originale de convergence proposée par Dempster, Laird et Rubin en 1977 comportait une lacune technique, corrigée seulement six ans plus tard par Wu (1983), qui établit les conditions précises sous lesquelles la suite des paramètres $\{\theta^t\}$ converge effectivement vers un point stationnaire de la vraisemblance, et non simplement la suite des valeurs de vraisemblance $\{\ell(\theta^t)\}$ une distinction subtile mais importante, puisqu'une suite de valeurs de vraisemblance convergente n'implique pas nécessairement, en toute généralité mathématique, la convergence de la suite des paramètres eux-mêmes. Cet épisode illustre un trait caractéristique du développement de l'algorithme EM : la simplicité de son principe opérationnel alterner deux étapes de calcul relativement élémentaires a longtemps contrasté avec la subtilité des conditions mathématiques précises garantissant son bon comportement, subtilité qui n'a été pleinement clarifiée que par les travaux théoriques ultérieurs de Wu (1983) et de Boyles (1983), puis, pour le versant bayésien de l'algorithme qui nous occupe à la section 1.7 de ce chapitre, par les travaux fondateurs de Jordan et al. (1999) évoqués à la section 1.7.1.

1.6.2 Formulation mathématique complète

a) L'algorithme en pseudocode

L'algorithme EM classique peut être formulé de manière concise. Il prend en entrée un jeu de données X = {x_1, ..., x_N}, un nombre de composantes K et un critère

d'arrêt ε, et retourne les paramètres estimés $\hat{\theta}$. Les grandes étapes en sont les suivantes : initialisation aléatoire ou par k-means des paramètres $\theta^0 = \{\pi_k^0, \mu_k^0, \Sigma_k^0\}_{k=1}^K$; puis, à chaque itération t, calcul des responsabilités à l'étape E, mise à jour des paramètres à l'étape M, et évaluation de la log-vraisemblance jusqu'à convergence, soit $|\ell(\theta^t) - \ell(\theta^{t-1})| < \varepsilon$, soit atteinte d'un nombre maximal d'itérations.

b) L'étape E en détail

L'étape E calcule la distribution conditionnelle q(Z) = p(Z | X, θ^t) des variables latentes sachant les observations et les paramètres courants. Cette distribution est représentée par les responsabilités r_{ik}, qui constituent les probabilités a posteriori d'appartenance :

$$r_{ik} = p(z_i = k|x_i,\theta^t) = \pi_k^t \mathcal{N}(x_i|\mu_k^t,\Sigma_k^t)/\Sigma_j \pi_j^t \mathcal{N}(x_i|\mu_j^t,\Sigma_j^t) \tag{1.15}$$

Ces responsabilités satisfont $r_{ik} \geq 0$ et $\Sigma_k\ r_{ik} = 1$ pour tout i. Elles peuvent être interprétées comme des affectations souples : plutôt que d'assigner chaque point à une seule composante, on lui attribue un poids continu pour chaque composante. Cette représentation souple capture naturellement l'incertitude sur l'appartenance dans les régions de chevauchement entre composantes.

Le calcul des responsabilités peut poser des problèmes numériques lorsque les densités gaussiennes sont très petites, ce qui conduit à un phénomène d'underflow. En pratique, on travaille en log-espace et on utilise l'astuce dite du log-sum-exp pour stabiliser les calculs.

c) L'étape M en détail

L'étape M maximise la fonction Q(θ | θ^t) = $\mathbb{E}_\{$p(Z|X,θ^t)$\}$[log p(X, Z | θ)] par rapport à θ. Pour le GMM, cette maximisation admet des solutions analytiques fermées. En différentiant Q par rapport à π_k, μ_k et Σ_k, et en annulant les gradients la contrainte $\Sigma_k\ \pi_k = 1$ étant traitée par un multiplicateur de Lagrange , on obtient les mises à jour :

$$\pi_k = N_k/N, \mu_k = (1/N_k)\Sigma_i r_{ik} x_i, \Sigma_k = (1/N_k)\Sigma_i r_{ik}(x_i - \mu_k)(x_i - \mu_k)^T \tag{1.16}$$

où $N_k = \Sigma_i\ r_{ik}$ désigne le nombre effectif de points assignés à la composante k une quantité réelle, pas nécessairement entière. Ces formules ont une interprétation intuitive claire : la nouvelle moyenne μ_k est la moyenne pondérée des données, avec des poids proportionnels aux responsabilités ; la nouvelle covariance Σ_k est la covariance empirique pondérée autour de cette nouvelle moyenne ; et le nouveau poids π_k est la

fraction effective de données assignées à la composante k. La dérivation complète de ces trois formules, par annulation des gradients de la fonction Q, est détaillée ci-après, dans un souci de complétude mathématique.

d) Dérivation complète des mises à jour de l'étape M

Cette section détaille la dérivation des formules de mise à jour de l'étape M annoncées ci-dessus, à partir de l'expression explicite de la fonction Q pour le modèle de mélange gaussien. En développant la log-vraisemblance complète pour le GMM, on obtient :

$$Q(\theta|\theta^t) = \Sigma_i \Sigma_k r_{ik} [\log \pi_k + \log \mathcal{N}(x_i|\mu_k,\Sigma_k)] \tag{1.17}$$

Mise à jour de π_k. Le terme de Q qui dépend de π s'écrit $\Sigma_i\ \Sigma_k\ r_{ik}\ \log\ \pi_k$, sous la contrainte $\Sigma_k\ \pi_k = 1$. On introduit le lagrangien $L(\pi,\ \lambda) = \Sigma_i\ \Sigma_k\ r_{ik}\ log\ \pi_k\ +\ \lambda(1\ -\ \Sigma_k\ \pi_k)$. En annulant la dérivée partielle par rapport à π_k, on obtient $\partial L/\partial\pi_k = \Sigma_i\ r_{ik}\ /\ \pi_k\ -\ \lambda = 0$, soit $\pi_k = N_k/\lambda$, où $N_k = \Sigma_i\ r_{ik}$. En sommant cette relation sur k et en utilisant la contrainte $\Sigma_k\ \pi_k = 1$, il vient $\Sigma_k\ N_k/\lambda = 1$, soit $\lambda = \Sigma_k\ N_k = \mathrm{N}$ (le nombre total d'observations, puisque $\Sigma_k\ r_{ik} = 1$ pour chaque i). On en déduit immédiatement $\pi_k = N_k/\mathrm{N}$, ce qui établit la première formule de mise à jour.

Mise à jour de μ_k. Le terme de Q qui dépend de μ_k s'écrit, à une constante additive près, $-\frac{1}{2}\ \Sigma_i\ r_{ik}\ (x_i-\mu_k)^T\ \Sigma_k^{-1}\ (x_i-\mu_k)$. En annulant le gradient par rapport à μ_k, on obtient $\partial Q/\partial\mu_k = \Sigma_i\ r_{ik}\ \Sigma_k^{-1}\ (x_i-\mu_k) = 0$. Comme Σ_k^{-1} est inversible, cette égalité équivaut à $\Sigma_i\ r_{ik}\ (x_i-\mu_k) = 0$, c'est-à-dire $\Sigma_i\ r_{ik}\ x_i = \mu_k\ \Sigma_i\ r_{ik} = \mu_k\ N_k$, d'où l'on tire directement $\mu_k = (1/N_k)\ \Sigma_i\ r_{ik}\ x_i$, ce qui établit la deuxième formule de mise à jour : la nouvelle moyenne est la moyenne empirique pondérée par les responsabilités.

Mise à jour de Σ_k. En substituant la nouvelle valeur de μ_k et en différentiant Q par rapport à Σ_k^{-1} (il est d'usage, pour cette dérivation, de différentier par rapport à la matrice de précision plutôt qu'à la covariance elle-même, ce qui simplifie les calculs matriciels), on utilise les deux identités classiques du calcul matriciel $\partial/\partial A\ \log|\mathrm{A}| = (A^T)^{-1}$ et $\partial/\partial A\ (x^TAx) = xx^T$. On obtient alors $\partial Q/\partial\Sigma_k^{-1} = \frac{1}{2}\ \Sigma_i\ r_{ik}\ [\Sigma_k\ -\ (x_i-\mu_k)(x_i-\mu_k)^T] = 0$, ce qui conduit à $\Sigma_k\ \Sigma_i\ r_{ik} = \Sigma_i\ r_{ik}\ (x_i-\mu_k)(x_i-\mu_k)^T$, soit finalement $\Sigma_k = (1/N_k)\ \Sigma_i\ r_{ik}\ (x_i-\mu_k)(x_i-\mu_k)^T$, ce qui établit la troisième et dernière formule de mise à jour.

Cette dérivation complète met en évidence un point important : chacune des trois mises à jour de l'étape M résulte d'une simple annulation de gradient sur une fonction Q qui, conditionnellement aux responsabilités r_{ik} fixées à l'étape E, est concave séparément en chacun des trois blocs de paramètres (π, μ, Σ). C'est précisément cette séparabilité qui rend l'étape M analytiquement soluble pour le GMM, alors que la log-vraisemblance incomplète $\ell(\theta)$, elle, ne l'est pas.

e) L'astuce du log-sum-exp

La stabilisation numérique du calcul des responsabilités, mentionnée à la section précédente, mérite d'être détaillée, car elle est systématiquement employée dans les implémentations pratiques de l'EM, qu'il s'agisse de l'EM classique ou de l'EM variationnel. Le problème se pose ainsi : on souhaite calculer $\log \Sigma_k \exp(a_k)$, où les $a_k = \log \pi_k + \log \mathcal{N}(x_i \mid \mu_k, \Sigma_k)$ peuvent être des nombres très négatifs (par exemple -500 ou moins), de sorte que $\exp(a_k)$ est numériquement indistinguable de zéro en précision flottante standard, ce qui provoquerait une division par zéro lors du calcul des responsabilités.

L'astuce consiste à factoriser par le maximum des a_k avant d'exponentier :

$$\log \Sigma_k \exp(a_k) = a_{max} + \log \Sigma_k \exp(a_k - a_{max}), \text{ où } a_{max} = \max_k a_k \tag{1.18}$$

Cette identité est mathématiquement triviale il suffit de factoriser exp(a_max) dans la somme puis de repasser au logarithme , mais elle change radicalement le comportement numérique du calcul : après soustraction de a_max, le terme correspondant au maximum vaut exp(0) = 1, et tous les autres termes valent exp(a_k - a_max) $\leq$ 1, ce qui élimine tout risque de dépassement de capacité (overflow) ou de perte de précision par underflow. C'est cette identité qui est utilisée, sous une forme ou une autre, dans les deux implémentations from scratch présentées au chapitre 3 (sections 3.2.1 et 3.3.1).

f) La fonction Q et la garantie de croissance

La propriété de croissance monotone de l'EM repose sur la décomposition $\ell(\theta)$ = Q(θ | θ^t) - $\mathbb{E}$_{p(Z|X,θ^t)}[log p(Z | X, θ)] + constante. Comme l'étape M assure Q(θ^{t+1} | θ^t) $\geq$ Q(θ^t | θ^t), et que le terme soustractif ne diminue pas quand θ passe de θ^t à θ^{t+1} par la non-négativité de la divergence KL établie à la section 1.5 , on conclut que $\ell(\theta^{t+1})$ $\geq$ $\ell(\theta^t)$: la log-vraisemblance ne peut donc que croître ou rester constante à chaque itération de l'algorithme.

1.6.3 Propriétés théoriques

a) Convergence : théorème principal

Le théorème de convergence de l'EM, établi par Dempster et al. (1977) et renforcé par Wu (1983), stipule que, sous des conditions de régularité standard (modèle lisse, espace des paramètres compact), la suite $\{\theta^t\}$ générée par l'algorithme EM satisfait les propriétés suivantes : la log-vraisemblance $\{\ell(\theta^t)\}$ est une suite croissante et bornée, donc convergente ; tout point d'accumulation de la suite $\{\theta^t\}$ est un point stationnaire de $\ell(\theta)$; la suite $\{\theta^t\}$ converge vers un point stationnaire si ℓ est unimodale, ou si la

suite elle-même est convergente.

Il est important de noter que l'EM n'est pas garanti de converger vers le maximum global de la vraisemblance. Dans les modèles de mélanges, la surface de vraisemblance est typiquement multimodale, et l'algorithme converge vers le maximum local le plus proche du point de départ, ce qui justifie la pratique courante consistant à relancer l'algorithme depuis plusieurs initialisations différentes.

b) Vitesse de convergence linéaire

La vitesse de convergence locale de l'algorithme EM est linéaire, avec un taux de convergence donné par la valeur propre maximale de la matrice d'information manquante :

$$r = \lambda max(I_{comp}^{-1} I_{man}) \in [0,1) \tag{1.19}$$

où I_comp désigne la matrice d'information complète (information de Fisher qu'on aurait si les variables latentes étaient observées) et I_man la matrice d'information manquante, de sorte que I_obs = I_comp - I_man. La matrice I_comp^{-1} I_man = I - I_comp^{-1} I_obs est la matrice de taux établie par Dempster, Laird et Rubin (1977, section 3). Plus r est proche de 1 c'est-à-dire lorsque la proportion d'information manquante est élevée , plus la convergence est lente ; plus r est proche de 0, plus la convergence est rapide. Ce résultat explique pourquoi l'EM peut se révéler très lent dans certains problèmes comportant un grand nombre de variables latentes ou de fortes ambiguïtés d'appartenance.

c) Erreurs standard et information de Fisher

Bien que l'EM classique ne fournisse pas directement d'estimations d'incertitude, des intervalles de confiance asymptotiques peuvent être obtenus a posteriori en utilisant la matrice d'information de Fisher observée, calculée à la solution $\hat{\theta}$. La formule de Louis (1982) permet de calculer cette matrice d'information observée à partir des quantités calculées lors des itérations EM, selon I_obs($\hat{\theta}$) = I_comp($\hat{\theta}$) - I_man($\hat{\theta}$), où I_comp désigne la matrice d'information complète. Cette formule, bien que théoriquement fondée, est rarement utilisée en pratique en raison de sa complexité computationnelle, ce qui constitue l'une des limitations pratiques de l'EM classique pour la quantification de l'incertitude limitation que l'EM variationnel permet précisément de lever, comme nous le verrons à la section 1.7.

1.6.4 Application au mélange gaussien : exemple numérique détaillé

Afin d'illustrer concrètement le fonctionnement de l'algorithme EM classique, considérons N = 8 observations scalaires : X = {0,5 ; 1,0 ; 1,2 ; 1,8 ; 7,1 ; 7,5 ; 8,0 ; 8,3}. Nous supposons K = 2 composantes gaussiennes, avec pour initialisation $\mu_1^0 = 2$, $\mu_2^0 = 6$, $\sigma_1^0 = \sigma_2^0 = 2$, $\pi_1^0 = \pi_2^0 = 0{,}5$. Le tableau 1.2 retrace l'évolution des paramètres au fil des itérations ; la log-vraisemblance indiquée est celle des paramètres obtenus à l'issue de l'étape M de l'itération considérée.

Itération	μ_1	μ_2	σ_1	σ_2	π_1	π_2	**log-vrais.**
0 (init)	2,000	6,000	2,000	2,000	0,500	0,500	non définie
1	1,284	7,368	1,134	1,549	0,484	0,516	-15,814
2	1,125	7,722	0,466	0,479	0,500	0,500	-10,742
3 (conv.)	1,125	7,725	0,466	0,460	0,500	0,500	-10,736

Table 1.2: Convergence de l'algorithme EM sur l'exemple numérique à K = 2 composantes

La convergence est très rapide : le point fixe est atteint en trois itérations, la variation de la log-vraisemblance devenant inférieure à 10^{-3} dès l'itération suivante. La log-vraisemblance croît de -15,814 à -10,736 sans jamais diminuer, conformément à la garantie théorique établie précédemment. La solution finale identifie correctement les deux groupes : le premier est centré sur 1,125 (moyenne empirique des observations 0,5 ; 1,0 ; 1,2 et 1,8) avec un écart-type de 0,466, le second sur 7,725 (moyenne empirique de 7,1 ; 7,5 ; 8,0 et 8,3) avec un écart-type de 0,460. Les deux groupes étant très éloignés l'un de l'autre, les responsabilités valent 1 ou 0 à la précision de la machine et les paramètres estimés coïncident avec les statistiques empiriques de chaque groupe.

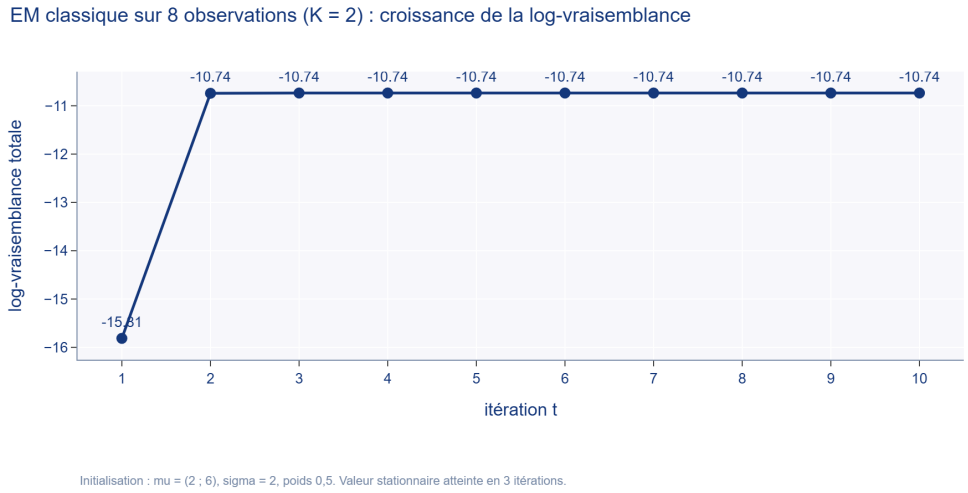


Figure 1.1: Évolution de la log-vraisemblance au fil des itérations (exemple numérique de la section 1.6.4). La courbe est croissante puis stationnaire dès la troisième itération, ce qui illustre la garantie de monotonie établie à la section 1.6.2.f.

1.6.5 Variantes de l'EM classique

Variante	**Modification**	**Avantage**	**Inconvénient**
EM standard	Aucune	Simple, convergence garantie	Lent sur grandes données
EM stochastique (SEM)	Étape E : tirage aléatoire	Évite certains optima locaux	Pas de monotonie
EM online	Mise à jour incrémentale	Scalable	Choix du taux d'apprentissage
EM généralisé (GEM)	Étape M approchée	Flexible	Convergence plus lente
EM accéléré (ECME)	Étape CM-E alternée	Convergence plus rapide	Complexité accrue

Table 1.3: Principales variantes de l'algorithme EM classique

L'EM stochastique, proposé par Celeux et Diebolt (1985), modifie l'étape E en remplaçant le calcul des responsabilités par un tirage aléatoire : chaque observation est assignée à une seule composante selon la distribution définie par les responsabilités. Cette introduction de stochasticité permet d'éviter certains optima locaux, de calculer des estimations d'incertitude empiriques, et d'explorer plus largement l'espace des paramètres, au prix de la perte de la propriété de monotonie de la vraisemblance.

Pour les grandes bases de données, l'EM standard, dit « batch », traite l'ensemble des données à chaque itération, ce qui peut se révéler prohibitif. L'EM online, proposé

par Sato et Ishii (2000) puis par Cappé et Moulines (2009), met à jour les paramètres après chaque observation ou mini-lot, réduisant drastiquement le coût par itération, et converge dans les mêmes conditions que l'EM standard sous des conditions de type Robbins-Monro sur le taux d'apprentissage.

L'EM généralisé remplace l'étape M exacte par une étape qui améliore, sans nécessairement maximiser, la fonction Q : on requiert seulement $\mathrm{Q}(\theta^{t+1} \mid \theta^t) \geq \mathrm{Q}(\theta^t \mid \theta^t)$, au lieu d'une maximisation exacte. La propriété de croissance monotone de la vraisemblance est préservée sous cette condition plus faible, ce qui rend le GEM utile lorsque l'étape M exacte est coûteuse ou n'admet pas de solution analytique.

Enfin, l'application originale ayant motivé l'article de Dempster et al. était précisément le traitement des données manquantes. Lorsque certaines composantes de x_i sont manquantes, on peut traiter les valeurs manquantes comme des variables latentes supplémentaires, et l'algorithme EM impute naturellement ces valeurs en calculant leur espérance conditionnelle à l'étape E. Cette approche est asymptotiquement plus efficace que l'imputation simple et produit des estimateurs cohérents sous l'hypothèse MCAR ou MAR.

1.6.6 Extension aux données séquentielles : l'algorithme de Baum-Welch

Une application particulièrement importante de l'algorithme EM, mentionnée à plusieurs reprises dans ce mémoire (tableau 1.1, sections 1.6.1 et 3.1.3), concerne les modèles de Markov cachés (HMM), utilisés notamment en bio-informatique et en finance. Contrairement au GMM, où les observations sont supposées indépendantes conditionnellement à leur variable latente, un HMM suppose que la séquence des variables latentes (z_1, …, z_T) forme elle-même une chaîne de Markov : z_t dépend de z_{t-1} selon une matrice de transition A, et chaque observation x_t dépend de l'état caché courant z_t selon une matrice d'émission B. Cette structure séquentielle empêche le calcul direct des responsabilités comme à l'équation (1.15), puisque les variables latentes ne sont plus indépendantes entre elles : l'étape E doit être remplacée par l'algorithme forward-backward, qui exploite la structure de chaîne pour calculer efficacement, par programmation dynamique, les probabilités a posteriori nécessaires. L'algorithme résultant, connu sous le nom d'algorithme de Baum-Welch (Baum et al., 1970), est historiquement antérieur à la formalisation générale de Dempster, Laird et Rubin, mais en constitue, comme le notent ces derniers dans leur article de 1977, un cas particulier.

Illustration numérique. Considérons un HMM discret à deux états cachés (« Beau temps » = B, « Pluvieux » = P), avec une matrice de transition A = [[0,7 ; 0,3], [0,4 ; 0,6]] ($A_{ij} = \mathrm{P}(z_{t+1}=\mathrm{j} \mid z_t=\mathrm{i})$), et une matrice d'émission B qui associe à chaque état une probabilité d'observer « Parapluie » (Pa) ou « Pas de parapluie » (NPa) :

B(B) = [0,1 ; 0,9] et B(P) = [0,8 ; 0,2]. Étant donné une courte séquence observée O = (NPa, Pa, Pa), l'étape E (forward-backward) calcule, pour chaque instant t et chaque état i, la probabilité a posteriori $\gamma_t(\mathrm{i}) = \mathrm{p}(z_t{=}\mathrm{i} \mid \mathrm{O})$, de la même manière que r_{ik} à l'équation (1.15) pour le GMM, mais en tenant compte de la dépendance séquentielle via les récursions forward $\alpha_t(\mathrm{i}) = [\Sigma_j\ \alpha_{t-1}(\mathrm{j})\ A_{ji}]\ B_i(o_t)$ et backward $\beta_t(\mathrm{i}) = \Sigma_j\ A_{ij}\ B_j(o_{t+1})\ \beta_{t+1}(\mathrm{j})$.

Une fois les quantités $\gamma_t(\mathrm{i})$ calculées, l'étape M du Baum-Welch réestime les matrices A et B par des formules directement analogues à l'équation (1.16) du GMM, en remplaçant les responsabilités r_{ik} par les quantités $\gamma_t(\mathrm{i})$ et $\xi_t(\mathrm{i,j}) = \mathrm{p}(z_t{=}\mathrm{i},\ z_{t+1}{=}\mathrm{j} \mid \mathrm{O})$: $\hat{\mathrm{A}}_{ij} = \Sigma_t\ \xi_t(\mathrm{i,j})\ /\ \Sigma_t\ \gamma_t(\mathrm{i})$, et $\hat{B}_i(\mathrm{v}) = \Sigma_\{\mathrm{t} : o_t{=}\mathrm{v}\}\ \gamma_t(\mathrm{i})\ /\ \Sigma_t\ \gamma_t(\mathrm{i})$. Cette correspondance formelle entre les mises à jour du Baum-Welch et celles du GMM illustre concrètement l'affirmation, faite à la section 1.6.1, selon laquelle l'EM constitue un cadre général unifiant des algorithmes historiquement développés de façon indépendante : le Baum-Welch n'est rien d'autre qu'un EM classique dans lequel l'étape E est rendue efficace, par programmation dynamique, pour tenir compte de la dépendance séquentielle entre variables latentes successives une adaptation que l'on retrouve, sous une forme analogue, dans les extensions séquentielles de l'EM variationnel évoquées à la section 1.7.7.

Calcul numérique complet de l'étape E. Pour rendre entièrement concret le mécanisme décrit ci-dessus, calculons explicitement les quantités forward $\alpha_t(\mathrm{i})$, backward $\beta_t(\mathrm{i})$ et les probabilités a posteriori $\gamma_t(\mathrm{i})$ pour la séquence observée O = (NPa, Pa, Pa), avec les matrices A et B et la distribution initiale π_0 = (0,6 ; 0,4) données ci-dessus. Le tableau 1.4 présente le détail de ce calcul.

t	**Observation** o_t	α_t**(B)**	α_t**(P)**	β_t**(B)**	β_t**(P)**	γ_t**(B)**	γ_t**(P)**
1	NPa	0,540	0,080	0,147	0,262	0,791	0,210
2	Pa	0,041	0,168	0,310	0,520	0,127	0,873
3	Pa	0,010	0,091	1,000	1,000	0,096	0,904

Table 1.4: Calcul détaillé des quantités forward, backward et a posteriori pour O = (NPa, Pa, Pa)

La dernière colonne du tableau se lit comme suit : après observation de la séquence complète (NPa, Pa, Pa), la probabilité a posteriori que le temps ait été « Beau » (B) au premier instant est de 0,791 (malgré l'absence de parapluie ce jour-là, cohérent avec un temps beau), tandis qu'elle chute à 0,127 au deuxième instant et à 0,096 au troisième, la présence du parapluie aux instants 2 et 3 rendant l'état « Pluvieux » (P) très majoritairement probable a posteriori ($\gamma_2(\mathrm{P}) = 0{,}873$ et $\gamma_3(\mathrm{P}) = 0{,}904$). On notera que $\Sigma_i\ \gamma_t(\mathrm{i}) = 1$ à chaque instant t, exactement comme $\Sigma_k\ r_{ik} = 1$ pour les responsabilités du GMM à l'équation (1.15) les quantités $\gamma_t(\mathrm{i})$ jouant, pour le HMM,

exactement le rôle que jouent les responsabilités r_{ik} pour le GMM. La probabilité totale de la séquence observée, obtenue en sommant α_t(i) sur i au dernier instant, vaut p(O | A, B, π_0) = α_3(B) + α_3(P) = 0,010 + 0,091 = 0,100.

1.6.7 Avantages de l'EM classique

- **Simplicité et transparence :** l'algorithme EM est conceptuellement clair et facile à comprendre. Les deux étapes E et M ont des interprétations directes, et les mises à jour sont souvent analytiques, ce qui facilite l'implémentation, le débogage et l'adaptation à de nouveaux modèles.

- **Garantie de convergence monotone :** la log-vraisemblance ne peut que croître ou rester constante à chaque itération. Cette propriété offre une sécurité précieuse : l'algorithme ne peut pas « se dégrader » au fil des itérations, et la log-vraisemblance constitue un critère de convergence naturel.

- **Efficacité computationnelle par itération :** chaque itération de l'EM a un coût fixe en O(NKD^2), aisément vectorisable. Pour des problèmes de taille modérée, l'EM est extrêmement rapide et se parallélise bien.

- **Applicabilité universelle :** l'EM s'applique à n'importe quel modèle probabiliste pour lequel p(Z | X, θ) est calculable et $\mathbb{E}$[log p(X, Z | θ)] est maximisable, ce qui couvre une immense variété de modèles en pratique.

- **Gestion naturelle des données incomplètes :** par construction, l'EM intègre sur les valeurs manquantes, produisant des estimateurs asymptotiquement efficaces sans recourir à des heuristiques d'imputation.

- **Convergence rapide loin de l'optimum :** bien que la convergence soit linéaire asymptotiquement, l'EM converge souvent très rapidement lors des premières itérations, permettant d'obtenir de bonnes solutions en peu d'itérations.

1.6.8 Inconvénients et limites de l'EM classique

- **Convergence vers des optima locaux :** l'EM converge vers un point stationnaire de la vraisemblance, généralement un maximum local. Sur les problèmes multimodaux, comme les GMM à composantes proches, la solution trouvée peut être significativement sous-optimale par rapport au maximum global.

- **Nécessité de spécifier K a priori :** le nombre de composantes K doit être fixé avant l'exécution de l'algorithme. Une mauvaise spécification de K conduit inévitablement à des résultats sous-optimaux : la sous-estimation entraîne des

composantes trop larges fusionnant des groupes distincts, tandis que la surestimation entraîne des composantes redondantes ou dégénérées. C'est précisément cette limite que nous illustrerons expérimentalement au chapitre 2, à travers un cas où K est volontairement surestimé.

- **Absence de quantification de l'incertitude :** l'EM classique fournit des estimations ponctuelles $\hat{\theta}$, sans aucune distribution autour de ces estimations, ce qui pose problème dans de nombreuses applications, en médecine ou en finance notamment, où l'incertitude sur les paramètres est aussi importante que les paramètres eux-mêmes pour la prise de décision.

- **Singularités et dégénérescences :** un problème bien connu du GMM est la possibilité de convergence vers des solutions singulières, une composante pouvant s'effondrer sur un seul point de données, sa variance tendant vers zéro, ce qui provoque une divergence de la log-vraisemblance vers l'infini. Des stratégies ad hoc, telles que la régularisation de la covariance, sont nécessaires pour éviter ce problème.

- **Lenteur de convergence :** le taux de convergence linéaire peut rendre l'algorithme très lent dans les régions proches de l'optimum, en particulier lorsque les composantes se chevauchent fortement.

- **Scalabilité limitée :** la version batch standard nécessite de traiter toutes les données à chaque itération, ce qui devient prohibitif pour des bases de plusieurs millions d'observations, sans recourir aux variantes online ou mini-batch.

1.7 L'algorithme EM variationnel

1.7.1 Motivation et cadre général

Les limitations identifiées à la section 1.6.8 motivent naturellement le développement d'une approche plus riche. L'EM variationnel émerge à la confluence de deux courants : l'inférence bayésienne, qui traite les paramètres comme des variables aléatoires et vise à caractériser leur distribution a posteriori complète, et l'optimisation variationnelle, qui transforme le problème d'inférence généralement intractable en un problème d'optimisation sur un espace de distributions plus simples.

Le cadre variationnel trouve ses racines dans les travaux de Hinton et van Camp (1993) sur les réseaux de neurones bayésiens, de Peterson et Anderson (1987) sur les champs moyens pour les réseaux de Boltzmann, et a été synthétisé et généralisé par Jordan, Ghahramani, Jaakkola et Saul (1999) dans leur article fondateur sur les méthodes variationnelles pour les modèles graphiques. Depuis lors, l'inférence variationnelle est

devenue l'une des méthodes d'inférence approchée les plus utilisées en apprentissage automatique probabiliste. Pour le mélange gaussien, l'application aux priors conjugués suit Attias (2000), Corduneanu et Bishop (2001) et Bishop (2006, chapitre 10).

1.7.2 Inférence variationnelle

a) Le problème d'inférence bayésienne

Dans le cadre bayésien complet, on cherche à calculer la distribution a posteriori jointe sur les paramètres et les variables latentes, p(θ, Z | X) = p(X | Z, θ) p(Z | θ) p(θ) / p(X). Le dénominateur, p(X) = $\int \int$ p(X | Z, θ) p(Z | θ) p(θ) dZ $d\theta$, est la vraisemblance marginale, ou évidence. Son calcul nécessite une intégrale de haute dimension, généralement intractable analytiquement pour des modèles non triviaux. Sans ce dénominateur, on ne peut pas normaliser la distribution a posteriori. L'inférence variationnelle contourne ce problème en remplaçant le calcul exact de p(θ, Z | X) par l'optimisation d'une approximation.

b) La famille variationnelle et le problème d'optimisation

L'inférence variationnelle introduit une famille de distributions $\mathcal{Q} = \{q(\theta, Z \mid \lambda), \lambda \in \Lambda\}$, paramétrée par des paramètres variationnels λ. L'objectif est de trouver le membre de cette famille le plus proche de la vraie distribution a posteriori, au sens de la divergence de Kullback-Leibler :

$$q*(\theta,Z) = \underset{q\in\mathcal{Q}}{\operatorname{argmin}}\, KL[q(\theta,Z)\|p(\theta,Z|X)] \tag{1.20}$$

Cette divergence ne peut pas être calculée directement, car elle fait intervenir p(X) dans sa normalisation. On montre cependant que minimiser cette divergence est équivalent à maximiser l'ELBO, qui lui est calculable :

$$ELBO(q) = \mathbb{E}_q[\log p(X,Z,\theta)] - \mathbb{E}_q[\log q(\theta,Z)] \tag{1.21}$$

Cette reformulation est la clé de l'inférence variationnelle : un problème d'inférence, à savoir le calcul d'une distribution, est transformé en un problème d'optimisation, à savoir la maximisation d'une fonctionnelle de q, qui peut être traité par des algorithmes d'optimisation standard.

c) L'ELBO comme objectif d'apprentissage

L'ELBO peut être décomposé de deux manières complémentaires, chacune offrant une interprétation différente. La première décomposition montre l'ELBO comme la différence entre la vraisemblance attendue et la complexité du modèle :

$$ELBO(q) = \mathbb{E}_q[\log p(X|Z,\theta)] - KL[q(\theta,Z)\|p(\theta,Z)] \tag{1.22}$$

Le premier terme, la log-vraisemblance attendue sous q, mesure la qualité de l'ajustement aux données ; le second terme, la divergence KL entre la distribution variationnelle et le prior joint, pénalise les distributions q qui s'écartent trop du prior, réalisant ainsi une forme de régularisation bayésienne automatique. La maximisation de l'ELBO réalise donc un compromis naturel entre ajustement aux données et fidélité au prior.

La seconde décomposition, déjà introduite à la section 1.5.3, montre l'ELBO comme une borne inférieure de la log-évidence : log p(X) = ELBO(q) + KL[q(θ, Z) ‖ p(θ, Z | X)] ≥ ELBO(q). Cette décomposition est fondamentale pour la sélection de modèle bayésien : un modèle dont l'ELBO est plus élevé à la convergence est préférable, car son évidence est plus grande, ce qui implique qu'il explique mieux les données tout en contrôlant la complexité.

1.7.3 Formulation de l'EM variationnel

a) L'hypothèse du champ moyen

L'hypothèse la plus courante pour définir la famille variationnelle $\mathcal{Q}$ est l'hypothèse du champ moyen, qui suppose une factorisation complète de la distribution variationnelle sur les différentes variables :

$$q(\theta,Z) = q(\theta)\prod_i q(z_i) = q(\pi)\prod_k q(\mu_k,\Lambda_k)\prod_i q(z_i) \tag{1.23}$$

Cette hypothèse ignore les corrélations a posteriori entre les paramètres θ et les variables latentes Z, ainsi qu'entre les variables latentes de différentes observations. Si cette indépendance est rarement exacte dans la vraie distribution a posteriori, elle simplifie considérablement le problème d'optimisation en le rendant décomposable en sous-problèmes indépendants, tout en restant une approximation raisonnable dans de nombreuses applications.

b) Les mises à jour variationnelles optimales

Sous l'hypothèse du champ moyen, la solution variationnelle optimale pour chaque facteur $q_j(z_j)$ est donnée par la formule du champ moyen variationnel :

$$\log q *_j (z_j) = \mathbb{E}_{q(z\setminus z_j)}[\log p(X,Z,\theta)] + constante \tag{1.24}$$

Cette mise à jour prend l'espérance de la log-distribution jointe par rapport à tous les autres facteurs, en maintenant z_j fixe, la constante additive assurant la normalisation. Cette formule est générale et s'applique à n'importe quel facteur, qu'il s'agisse d'une

variable latente ou d'un paramètre. Dans le cas des modèles conjugués, cette espérance peut être calculée analytiquement et conduit à une distribution de la même famille que le prior correspondant.

c) Dérivation détaillée pour le mélange gaussien bayésien

Cette section applique explicitement la formule générale du champ moyen variationnel, énoncée ci-dessus, au cas particulier du mélange gaussien bayésien, afin de montrer comment les formes de Dirichlet et de Normal-Wishart, utilisées dans l'algorithme de la section 1.7.3.d, se déduisent directement de cette formule générale plutôt que d'être postulées arbitrairement.

Facteur q(π). En isolant, dans le logarithme de la distribution jointe complète log p(X, Z, π, μ, Λ), les seuls termes qui dépendent de π, on obtient log p(Z | π) + log p(π) = $\Sigma_i \Sigma_k z_{ik} \log \pi_k + \Sigma_k (\alpha_0\text{-}1) \log \pi_k$ + constante, où z_{ik} désigne l'indicatrice z_i = k. En appliquant la formule du champ moyen, on prend l'espérance de cette expression par rapport à q(Z), ce qui revient à remplacer z_{ik} par son espérance $r_{ik} = \mathbb{E}_q[z_{ik}]$:

$$\log q*(\pi) = \Sigma_k(\alpha_0 - 1 + N_k)\log \pi_k + \text{constante, où } N_k = \Sigma_i r_{ik} \tag{1.25}$$

Cette expression est, à une constante de normalisation près, exactement le logarithme de la densité d'une distribution de Dirichlet de paramètres $\alpha_k = \alpha_0 + N_k$. On retrouve ainsi directement, par simple application de la formule générale, la mise à jour $\alpha_k = \alpha_0 + N_k$ utilisée à la section 1.7.3.d, sans qu'il soit nécessaire de la postuler : elle découle mécaniquement de la conjugaison entre le prior de Dirichlet et la vraisemblance catégorielle.

Facteur q(μ_k, Λ_k). De façon similaire, en isolant les termes qui dépendent de (μ_k, Λ_k) dans log p(X, Z, π, μ, Λ), on obtient une contribution de la vraisemblance gaussienne, $\Sigma_i r_{ik} \log \mathcal{N}(x_i \mid \mu_k, \Lambda_k^{-1})$, à laquelle s'ajoute le logarithme du prior Normal-Wishart log p(μ_k, Λ_k). Cette somme, une fois réorganisée en complétant le carré sur μ_k une manipulation algébrique standard pour les formes quadratiques gaussiennes , se réécrit comme le logarithme d'une densité Normal-Wishart de paramètres mis à jour :

$$\beta_k = \beta_0 + N_k, m_k = (\beta_0 m_0 + N_k \bar{x}_k)/\beta_k, \nu_k = \nu_0 + N_k, W_k^{-1} = W_0^{-1} + N_k S_k + [\beta_0 N_k/(\beta_0 + N_k)](\bar{x}_k - m_0)(\bar{x}_k - m_0)^T \tag{1.26}$$

où $\bar{x}_k$ et S_k désignent respectivement la moyenne et la covariance empiriques pondérées par les responsabilités, telles que définies à la section 1.7.3.d. Le terme β_k s'interprète comme une « taille d'échantillon effective augmentée » (la taille d'échantillon effective N_k à laquelle s'ajoute la pseudo-taille d'échantillon β_0 du prior), tandis que le dernier terme correctif dans la mise à jour de W_k^{-1} traduit l'incertitude supplémentaire due à

l'écart entre la moyenne empirique du cluster $\bar{x}_k$ et la moyenne a priori m_0 : plus cet écart est grand par rapport à la confiance relative accordée au prior (mesurée par le rapport $\beta_0 N_k/(\beta_0+N_k)$), plus la matrice de précision W_k^{-1} voit sa valeur augmenter, ce qui élargit d'autant la distribution a posteriori sur μ_k.

Cette dérivation confirme, de façon rigoureuse, que les formules de mise à jour utilisées à la section 1.7.3.d ne sont pas de simples analogues heuristiques de l'étape M classique : elles sont la conséquence mathématique directe et nécessaire de l'application de la formule générale du champ moyen variationnel au choix spécifique de priors conjugués effectué à la section 1.4.2. Ce même schéma de dérivation isoler les termes pertinents dans la log-distribution jointe, prendre leur espérance sous les autres facteurs, puis reconnaître la forme conjuguée résultante se généralise sans difficulté conceptuelle à tout modèle à variables latentes construit à partir de distributions conjuguées, ce qui explique la popularité de l'hypothèse du champ moyen dans la pratique de l'inférence variationnelle.

d) L'algorithme EM variationnel

L'algorithme EM variationnel alterne entre deux étapes, analogues aux étapes E et M de l'EM classique, mais opérant sur des distributions plutôt que sur des estimations ponctuelles. L'algorithme prend en entrée les observations X, un nombre maximal de composantes Kmax, les hyperparamètres du prior $\{\alpha_0, m_0, \beta_0, W_0, \nu_0\}$, ainsi qu'un critère d'arrêt ε.

Initialisation. Les paramètres variationnels sont initialisés : $\alpha_k = \alpha_0 + \mathrm{N/K}$, $N_k = \mathrm{N/K}$, $m_k = m_0 +$ perturbation aléatoire, $\beta_k = \beta_0 + N_k$, $\nu_k = \nu_0 + N_k$, $W_k = W_0$.

Étape E variationnelle. Pour chaque observation i et chaque composante k, on calcule :

$$\log \rho_{ik} = \mathbb{E}[\log \pi_k] + \tfrac{1}{2}\mathbb{E}[\log |\Lambda_k|] - (D/2)\log(2\pi) - \tfrac{1}{2}\mathbb{E}[(x_i - \mu_k)^T \Lambda_k (x_i - \mu_k)] \quad (1.27)$$

puis on normalise pour obtenir les responsabilités : $r_{ik} = \exp(\log \rho_{ik}) \,/\, \Sigma_j \exp(\log \rho_{ij})$.

Étape M variationnelle. Pour chaque composante k, on calcule $N_k = \Sigma_i\, r_{ik}$, $\bar{x}_k = (1/N_k)\ \Sigma_i\, r_{ik}\ x_i$, et $S_k = (1/N_k)\ \Sigma_i\, r_{ik}\ (x_i\text{-}\bar{x}_k)(x_i\text{-}\bar{x}_k)^T$, avant de mettre à jour les paramètres des lois de Dirichlet et Normal-Wishart :

$$\alpha_k = \alpha_0 + N_k, \beta_k = \beta_0 + N_k, m_k = (\beta_0 m_0 + N_k \bar{x}_k)/\beta_k, \nu_k = \nu_0 + N_k \quad (1.28)$$

ainsi que la mise à jour de la matrice d'échelle $W_k^{-1} = W_0^{-1} + N_k S_k + [\beta_0 N_k/(\beta_0+N_k)]$ $(\bar{x}_k\text{-}m_0)(\bar{x}_k\text{-}m_0)^T$. L'ELBO est ensuite calculé, et les composantes telles que $\alpha_k \approx \alpha_0$

sont élaguées, selon le mécanisme détaillé à la section 1.7.4. Ces deux étapes sont répétées jusqu'à convergence de l'ELBO, l'algorithme retournant alors la distribution q*(θ) ainsi que le nombre effectif de composantes actives, K_effectif.

e) Calcul des espérances nécessaires

L'étape E variationnelle requiert le calcul de plusieurs espérances par rapport aux distributions variationnelles courantes, espérances qui admettent des formes analytiques pour les distributions conjuguées utilisées. Pour la distribution de Dirichlet q(π) = Dir(α_1, ..., αK), l'espérance du logarithme du poids est donnée par :

$$\mathbb{E}[\log \pi_k] = \psi(\alpha_k) - \psi(\Sigma_j \alpha_j) \tag{1.29}$$

où ψ désigne la fonction digamma. Pour la distribution de Wishart q(Λ_k) = $\mathcal{W}(W_k$, ν_k), on obtient $\mathbb{E}[\log|\Lambda_k|] = \Sigma_{j=1}^{D}\ \psi[(\nu_k$+1-j)/2] + D log 2 + log$|W_k|$. Enfin, pour la distribution normale q($\mu_k|\Lambda_k$) = $\mathcal{N}(m_k$, $(\beta_k\Lambda_k)^{-1}$), l'espérance de la forme quadratique vaut :

$$\mathbb{E}[(x-\mu_k)^T\Lambda_k(x-\mu_k)] = D/\beta_k + \nu_k(x-m_k)^T W_k(x-m_k) \tag{1.30}$$

1.7.4 Sélection automatique du nombre de composantes

L'une des propriétés les plus remarquables et pratiquement importantes de l'EM variationnel est sa capacité à déterminer automatiquement le nombre effectif de composantes dans un mélange gaussien bayésien. Ce mécanisme, qui constitue l'un des apports majeurs de l'EM variationnel par rapport à l'EM classique, mérite une explication détaillée, car il sera au cœur de l'étude expérimentale présentée au chapitre 2.

La clé de ce mécanisme réside dans le prior de Dirichlet Dir(α_0, ..., α_0) placé sur les poids de mélange. Lorsque α_0 est petit typiquement $\alpha_0 = 1/\text{Kmax}$ ou $\alpha_0 = 10^{-3}$, ce prior favorise fortement les solutions parcimonieuses, où la plupart des poids sont proches de zéro, c'est-à-dire les solutions n'utilisant qu'un petit nombre de composantes actives.

Lors de l'optimisation variationnelle, une composante k pour laquelle les données ne fournissent pas de support suffisant voit son paramètre α_k rester proche de α_0, faute d'augmentation significative lors de l'étape M. Cela se traduit par $\mathbb{E}[\log\ \pi_k]$ proche d'une constante fortement négative à l'étape E, ce qui rend les responsabilités r_{ik} proches de zéro pour toutes les observations : la composante est alors effectivement « éteinte » et n'influence plus le modèle. En pratique, on considère qu'une composante est inactive lorsque $N_k = \Sigma_i\ r_{ik} < 1$.

Ce mécanisme d'élagage automatique constitue une forme de rasoir d'Occam bayésien : le modèle utilise spontanément le nombre minimal de composantes nécessaires

pour expliquer les données, sans sur-ajustement. En initialisant avec un Kmax suffisamment grand par exemple Kmax = 20 pour un problème où l'on attend entre 3 et 5 groupes , l'algorithme converge naturellement vers une solution utilisant uniquement le nombre approprié de composantes actives, sans qu'il soit nécessaire d'exécuter l'algorithme séparément pour chaque valeur candidate de K, comme l'exigerait l'EM classique.

1.7.5 Exemple numérique illustratif

Afin de rendre plus concret le mécanisme d'élagage décrit ci-dessus, reprenons l'exemple numérique de la section 1.6.4 : les huit observations scalaires X = {0,5 ; 1,0 ; 1,2 ; 1,8 ; 7,1 ; 7,5 ; 8,0 ; 8,3}, qui forment visiblement deux groupes bien séparés. Appliquons cette fois l'EM variationnel avec Kmax = 3 composantes soit une de plus que le nombre réel de groupes , un prior de Dirichlet de concentration $\alpha_0 = 1/3$, et des priors Normal-Wishart non informatifs ($\beta_0 = 10^{-3}$, $\nu_0 = 1$, $W_0 = 1$).

L'algorithme converge en 5 à 15 itérations selon l'initialisation (critère $|\Delta\text{ELBO}| < 10^{-6}$), vers la même solution dans les dix essais réalisés (graines 0 à 4, initialisation par les hyperparamètres ou par k-means). Le tableau 1.5 présente les paramètres finaux obtenus pour chacune des trois composantes.

Composante k	α_k	**Poids** $\pi_k = \alpha_k/\Sigma\alpha$	**Moyenne** m_k	**Interprétation**
1	4,333	0,482	7,724	Groupe des grandes valeurs (actif)
2	4,333	0,482	1,126	Groupe des petites valeurs (actif)
3	0,333	0,037	4,425	Composante élaguée ($\alpha_k = \alpha_0$)

Table 1.5: Résultat de l'EM variationnel sur l'exemple numérique à 8 observations (Kmax = 3)

Ce résultat illustre de façon particulièrement nette le mécanisme théorique décrit ci-dessus. Les deux premières composantes se partagent presque également la masse de probabilité (poids 0,482 chacune), avec des moyennes de 7,724 et 1,126 qui coïncident, à 10^{-3} près, avec les moyennes empiriques des deux groupes et avec la solution de l'EM classique de la section 1.6.4 (7,725 et 1,125). La troisième composante, en revanche, voit son paramètre α_k rester exactement égal à sa valeur initiale $\alpha_0 = 1/3 = 0{,}333$, signe que $N_k = \alpha_k - \alpha_0 = 0$: aucune observation, même partiellement, ne lui a été

attribuée au cours de la convergence. Cette composante est donc entièrement inactive, et le modèle retient effectivement K_effectif = 2 composantes actives, exactement le nombre réel de groupes un résultat que l'EM classique, avec K fixé à l'avance, ne peut pas produire de lui-même : il faudrait, pour l'EM classique, comparer explicitement les solutions à K = 2 et K = 3 au moyen d'un critère externe tel que le BIC, alors que l'EM variationnel effectue cette sélection de façon endogène, en une seule exécution.

1.7.6 Calcul de l'ELBO et critère de convergence

Pour le mélange gaussien bayésien sous l'hypothèse du champ moyen, l'ELBO peut être calculé analytiquement à chaque itération. Il se décompose sous la forme :

$$ELBO = \mathbb{E}_q[\log p(X|Z,\theta)] + \mathbb{E}_q[\log p(Z|\pi)] + \mathbb{E}_q[\log p(\pi)] + \Sigma_k \mathbb{E}_q[\log p(\mu_k,\Lambda_k)] - \mathbb{E}_q[\log q(Z)] - \mathbb{E}_q[\log q(\pi)] - \Sigma_k \mathbb{E}_q[\log q(\mu_k,\Lambda_k)] \tag{1.31}$$

Chacun de ces termes peut être calculé analytiquement grâce aux formes conjuguées des distributions considérées ; leur expression complète est donnée par Bishop (2006, éq. 10.71 à 10.77) et implémentée à l'annexe B. L'ELBO constitue le critère de convergence naturel de l'EM variationnel : on arrête l'algorithme lorsque |ELBO(t) - ELBO(t-1)| $<$ ε. Contrairement à l'EM classique, où la log-vraisemblance peut diverger vers l'infini en présence de singularités, l'ELBO de l'EM variationnel reste toujours borné, grâce à la régularisation bayésienne induite par les priors.

1.7.7 Variantes avancées de l'EM variationnel

Inférence variationnelle naturelle. L'inférence variationnelle standard utilise le gradient ordinaire de l'ELBO pour optimiser les paramètres variationnels. Une variante plus efficace utilise le gradient naturel, qui prend en compte la géométrie de l'espace des distributions à travers la métrique de Fisher. Les mises à jour par gradient naturel sont invariantes aux reparamétrisations et convergent généralement beaucoup plus rapidement que les mises à jour utilisant le gradient ordinaire.

Inférence variationnelle stochastique (SVI). Proposée par Hoffman et al. (2013), la SVI étend l'EM variationnel aux très grandes bases de données en utilisant des mini-lots. À chaque itération, les mises à jour variationnelles sont calculées sur un sous-ensemble des données avant de mettre à jour les paramètres globaux à l'aide d'un taux d'apprentissage décroissant. La SVI constitue l'analogue de l'EM online pour l'EM variationnel et a permis l'application de modèles bayésiens complexes à des corpus contenant plusieurs millions de documents.

Inférence variationnelle amortie (VAE). Les auto-encodeurs variationnels, introduits par Kingma et Welling (2014), représentent une généralisation profonde de

l'EM variationnel aux modèles génératifs profonds. Le principe consiste à apprendre une fonction d'encodage q(Z|X, ϕ), paramétrée par un réseau de neurones, qui approxime directement la distribution a posteriori. Cette stratégie, appelée amortisation de l'inférence, permet de traiter efficacement de nouvelles observations sans réoptimiser les paramètres variationnels pour chacune d'elles. Aujourd'hui, le VAE constitue l'un des modèles génératifs les plus influents en apprentissage profond.

Normalizing Flows. Une limitation importante de l'hypothèse du champ moyen est qu'elle restreint les distributions variationnelles à des familles factorisées souvent trop simples pour représenter fidèlement les corrélations de la distribution a posteriori. Les Normalizing Flows, proposés par Rezende et Mohamed (2015), permettent de construire des familles variationnelles beaucoup plus expressives en composant des transformations bijectives inversibles, tout en conservant la différentiabilité. Cette approche combine ainsi la flexibilité des modèles génératifs profonds avec la rigueur du cadre variationnel bayésien.

1.7.8 Avantages de l'EM variationnel

- **Quantification complète de l'incertitude :** l'EM variationnel fournit une distribution q*(θ) sur les paramètres, permettant de calculer des intervalles de crédibilité, des quantiles, et d'intégrer l'incertitude paramétrique dans les prédictions, propriété fondamentale dans les applications où la décision doit tenir compte de l'incertitude.

- **Sélection automatique du nombre de composantes :** le mécanisme d'élagage variationnel détermine automatiquement le nombre effectif de composantes, éliminant la nécessité d'une sélection externe de K et permettant une inférence en une seule exécution plutôt qu'une série d'exécutions pour différentes valeurs de K.

- **Régularisation bayésienne et robustesse :** l'intégration des distributions a priori protège naturellement contre le surapprentissage et les singularités, la divergence de Kullback-Leibler vers le prior dans l'ELBO agissant comme une régularisation qui empêche les composantes de s'effondrer sur des observations individuelles.

- **ELBO comme critère de convergence et de sélection :** l'ELBO constitue une borne inférieure de la log-évidence et représente un critère de convergence naturel, informatif et comparable entre différents modèles, ce qui fournit un critère de sélection bayésien intégré.

- **Cadre extensible aux modèles complexes :** le cadre variationnel s'étend

naturellement aux modèles hiérarchiques complexes, aux modèles génératifs profonds tels que les VAE, ainsi qu'aux modèles comportant plusieurs niveaux de variables latentes, là où l'EM classique devient difficilement applicable.

- **Convergence monotone de l'ELBO :** comme l'EM classique, l'EM variationnel garantit que l'ELBO ne décroît jamais entre deux itérations, propriété qui assure une grande stabilité numérique et facilite le diagnostic de la convergence.

1.7.9 Inconvénients et limites de l'EM variationnel

- **Biais d'approximation systématique :** l'hypothèse du champ moyen introduit un biais systématique dans l'approximation. La distribution variationnelle optimale q* tend généralement à sous-estimer la variance de la véritable distribution a posteriori, phénomène de zero-forcing lié à la divergence KL. Lorsque la distribution a posteriori est fortement multimodale ou présente de fortes corrélations, cette approximation peut devenir relativement grossière.

- **Complexité d'implémentation :** la dérivation des mises à jour variationnelles pour un nouveau modèle nécessite une bonne maîtrise du calcul variationnel bayésien ainsi que de la famille exponentielle. L'implémentation correcte du calcul de l'ELBO pour le suivi de la convergence est sensiblement plus complexe que celle de l'EM classique.

- **Sensibilité aux distributions a priori :** le choix des hyperparamètres des distributions a priori (α_0, m_0, β_0, W_0, ν_0) peut influencer de manière significative les résultats, particulièrement lorsque le nombre de données disponibles est limité, ce qui nécessite une expertise ou une procédure de calibration empirique.

- **Convergence vers des optima locaux de l'ELBO :** la maximisation de l'ELBO reste susceptible de converger vers des optima locaux. Bien que la régularisation bayésienne rende généralement la surface d'optimisation plus lisse que dans l'EM classique, ce problème n'est pas totalement éliminé, et plusieurs initialisations demeurent recommandées.

- **Coût computationnel par itération plus élevé :** le calcul des différentes espérances variationnelles, impliquant notamment les fonctions digamma, les déterminants et les opérations matricielles, entraîne un surcoût computationnel à chaque itération par rapport à l'EM classique. Pour des problèmes simples, ce coût supplémentaire peut ne pas être compensé par les bénéfices apportés.

- **Applicabilité restreinte aux modèles conjugués :** les mises à jour analytiques fermées de l'EM variationnel ne sont disponibles que pour les modèles

utilisant des distributions conjuguées. Pour les modèles non conjugués, il est nécessaire de recourir à des méthodes d'approximation supplémentaires, telles que les gradients stochastiques, les approximations de Laplace ou les méthodes de quadrature, ce qui augmente davantage la complexité de l'algorithme.

1.8 Conclusion du chapitre 1

Ce premier chapitre a permis de poser l'ensemble des fondements théoriques nécessaires à la compréhension des deux algorithmes centraux de ce mémoire. Nous avons vu que les modèles à variables latentes, et en particulier le modèle de mélange gaussien, fournissent un cadre unificateur dans lequel s'inscrivent naturellement l'EM classique et l'EM variationnel. L'inégalité de Jensen et la borne ELBO constituent le socle mathématique commun aux deux algorithmes, tandis que le cadre bayésien et la divergence de Kullback-Leibler permettent de comprendre précisément ce qui distingue l'approche variationnelle de l'approche classique.

Ce parcours théorique n'a pas été conduit de façon purement abstraite : chaque notion introduite la borne ELBO, la conjugaison Dirichlet–Catégorielle, le mécanisme d'élagage bayésien, la correspondance avec l'algorithme de Baum-Welch a été systématiquement illustrée par un exemple numérique entièrement calculé, du plus simple (l'évaluation de la densité d'un GMM à deux composantes, section 1.3.5) au plus élaboré (le calcul complet des quantités forward, backward et a posteriori d'un modèle de Markov caché, section 1.6.6). Cette démarche, délibérément choisie, répond à la conviction que la compréhension approfondie d'un algorithme d'inférence statistique passe nécessairement par la capacité à en dérouler, à la main ou par le calcul, chacune des étapes sur un exemple de taille réduite, avant d'en confier l'exécution à une bibliothèque logicielle démarche qui trouve son prolongement naturel dans les implémentations from scratch présentées au chapitre 3.

L'étude détaillée de l'EM classique a mis en évidence sa simplicité, son efficacité computationnelle et la garantie de convergence monotone qui le caractérisent, mais également ses limites majeures : nécessité de fixer K a priori, absence de quantification de l'incertitude, et sensibilité aux singularités. L'étude de l'EM variationnel a montré comment le cadre bayésien permet de lever une grande partie de ces limites, au prix d'une complexité d'implémentation et d'un coût computationnel plus élevés. La dérivation détaillée du champ moyen variationnel pour le cas particulier du GMM bayésien (section 1.7.3.c) a par ailleurs montré que les formules de mise à jour de l'EM variationnel, loin d'être postulées par analogie avec l'EM classique, se déduisent rigoureusement de l'application d'un principe unique la formule générale du champ moyen à un choix spécifique de distributions conjuguées, ce qui confère à l'ensemble de l'édifice théorique une cohérence interne que l'on ne perçoit pas toujours dans les présentations

plus informelles de la littérature d'introduction.

Le chapitre suivant s'attache à confronter directement ces deux algorithmes, sur les plans théorique, algorithmique et expérimental, à travers une étude de cas fondée sur un mélange de gaussiennes dont le nombre de composantes est volontairement surestimé étude qui permettra de vérifier empiriquement, et non plus seulement théoriquement, chacune des propriétés établies dans ce premier chapitre.

Chapitre 2

COMPARAISON ENTRE L'EM CLASSIQUE ET L'EM VARIATIONNEL

Après avoir posé, au chapitre précédent, les fondements théoriques communs aux deux algorithmes et détaillé leur formulation mathématique respective, ce deuxième chapitre propose une confrontation directe et systématique entre l'EM classique et l'EM variationnel. Cette confrontation est menée en trois temps : une comparaison théorique, qui synthétise les différences conceptuelles entre les deux approches ; une comparaison algorithmique, qui examine leurs différences de complexité, de coût computationnel et de comportement numérique ; et enfin une étude expérimentale, fondée sur un mélange de gaussiennes dont le nombre de composantes est volontairement surestimé, destinée à illustrer concrètement les différences de comportement observées entre les deux algorithmes.

Cette étude expérimentale se distingue par son ampleur : plutôt que de se limiter à une unique comparaison sur un jeu de données synthétique, elle a été délibérément étendue, au fil de la section 2.3, à douze expériences complémentaires et indépendantes allant de la sensibilité au choix du prior de Dirichlet (section 2.3.5) à la validation sur un jeu de données réel (section 2.3.11), en passant par la scalabilité computationnelle (section 2.3.6), la robustesse à l'initialisation (section 2.3.9) et au type de covariance (section 2.3.10), ou encore le comportement face à des valeurs aberrantes (section 2.3.12), puis répétée sur trente jeux de données indépendants pour chiffrer l'incertitude des conclusions (section 2.3.13). Ce choix méthodologique, plus exigeant qu'une comparaison isolée, répond à une préoccupation centrale de ce mémoire, déjà annoncée en introduction générale : ne pas se contenter d'affirmer la supériorité de l'un des deux algorithmes sur la base d'un unique résultat favorable, mais tester la robustesse de cette conclusion sous plusieurs angles complémentaires, quitte à devoir la nuancer lorsque les résultats l'exigent ce que fera précisément la section 2.3.11 consacrée au jeu de données réel Iris.

2.1 Comparaison théorique

Sur le plan théorique, l'EM classique et l'EM variationnel partagent un socle commun la maximisation alternée d'une borne inférieure de la vraisemblance, l'ELBO mais di-

vergent fondamentalement sur la nature de l'objet qu'ils cherchent à estimer. L'EM classique traite les paramètres θ comme des constantes inconnues, dont on cherche l'estimateur ponctuel du maximum de vraisemblance $\hat{\theta} = \text{argmax_}\theta\ \ell(\theta)$. L'EM variationnel, en revanche, traite θ comme une variable aléatoire dont on cherche à approximer la distribution a posteriori complète q*(θ), au moyen d'une famille variationnelle factorisée sous l'hypothèse du champ moyen.

Cette différence de nature a des conséquences directes sur l'interprétation des résultats produits par chacun des deux algorithmes. Le tableau 2.1 synthétise les principales différences théoriques entre les deux approches.

Aspect théorique	**EM classique**	**EM variationnel**
Nature des paramètres	Constantes inconnues (fréquentiste)	Variables aléatoires (bayésien)
Objet estimé	Estimation ponctuelle $\hat{\theta}$	Distribution a posteriori q*(θ)
Objectif optimisé	Vraisemblance $\ell(\theta)$, via l'ELBO à θ fixé	ELBO(q) conjointement sur q(θ) et q(Z)
Rôle du prior	Absent (ou implicite dans la régularisation)	Central : Dirichlet sur π, Normal-Wishart sur (μ, Λ)
Traitement de K	Fixé a priori, choisi par critère externe (BIC, AIC)	Sélectionné automatiquement par élagage bayésien
Nature de la borne optimisée	Égalité atteinte à l'étape E (q = p(Z|X,θ))	Approximation permanente (hypothèse du champ moyen)
Quantification de l'incertitude	Absente en natif (calcul a posteriori possible)	Native, via la distribution q*(θ)

Table 2.1: Comparaison théorique entre l'EM classique et l'EM variationnel

Il convient de noter que l'EM classique peut, dans une certaine mesure, être vu comme un cas particulier de l'inférence variationnelle : lorsque l'on choisit, pour la distribution variationnelle sur les paramètres, une masse de Dirac q(θ) = $\delta(\theta$ - $\hat{\theta})$, l'ELBO se réduit exactement à la log-vraisemblance $\ell(\hat{\theta})$, et l'algorithme obtenu coïncide avec l'EM classique. Cette observation, due notamment à Neal et Hinton (1998), éclaire d'un jour nouveau la relation entre les deux algorithmes : l'EM variationnel n'est pas une méthode radicalement différente de l'EM classique, mais une généralisation de celui-ci, dans laquelle la contrainte de Dirac sur θ est relâchée au profit d'une famille de distributions plus riche.

2.1.1 Dérivation de l'équivalence de Neal et Hinton

Cette section détaille la dérivation de l'équivalence énoncée ci-dessus, qui permet de replacer l'EM classique comme un cas particulier, plutôt qu'une alternative, de l'EM variationnel. Reprenons la définition de l'ELBO donnée à la section 1.2.2, en distinguant explicitement les variables latentes Z des paramètres θ dans la distribution variationnelle jointe q(Z, θ) = q(Z) q(θ) :

$$ELBO(q(Z),q(\theta)) = \mathbb{E}_{q(Z)q(\theta)}[\log p(X,Z|\theta)] + \mathbb{E}_{q(\theta)}[\log p(\theta)] - \mathbb{E}_{q(Z)}[\log q(Z)] - \mathbb{E}_{q(\theta)}[\log q(\theta)] \tag{2.1}$$

Choisissons à présent, pour la distribution variationnelle sur les paramètres, une masse de Dirac centrée en un point $\hat{\theta}$: q(θ) = δ(θ - $\hat{\theta}$). Par définition de la mesure de Dirac, l'espérance sous q(θ) de toute fonction f(θ) se réduit simplement à l'évaluation de f au point $\hat{\theta}$, c'est-à-dire $\mathbb{E}_${q(θ)}[f(θ)] = f($\hat{\theta}$). En appliquant cette propriété à chacun des quatre termes de l'ELBO ci-dessus :

$$ELBO(q(Z),\delta_{\hat{\theta}}) = \mathbb{E}_{q(Z)}[\log p(X,Z|\hat{\theta})] + \log p(\hat{\theta}) - \mathbb{E}_{q(Z)}[\log q(Z)] - \log \delta(0) \tag{2.2}$$

Le dernier terme, log δ(0), est une constante formellement infinie mais indépendante à la fois de q(Z) et de $\hat{\theta}$; elle n'affecte donc ni la localisation de l'optimum en q(Z), ni celle en $\hat{\theta}$, et peut être ignorée dans l'optimisation (cette manipulation, bien que peu rigoureuse au sens de la théorie de la mesure la masse de Dirac n'est pas une densité , est standard dans la littérature et se justifie proprement en considérant une suite de gaussiennes de variance tendant vers zéro). En supposant de plus un prior p(θ) non informatif, de sorte que le terme log p($\hat{\theta}$) devienne lui aussi négligeable ou constant, l'ELBO se réduit à :

$$ELBO(q(Z),\delta_{\hat{\theta}}) \approx \mathbb{E}_{q(Z)}[\log p(X,Z|\hat{\theta})] - \mathbb{E}_{q(Z)}[\log q(Z)] = ELBO(q(Z),\hat{\theta}) \tag{2.3}$$

Cette dernière expression est exactement l'ELBO à paramètres fixés $\hat{\theta}$, tel que défini à la section 1.2.2 pour l'EM classique. Sa maximisation par rapport à q(Z) donne, comme établi à la section 1.2.2, q*(Z) = p(Z | X, $\hat{\theta}$) soit exactement l'étape E de l'EM classique (section 1.6.2.b) , et sa maximisation par rapport à $\hat{\theta}$, une fois q(Z) fixée à cette valeur optimale, correspond exactement à l'étape M de l'EM classique (section 1.6.2.c), puisque $\mathbb{E}_${q(Z)}[log p(X,Z|θ)] est alors précisément la fonction Q($\theta|\theta^t$) définie à la section 1.6.2.c. L'EM classique apparaît ainsi comme le cas particulier de l'EM variationnel obtenu en restreignant la famille variationnelle sur les paramètres θ aux seules

masses de Dirac, c'est-à-dire à la famille la moins expressive possible une famille qui ne peut, par construction, représenter aucune incertitude sur θ, ce qui explique directement l'absence de quantification de l'incertitude relevée comme limite de l'EM classique à la section 1.6.8. Réciproquement, l'EM variationnel décrit à la section 1.7 s'obtient en remplaçant cette famille de Dirac par la famille, plus riche, des distributions factorisées sous l'hypothèse du champ moyen (section 1.7.3.a) : les deux algorithmes se déduisent donc d'un seul et même principe variationnel, appliqué à deux familles variationnelles emboîtées, l'une (Dirac) étant un cas particulier dégénéré de l'autre (champ moyen).

2.1.2 Trois niveaux d'inférence : MLE, MAP et Bayes complet

La dérivation de la section 2.1.1 permet de situer précisément l'EM classique et l'EM variationnel au sein d'une hiérarchie plus large de trois niveaux d'inférence statistique, qu'il est utile de distinguer explicitement, tant la confusion entre eux est fréquente dans la littérature d'introduction à l'apprentissage automatique.

Premier niveau : le maximum de vraisemblance (MLE). C'est le niveau auquel opère l'EM classique tel que présenté à la section 1.6 : on cherche $\hat{\theta}_\text{MLE} = \text{argmax}_\theta\ p(X|\theta)$, sans faire intervenir de prior $p(\theta)$ du tout. C'est l'estimateur le plus simple, mais aussi le plus sujet au surapprentissage en présence de peu de données, puisque rien ne pénalise les valeurs de θ qui ajustent excessivement le bruit d'échantillonnage c'est notamment ce mécanisme qui explique la propension de l'EM classique aux singularités décrite à la section 1.6.8.

Deuxième niveau : le maximum a posteriori (MAP). Ce niveau intermédiaire introduit un prior $p(\theta)$, mais continue de ne retenir qu'une estimation ponctuelle : $\hat{\theta}_\text{MAP} = \text{argmax}_\theta\ p(\theta|X) = \text{argmax}_\theta\ [p(X|\theta)\ p(\theta)]$. Le MAP peut se voir comme une régularisation du MLE par exemple, un prior gaussien centré en zéro sur les paramètres d'un modèle linéaire donne lieu, au MAP, à la régularisation dite de Tikhonov (ridge regression). Il serait possible de définir un « EM-MAP », qui maximiserait à l'étape M non plus la fonction $Q(\theta|\theta^t)$ seule mais $Q(\theta|\theta^t) + \log p(\theta)$, procédure hybride qui n'a pas été développée dans ce mémoire mais qui constitue un point intermédiaire naturel entre l'EM classique et l'EM variationnel complet.

Troisième niveau : l'inférence bayésienne complète. C'est le niveau auquel opère l'EM variationnel : on ne retient plus une estimation ponctuelle de θ, mais on caractérise sa distribution a posteriori complète $p(\theta|X)$ ou, dans le cas de l'EM variationnel, son approximation $q^*(\theta)$. Ce troisième niveau seul permet la quantification de l'incertitude mise en avant à la section 1.7.8 : les deux premiers niveaux, MLE et MAP, ne produisant chacun qu'un unique point de l'espace des paramètres, ne permettent structurellement aucune caractérisation de l'incertitude associée à ce point.

Le tableau 2.2 synthétise cette hiérarchie à trois niveaux, qui complète la compa-

raison à deux termes (classique / variationnel) menée jusqu'ici dans ce chapitre en y insérant le niveau intermédiaire du MAP, non développé algorithmiquement dans ce mémoire mais utile à la compréhension d'ensemble du paysage des méthodes d'estimation pour les modèles à variables latentes.

Niveau	**Objectif optimisé**	**Sortie**	**Algorithme correspondant**
MLE	argmax_θ p(X|θ)	Un point $\hat{\theta}$	EM classique (chapitre 1.6)
MAP	argmax_θ p(X|θ)p(θ)	Un point $\hat{\theta}$ régularisé	« EM-MAP » (non développé ici)
Bayes complet	Caractériser p(θ|X) en entier	Une distribution q*(θ)	EM variationnel (chapitre 1.7)

Table 2.2: Les trois niveaux d'inférence statistique pour les modèles à variables latentes

2.2 Comparaison algorithmique

2.2.1 Complexité computationnelle

Sur le plan algorithmique, les deux méthodes partagent la même structure itérative en deux étapes, mais diffèrent dans le détail du coût de chaque étape. Le tableau 2.3 détaille la complexité de chaque étape pour un mélange de K gaussiennes en dimension D, sur un jeu de données de taille N, avec covariances pleines.

Étape	**EM classique**	**EM variationnel**
Étape E	O(NKD^2) (formes quadratiques), plus O(KD^3) pour les factorisations de Cholesky	O(NKD^2), plus O(KD) évaluations de la fonction digamma
Étape M	O(NKD^2)	O(NKD^2), plus O(KD^3) pour l'inversion des W_k
Calcul du critère de convergence	O(NK) (log-vraisemblance, déjà calculée à l'étape E)	O($NK + KD^3$) (ELBO complet, statistiques déjà calculées)
Complexité totale par itération	O($NKD^2 + KD^3$)	O($NKD^2 + KD^3$)
Nombre d'itérations typique	Faible à modéré	Modéré à élevé (dépend de Kmax et des priors)

Table 2.3: Complexité algorithmique comparée (par itération)

Les deux algorithmes ont donc le même ordre de grandeur asymptotique par ité-

ration : le terme en KD^3, lié aux factorisations ou inversions de matrices $D \times D$, est indépendant de N et commun aux deux méthodes. Le surcoût observé de l'EM variationnel provient de facteurs constants plus élevés (fonctions digamma, déterminants, inversions explicites, calcul de l'ELBO à chaque itération) et surtout d'un nombre d'itérations plus grand, qui dépend du critère d'arrêt et des priors (sections 2.3.4 et 2.3.13). En pratique, pour les valeurs de D rencontrées dans la plupart des applications ($D \leq 50$), ce surcoût reste acceptable au regard des bénéfices obtenus en matière de sélection de modèle.

2.2.2 Comportement numérique et stabilité

Un autre axe de comparaison important concerne le comportement numérique des deux algorithmes. L'EM classique, en l'absence de régularisation, est sujet à des singularités : lorsqu'une composante s'ajuste à un très petit nombre de points, voire à un unique point, sa variance peut s'effondrer vers zéro, entraînant une divergence de la log-vraisemblance vers l'infini. Ce phénomène impose en pratique l'ajout d'une régularisation ad hoc de la covariance (par exemple $\Sigma_k \leftarrow \Sigma_k + \varepsilon I$).

L'EM variationnel, en revanche, est intrinsèquement protégé contre ce type de dégénérescence, grâce à la régularisation introduite par les priors conjugués : le prior Normal-Wishart empêche la matrice de précision Λ_k de diverger, et le prior de Dirichlet empêche les poids π_k de s'effondrer sur une seule observation. Cette robustesse accrue constitue l'un des arguments pratiques les plus solides en faveur de l'EM variationnel dans les contextes où la qualité et la propreté des données ne peuvent être garanties.

Un second aspect du comportement numérique concerne le conditionnement des matrices de covariance. L'évaluation de la densité gaussienne par inversion explicite de Σ_k devient numériquement instable lorsque cette matrice est proche de la singularité, situation qui survient lorsqu'une composante concentre son support sur un sous-espace de dimension inférieure à D. L'implémentation présentée au chapitre 3 évalue donc les densités à partir de la factorisation de Cholesky $\Sigma_k = LL^T$: le terme quadratique $(x-\mu_k)^T\Sigma_k^{-1}(x-\mu_k)$ est obtenu par résolution d'un système triangulaire et $\log|\Sigma_k|$ vaut $2\sum_j \log L_{jj}$. Cette technique, standard dans les bibliothèques de production, améliore la stabilité et réduit le coût, sans modifier le résultat mathématique de l'algorithme.

2.2.3 Critères de sélection de modèle

Lorsque l'on utilise l'EM classique, la sélection du nombre de composantes K repose généralement sur des critères d'information calculés a posteriori, après avoir exécuté l'algorithme pour plusieurs valeurs candidates de K. Les critères les plus couramment employés sont le critère d'information bayésien (BIC), le critère d'information d'Akaike

(AIC), et le critère de vraisemblance classifiante intégrée (ICL), définis respectivement par :

$$BIC(K) = -2\ell(\hat{\theta}K) + p(K)\log N \tag{2.4}$$

$$AIC(K) = -2\ell(\hat{\theta}K) + 2p(K) \tag{2.5}$$

où p(K) désigne le nombre de paramètres libres du modèle à K composantes. Ces critères pénalisent la complexité du modèle, mais nécessitent d'exécuter l'algorithme EM classique pour chaque valeur de K considérée, ce qui peut se révéler coûteux lorsque la plage de valeurs candidates est large. L'EM variationnel, en produisant directement un ELBO comparable entre modèles et en élaguant automatiquement les composantes superflues au sein d'une seule exécution avec Kmax fixé, offre une alternative particulièrement efficace à cette procédure de sélection externe.

Afin d'illustrer concrètement cette procédure de sélection externe, et de la comparer à l'approche endogène de l'EM variationnel, nous avons calculé le BIC et l'AIC de l'EM classique pour K variant de 1 à 8 sur le jeu de données à trois groupes réels utilisé tout au long de ce chapitre. La figure 2.1 présente les résultats obtenus.

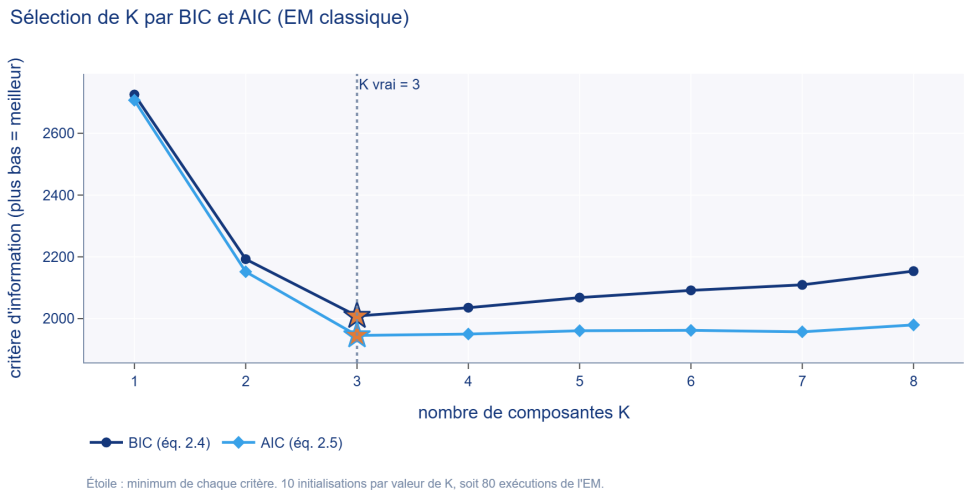


Figure 2.1: Critères BIC et AIC de l'EM classique en fonction du nombre de composantes K, sur le jeu de données à trois groupes réels.

Les deux critères atteignent leur minimum exactement à K = 3, ce qui correspond au nombre réel de groupes, confirmant que la procédure de sélection externe par BIC ou AIC fonctionne correctement sur ce jeu de données lorsqu'elle est appliquée avec soin. Ce résultat mérite cependant d'être mis en perspective : l'obtention de la figure 2.1 a nécessité l'exécution complète de l'EM classique pour chacune des huit valeurs de K considérées, soit huit ajustements indépendants (avec, de surcroît, dix initialisations chacun pour éviter les optima locaux, soit quatre-vingts exécutions au total),

alors que l'EM variationnel de la section 2.3.3 parvient à une conclusion équivalente trois composantes actives en une seule exécution avec Kmax = 6. Ce contraste illustre concrètement l'avantage pratique de la sélection endogène de l'EM variationnel évoqué à la section 2.2.3 ci-dessus : le gain n'est pas seulement conceptuel, il se traduit par un facteur d'économie de calcul proportionnel au nombre de valeurs de K testées lors d'une sélection par BIC ou AIC.

2.3 Étude expérimentale

2.3.1 Protocole expérimental

Afin d'illustrer concrètement les différences de comportement entre l'EM classique et l'EM variationnel mises en évidence sur le plan théorique, nous avons conduit une étude expérimentale fondée sur un jeu de données synthétique bidimensionnel. Ce choix méthodologique permet une visualisation directe des résultats et une interprétation sans ambiguïté des mécanismes en jeu.

Le jeu de données est constitué de N = 300 observations bidimensionnelles, générées à partir de trois groupes gaussiens, de centres respectifs (0, 0), (5, 5) et (0, 5), chacun comportant 100 observations et une covariance isotrope d'écart-type 0,7 (variance 0,49). Les centres sont distants de 5 à 7,1 unités, soit de 7 à 10 écarts-types : les trois groupes sont donc bien séparés. Ce jeu de données constitue, par construction, un mélange à trois composantes gaussiennes. Il est tiré avec la graine 42 ; la section 2.3.13 répète le protocole sur trente jeux indépendants.

L'expérience consiste à ajuster, sur ce même jeu de données, un GMM classique via l'algorithme EM classique et un GMM bayésien via l'algorithme EM variationnel, en imposant dans les deux cas un nombre de composantes délibérément surestimé, K = 6, soit le double du nombre réel de groupes. L'objectif est d'observer comment chacun des deux algorithmes gère cette surestimation. Deux jeux d'hyperparamètres sont considérés pour l'EM variationnel : des priors empiriques (centrés sur les données : $\beta_0 = 1$, W_0^{-1} égale à la covariance empirique) et des priors diffus ($\beta_0 = 10^{-3}$, $W_0 =$ I). Sauf mention contraire, les résultats rapportés utilisent les priors empiriques.

Paramètre	**Valeur**
Nombre d'observations (N)	300 (3 groupes de 100)
Dimension (D)	2
Nombre de composantes imposé (K)	6 (nombre réel de groupes : 3)
Covariance des groupes générateurs	Isotrope, écart-type 0,7 (variance 0,49)
Graine de génération des données	42
EM classique	Mélange gaussien, covariance pleine, initialisation par k-means
EM variationnel	GMM bayésien, covariance pleine, prior de Dirichlet fini de concentration α_0
Priors Normal-Wishart (empiriques)	m_0 = moyenne des données, $\beta_0 = 1$, $\nu_0 = \mathrm{D}$, W_0^{-1} = covariance empirique
Concentration du prior de Dirichlet (α_0)	1 / K = 0,167
Critère d'arrêt (ε)	10^{-3} (EM : variation de la log-vraisemblance par observation ; EM variationnel : variation totale de l'ELBO)
Nombre maximal d'itérations	200 (EM), 500 (EM variationnel)
Nombre d'initialisations	10 (meilleur critère retenu)
Composante active	$N_k = \sum_i r_{ik} \geq 1$

Table 2.4: Paramètres du protocole expérimental

Source : implémentation PyTorch décrite à l'annexe A.

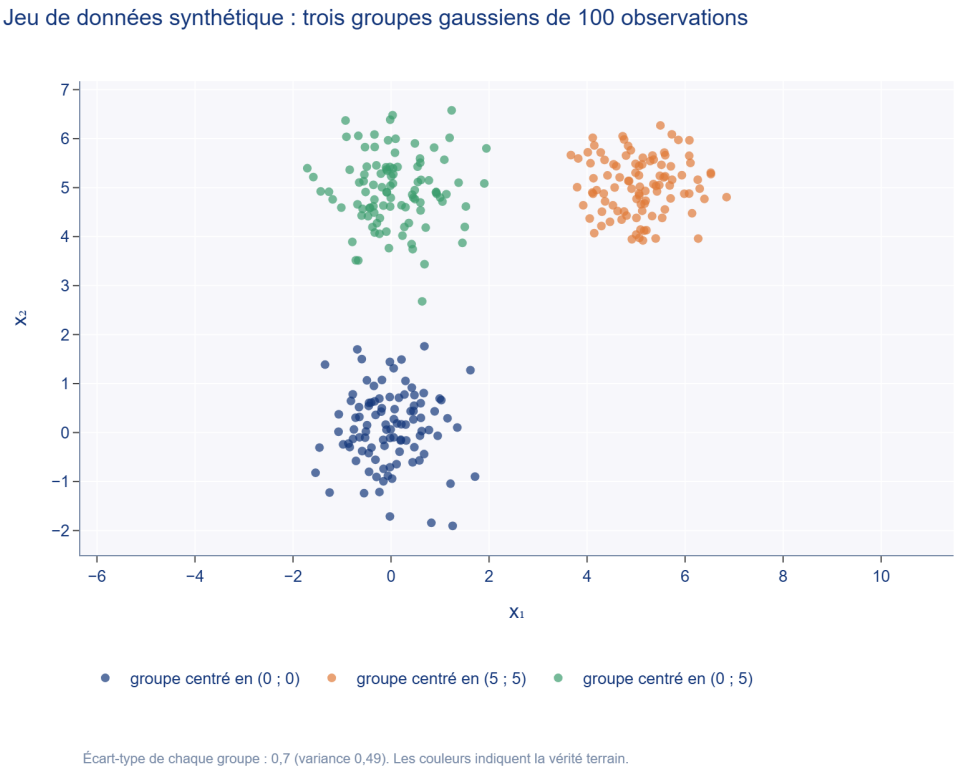


Figure 2.2: Jeu de données synthétique utilisé pour l'étude expérimentale : trois groupes gaussiens réels de 100 observations chacun (N = 300 au total).

2.3.2 Résultats de l'EM classique (K = 6 imposé)

L'exécution de l'EM classique avec K = 6 composantes converge en 8 itérations vers une log-vraisemblance totale de -946,00. Le tableau 2.5 présente les poids finaux obtenus pour chacune des six composantes.

Composante	**1**	**2**	**3**	**4**	**5**	**6**
Poids π_k	0,333	0,013	0,167	0,148	0,185	0,154

Table 2.5: Poids de mélange obtenus par l'EM classique (K = 6)

Ce résultat illustre la limite théorique décrite au chapitre 1 : en l'absence de mécanisme de régularisation sur le nombre de composantes, l'EM classique conserve ses six composantes. Le poids le plus faible, 0,013 (soit $N_k \approx 4$), reste supérieur au seuil d'activité $N_k = 1$: aucune composante n'est écartée. La composante 1, de poids 0,333, capture à elle seule l'un des trois groupes, tandis que les cinq autres composantes se partagent les deux groupes restants, pourtant bien séparés, en les fragmentant de façon arbitraire (figure 2.3). L'algorithme ne dispose d'aucun mécanisme interne pour reconnaître que K = 6 est surestimé par rapport à la structure réelle des données.

2.3.3 Résultats de l'EM variationnel (Kmax = 6)

L'exécution de l'EM variationnel avec Kmax = 6 composantes, des priors empiriques et un prior de Dirichlet de concentration $\alpha_0 = 1/\mathrm{K}$ converge en 25 itérations vers une ELBO de -1046,62, calculée exactement selon l'équation (1.31). Cette valeur est inférieure à la log-vraisemblance maximale de l'EM classique (-946,00), comme l'exige la chaîne d'inégalités ELBO $\leq \log p(X) \leq \max_\theta \log p(X|\theta)$: les deux quantités ne mesurent pas la même chose (l'ELBO intègre la pénalité de complexité due aux priors) et ne se comparent pas comme deux scores d'ajustement. Le tableau 2.6 présente les poids finaux obtenus.

Composante	**1**	**2**	**3**	**4**	**5**	**6**
Poids π_k	0,001	0,335	0,331	0,333	0,001	0,001

Table 2.6: Poids de mélange obtenus par l'EM variationnel (Kmax = 6)

Contrairement à l'EM classique, l'EM variationnel identifie nettement trois composantes dominantes, de poids 0,335, 0,331 et 0,333, très proches de la proportion réelle d'un tiers par groupe, tandis que les trois composantes restantes voient leur poids s'effondrer à 0,001, soit un nombre effectif de points assignés $N_k \approx 0,3$, en deçà du seuil d'activité $N_k < 1$ défini à la section 1.7.4. Ces trois dernières composantes sont donc éteintes et le modèle final ne conserve que trois composantes actives, en adéquation avec la structure du jeu de données, alors même que l'algorithme n'a jamais reçu cette information en entrée. La figure 2.3 illustre ce contraste : les ellipses de covariance de l'EM classique se répartissent de façon désordonnée sur deux des groupes, tandis que celles de l'EM variationnel épousent les trois groupes réels.

Ce résultat dépend des priors. Avec les priors diffus ($\beta_0 = 10^{-3}$, $W_0 = \mathrm{I}$), la même expérience conserve quatre composantes actives (poids 0,326 ; 0,333 ; 0,332 et 0,008, ARI de 0,985) et demande 103 itérations. La section 2.3.13 quantifie cette dépendance sur trente jeux de données.

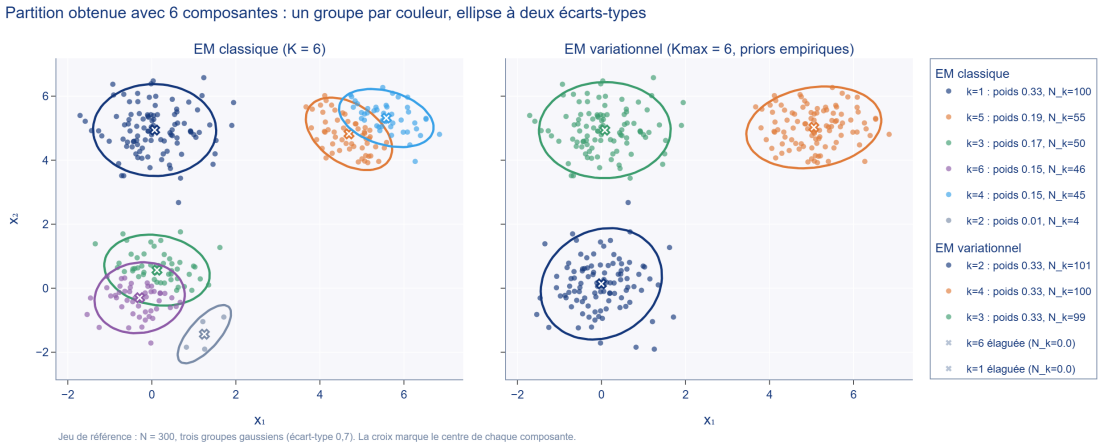


Figure 2.3: Partition obtenue avec six composantes : une couleur par composante, ellipses de covariance à deux écarts-types, croix aux centres. À gauche, l'EM classique conserve ses six composantes ; à droite, l'EM variationnel n'en retient que trois (les composantes élaguées figurent en gris dans la légende).

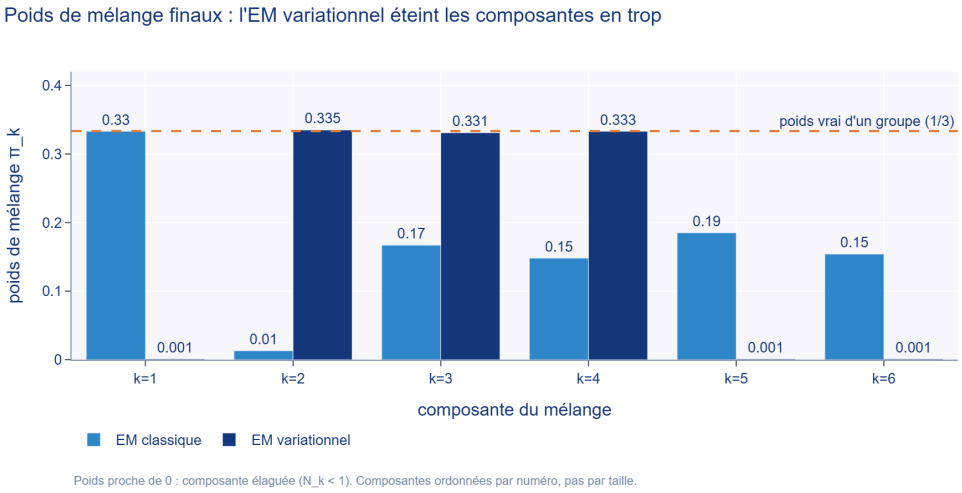


Figure 2.4: Poids de mélange π_k finaux pour les six composantes : EM classique et EM variationnel. Le contraste traduit le mécanisme d'élagage bayésien décrit à la section 1.7.4.

2.3.4 Comparaison visuelle et qualité de partition

Au-delà des poids de mélange, la qualité des partitions obtenues par les deux algorithmes peut être évaluée à l'aide de métriques externes de clustering, en comparant les affectations produites aux véritables étiquettes de groupe. Le tableau 2.7 présente les résultats obtenus pour l'indice de Rand ajusté (ARI) et le score de silhouette, calculés sur les affectations dures obtenues en prenant, pour chaque observation, la composante de responsabilité maximale.

Métrique	EM classique	EM variationnel
Nombre de composantes effectivement actives	6	3
Indice de Rand ajusté (ARI)	0,715	0,990
Score de silhouette moyen	0,450	0,739
Nombre d'itérations jusqu'à convergence	8	25
Temps moyen d'exécution (10 initialisations, 30 jeux)	0,21 s	0,98 s

Table 2.7: Métriques de qualité de partition (K = 6 imposé)

Source : implémentation PyTorch (graine 42). Les temps sont des moyennes sur les 30 jeux de la section 2.3.13.

Ces résultats appellent plusieurs commentaires. Premièrement, l'indice de Rand ajusté atteint 0,990 pour l'EM variationnel, c'est-à-dire une partition quasi identique aux trois groupes ayant servi à générer les données (seules quelques observations de frontière sont affectées différemment), sans qu'aucune information sur le nombre réel de groupes n'ait été fournie, si ce n'est la borne supérieure Kmax = 6. L'EM classique, avec un ARI de 0,715, produit une partition nettement moins fidèle, ses six composantes fragmentant artificiellement les groupes réels. Deuxièmement, le score de silhouette, qui mesure la cohésion interne et la séparation des clusters indépendamment de toute étiquette de référence, confirme ce constat : 0,739 pour l'EM variationnel contre 0,450 pour l'EM classique. Troisièmement, le nombre d'itérations est environ trois fois plus élevé pour l'EM variationnel (25 contre 8) et le temps d'exécution environ 4,7 fois supérieur, ce qui illustre le compromis computationnel identifié à la section 2.2.1. Le nombre d'itérations dépend toutefois de la normalisation du critère d'arrêt (par observation pour l'EM, totale pour l'EM variationnel) et ne doit pas être lu comme une propriété intrinsèque des algorithmes.

2.3.5 Sensibilité au choix du prior de Dirichlet (α_0)

La section 1.7.4 a montré, sur le plan théorique, que le paramètre de concentration α_0 du prior de Dirichlet contrôle l'intensité du mécanisme d'élagage automatique : plus α_0 est petit, plus le prior favorise les solutions parcimonieuses. Afin de vérifier empiriquement cette prédiction, nous avons répété l'expérience de la section 2.3.3 en faisant varier α_0 sur une grille logarithmique $\{0{,}001 ; 0{,}01 ; 0{,}1 ; 1 ; 10\}$, toutes choses égales par ailleurs (Kmax = 6, mêmes données, dix initialisations), pour les deux jeux de priors. Le tableau 2.8 et la figure 2.5 présentent les résultats obtenus.

α_0	**0,001**	**0,01**	**0,1**	**1**	**10**
Composantes actives, priors empiriques	3	3	3	3	6
Composantes actives, priors diffus	3	3	3	4	6
ARI, priors empiriques	0,990	0,990	0,990	0,990	0,960
ARI, priors diffus	1,000	1,000	1,000	0,981	0,618

Table 2.8: Nombre de composantes actives ($N_k \geq 1$) et ARI en fonction de la concentration α_0 du prior de Dirichlet

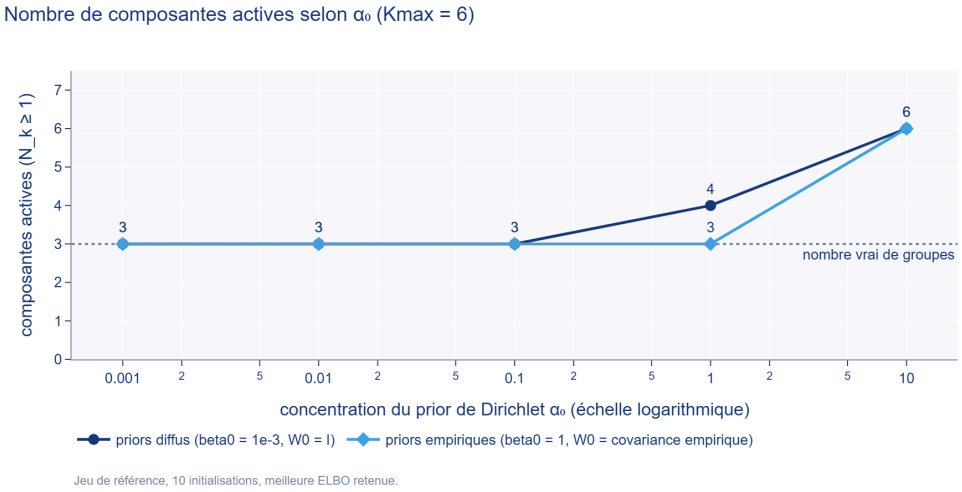


Figure 2.5: Nombre de composantes actives en fonction de la concentration α_0 du prior de Dirichlet (échelle logarithmique), pour les deux jeux de priors. Le mécanisme d'élagage disparaît lorsque α_0 devient grand.

Ces résultats confirment la prédiction théorique : pour $\alpha_0 \leq 0{,}1$, l'algorithme identifie exactement trois composantes actives avec les deux jeux de priors. L'élagage disparaît lorsque α_0 atteint 10 : les six composantes redeviennent actives et la qualité de la partition se dégrade (ARI de 0,960 avec les priors empiriques, 0,618 avec les priors diffus). Le comportement à $\alpha_0 = 1$, concentration pour laquelle le prior de Dirichlet est uniforme et n'encode plus de préférence pour la parcimonie, est plus nuancé que ne le laissait attendre la théorie : l'élagage persiste (trois composantes actives avec les priors empiriques, quatre avec les priors diffus). Le prior de Dirichlet n'est donc pas le seul mécanisme à l'œuvre ; le prior Normal-Wishart contribue lui aussi à pénaliser les composantes superflues, par le terme de divergence de Kullback-Leibler de l'ELBO (équation 1.22), interprétation compatible avec ces observations mais non testée isolément ici. Pour le praticien, α_0 reste un véritable levier de contrôle de la parcimonie, qu'il convient de fixer à une valeur faible (typiquement $\alpha_0 = 1/\text{Kmax}$ ou inférieure) si

l'objectif est la sélection automatique du nombre de composantes.

2.3.6 Étude de la scalabilité computationnelle

Afin de quantifier le surcoût computationnel de l'EM variationnel identifié sur le plan théorique à la section 2.2.1, nous avons mesuré le temps d'exécution des deux algorithmes en faisant varier, d'une part, le nombre d'observations N (à dimension D = 2 fixée), et d'autre part, la dimension D (à N = 2000 fixé), avec trois initialisations et au plus 100 itérations par ajustement. Le tableau 2.9 et la figure 2.6 présentent les résultats obtenus.

Paramètre variable	**Valeur**	**EM classique (s)**	**EM variationnel (s)**	**Ratio (var./class.)**
N (D=2)	300	0,123	0,839	6,8
N (D=2)	1 000	0,120	1,390	11,6
N (D=2)	5 000	0,420	2,377	5,7
N (D=2)	20 000	0,856	4,102	4,8
D (N=2000)	2	0,207	1,717	8,3
D (N=2000)	5	0,182	1,504	8,3
D (N=2000)	10	0,202	1,476	7,3
D (N=2000)	20	0,481	1,122	2,3

Table 2.9: Temps d'exécution (en secondes) en fonction de N et de D

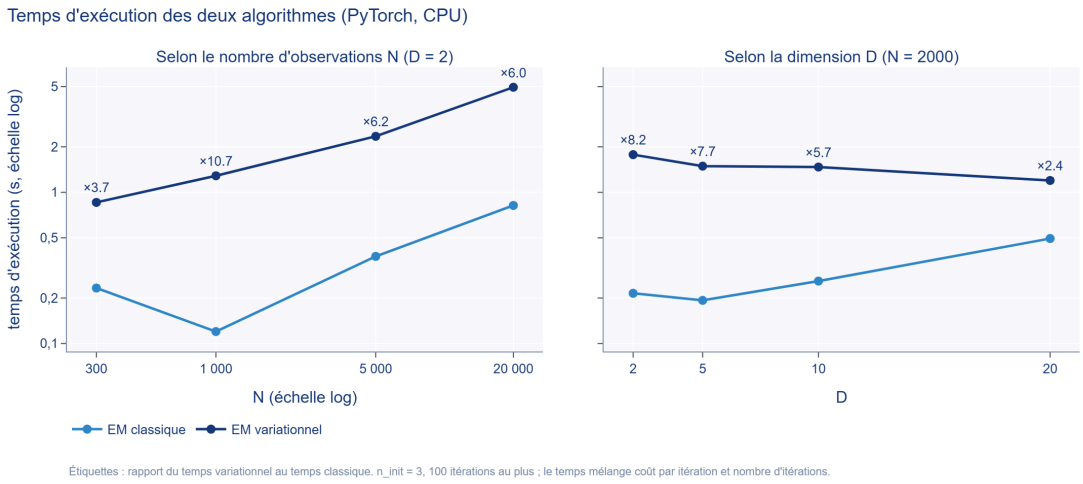


Figure 2.6: Temps d'exécution des deux algorithmes en fonction du nombre d'observations N (à gauche, échelles logarithmiques) et de la dimension D (à droite). Les étiquettes donnent le rapport du temps variationnel au temps classique.

Source : implémentation PyTorch (CPU, machine unique, graine 42).

Ces mesures confirment que l'EM variationnel demeure plus coûteux que l'EM classique, avec un facteur multiplicatif compris entre 2,3 et 11,6 selon les régimes. Le rapport ne varie pas de façon monotone avec N (6,8 ; 11,6 ; 5,7 ; 4,8) : le point à N = 1 000 est vraisemblablement affecté par le bruit de mesure, mais le rapport décroît nettement pour N grand. En fonction de D, il reste voisin de 8 jusqu'à D = 10 puis tombe à 2,3 pour D = 20. Cette dernière chute vient surtout de l'EM variationnel, dont le temps diminue (1,1 s contre 1,7 s à D = 2) alors que celui de l'EM classique augmente : le nombre d'itérations, non contrôlé, diffère d'une configuration à l'autre, et ces mesures ne suffisent pas à elles seules à valider l'analyse de complexité de la section 2.2.1. Les temps mesurés combinent toutefois le coût par itération et le nombre d'itérations, qui n'est pas contrôlé, et la mesure unique par configuration est bruitée (le temps de l'EM classique à N = 1 000 est ainsi égal à celui mesuré à N = 300). En valeur absolue, les deux algorithmes restent d'un coût raisonnable sur les tailles considérées (environ 4 secondes pour 20 000 observations), et le choix entre eux devrait être dicté davantage par la qualité de la solution recherchée que par cette seule considération de temps de calcul.

2.3.7 Cas de référence : nombre de composantes correctement spécifié (K = 3)

L'ensemble des résultats précédents portent sur le cas, volontairement défavorable à l'EM classique, où le nombre de composantes est surestimé. Afin de vérifier que les deux algorithmes se comportent de façon équivalente lorsque cette surestimation est absente, nous avons répété l'expérience avec K = 3, c'est-à-dire avec le nombre exact de groupes réels du jeu de données. Le tableau 2.10 présente les résultats obtenus.

Métrique	**EM classique**	**EM variationnel**
Poids de mélange	0,333 / 0,334 / 0,332	0,333 / 0,335 / 0,331
Indice de Rand ajusté (ARI)	1,000	0,990
Score de silhouette	0,740	0,739
Nombre d'itérations	2	4

Table 2.10: Résultats de référence avec le nombre de composantes correctement spécifié (K = 3)

Ce cas de référence est riche d'enseignement : lorsque le nombre de composantes est correctement spécifié, les deux algorithmes convergent en 2 et 4 itérations vers des solutions quasi identiques (ARI de 1,000 et 0,990, silhouettes de 0,740 et 0,739). L'écart d'ARI correspond à la régularisation exercée par les priors sur les observations de frontière. Ce résultat confirme que la supériorité de l'EM variationnel observée aux sections 2.3.3 et 2.3.4 n'est pas une propriété intrinsèque qui le rendrait préférable en

toute circonstance ; elle ne se manifeste que lorsque le nombre réel de composantes est inconnu et que le nombre utilisé pour l'ajustement s'en écarte. Ce constat rejoint la synthèse théorique du tableau 2.1 : le choix entre les deux algorithmes doit être guidé par le degré de confiance dont dispose le praticien quant au nombre réel de composantes sous-jacentes à ses données.

2.3.8 Convergence de l'ELBO

La section 1.7.6 a établi que l'ELBO constitue, pour l'EM variationnel, un critère de convergence croissant et borné. Afin d'illustrer cette propriété sur notre étude de cas, la figure 2.7 retrace l'évolution de l'ELBO, calculée exactement à chaque itération selon l'équation (1.31), pour le modèle à Kmax = 6 composantes de la section 2.3.3, avec un critère d'arrêt de 10^{-6}. Elle présente aussi la log-vraisemblance de l'EM classique pour K = 6.

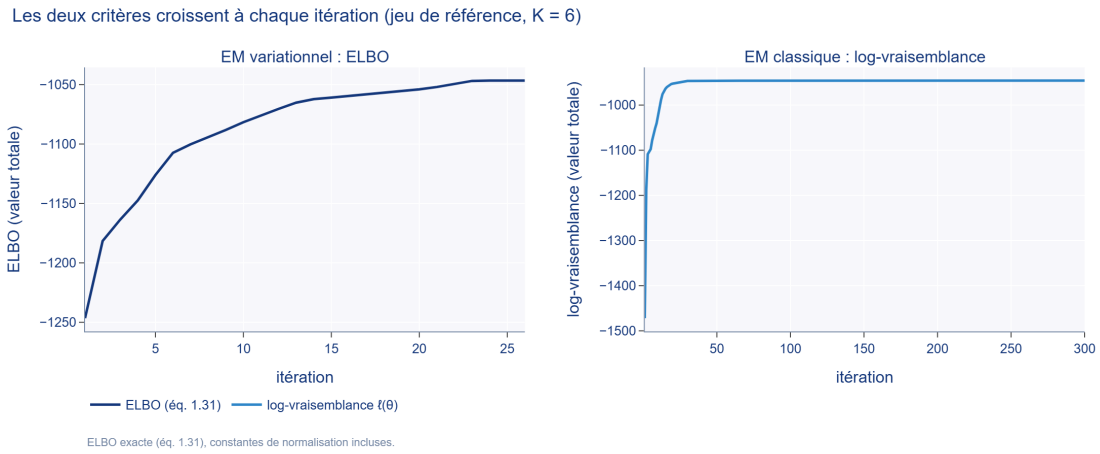


Figure 2.7: Évolution de l'ELBO (EM variationnel, à gauche) et de la log-vraisemblance (EM classique, à droite) en fonction du nombre d'itérations. Les deux courbes sont croissantes, conformément aux garanties théoriques des sections 1.6.2.f et 1.7.6.

On observe une croissance strictement monotone de l'ELBO, qui passe de -1246,54 à la première itération à -1046,62 à convergence (26 itérations), sans jamais décroître, conformément à la garantie théorique de la section 1.7.6. Cette monotonie a été vérifiée numériquement à chaque itération, de même que l'inégalité ELBO $\leq \log p(X) \leq \ell(\hat{\theta})$ et l'égalité de Neal et Hinton, $F(q^*, \theta) = \ell(\theta)$ après l'étape E, vérifiée à 10^{-13} près. L'ELBO gagne environ 140 unités en six itérations (de -1246,54 à environ -1107), puis progresse plus lentement et se stabilise vers la vingt-troisième itération. La log-vraisemblance de l'EM classique atteint son plateau en une trentaine d'itérations ; sa courbe est tracée jusqu'à 300 itérations parce que le critère d'arrêt de 10^{-6} est très strict.

2.3.9 Robustesse à l'initialisation

L'ensemble des résultats précédents ont été obtenus avec dix initialisations indépendantes, dont la meilleure solution est conservée, pratique recommandée pour se prémunir contre la convergence vers un optimum local médiocre (sections 1.6.3.a et 1.7.9). Afin de quantifier la sensibilité propre de chaque algorithme à l'initialisation, nous avons répété l'expérience de la section 2.3.1 avec une seule initialisation par exécution, pour 20 graines indépendantes et deux stratégies : initialisation par k-means, et initialisation aléatoire (K points de données tirés au hasard, section 1.6.2.a pour l'EM classique et section 1.7.3.d pour l'EM variationnel).

Initialisation	**Algorithme**	**3 actives**	**4 actives**	**5 actives**	**6 actives**
k-means	EM classique	0 / 20	0 / 20	0 / 20	20 / 20
k-means	EM variationnel, priors diffus	0 / 20	12 / 20	8 / 20	0 / 20
k-means	EM variationnel, priors empiriques	20 / 20	0 / 20	0 / 20	0 / 20
aléatoire	EM classique	0 / 20	1 / 20	1 / 20	18 / 20
aléatoire	EM variationnel, priors diffus	4 / 20	14 / 20	2 / 20	0 / 20
aléatoire	EM variationnel, priors empiriques	20 / 20	0 / 20	0 / 20	0 / 20

Table 2.11: Nombre de composantes actives obtenu sur 20 initialisations indépendantes (une initialisation par exécution)

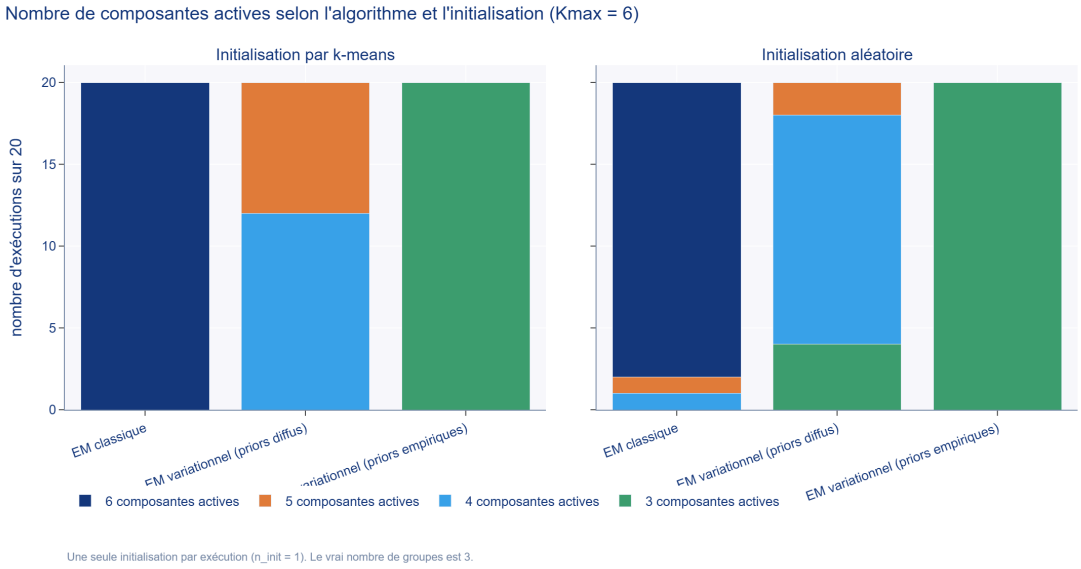


Figure 2.8: Répartition du nombre de composantes actives selon l'algorithme et la stratégie d'initialisation (20 exécutions chacune, une initialisation par exécution).

Avec les priors empiriques, l'EM variationnel retrouve exactement trois composantes

actives dans 20 exécutions sur 20, quelle que soit la stratégie d'initialisation. Avec les priors diffus, le résultat est nettement plus sensible à l'initialisation : trois composantes seulement dans 4 exécutions sur 20 avec l'initialisation aléatoire, aucune avec k-means, les autres exécutions conservant quatre ou cinq composantes (une composante supplémentaire de faible poids n'étant pas complètement éliminée). L'EM classique conserve ses six composantes dans 20 exécutions sur 20 avec k-means et dans 18 sur 20 avec l'initialisation aléatoire ; dans les deux exécutions restantes, une composante s'effondre ($N_k < 1$), ce qui est un accident d'optimisation et non une sélection de modèle. La log-vraisemblance finale de l'EM classique varie entre -953,29 et -948,43 (écart-type 1,55) avec k-means et entre -955,67 et -944,12 (écart-type 2,35) avec l'initialisation aléatoire, ce qui confirme sa sensibilité aux optima locaux (section 1.6.3.a) sans résoudre le problème de fond : à K fixé à 6, aucune initialisation ne permet à l'EM classique de reconnaître que trois composantes suffiraient. Le mécanisme d'élagage de l'EM variationnel est donc robuste à l'initialisation lorsque les priors sont adaptés à l'échelle des données, et fragile lorsqu'ils ne le sont pas.

2.3.10 Sensibilité au type de covariance

L'ensemble des expériences précédentes ont été conduites avec des matrices de covariance pleines (full), c'est-à-dire sans contrainte sur la forme des ellipses de covariance de chaque composante. En pratique, il est fréquent de contraindre cette forme, tant pour réduire le nombre de paramètres libres et le risque de surapprentissage que pour accélérer les calculs. Les quatre contraintes les plus courantes, par ordre décroissant de flexibilité, sont : full (une matrice pleine par composante), tied (une unique matrice pleine partagée par toutes les composantes), diag (une matrice diagonale par composante) et spherical (une variance scalaire par composante). Pour l'EM variationnel, les variantes tied, diag et spherical ne sont pas dérivées au chapitre 1 ; elles utilisent un prior de Wishart partagé (tied) ou des priors Gamma sur les précisions (diag, spherical), également conjugués. Le tableau 2.12 reproduit l'expérience de la section 2.3.4 pour chacune de ces contraintes.

Type de covariance	**ARI : EM classique**	**ARI : EM variationnel**	**Composantes actives (variationnel)**
full	0,715	0,990	3
tied	0,685	1,000	3
diag	0,727	0,990	3
spherical	0,726	1,000	3

Table 2.12: Indice de Rand ajusté et nombre de composantes actives selon le type de covariance (K = 6 imposé)

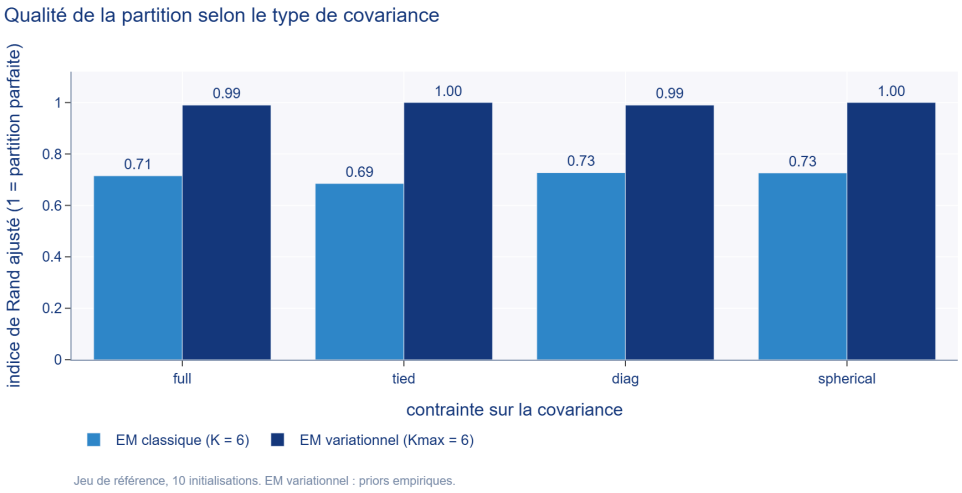


Figure 2.9: Indice de Rand ajusté selon la contrainte imposée sur la covariance (K = 6 imposé).

Ce résultat montre que le mécanisme d'élagage automatique de l'EM variationnel ne dépend pas de la forme de covariance retenue : dans les quatre cas, il identifie exactement les trois composantes attendues, avec un ARI compris entre 0,990 et 1,000. L'EM classique voit son ARI varier entre 0,685 et 0,727 selon la contrainte choisie, sans jamais atteindre la qualité de partition de l'EM variationnel. L'élagage n'est donc pas une particularité du cas des covariances pleines, mais une propriété du cadre bayésien variationnel, au moins pour les priors empiriques utilisés ici.

2.3.11 Validation sur un jeu de données réel (Iris)

L'ensemble des résultats précédents ont été obtenus sur un jeu synthétique, construit avec trois groupes gaussiens bien séparés. Afin de tester la robustesse des conclusions dans un cadre moins idéalisé, nous reproduisons l'expérience centrale de la section 2.3.1 sur le jeu de données Iris de Fisher (1936), qui comporte N = 150 observations en dimension D = 4 (longueur et largeur des pétales et des sépales), réparties en trois espèces réelles (Iris setosa, versicolor et virginica), avec K = 6 composantes imposées.

Algorithme	Poids de mélange (arrondis)	ARI	Silhouette	Actives
EM classique, K = 6	0,333 / 0,214 / 0,020 / 0,053 / 0,204 / 0,175	0,641	0,446	6
EM variationnel, Kmax = 6	0,007 / 0,073 / 0,001 / 0,001 / 0,585 / 0,332	0,562	0,276	3
EM classique, K = 3	0,333 / 0,364 / 0,303	0,904	0,501	3
EM variationnel, Kmax = 3	0,190 / 0,333 / 0,476	0,684	0,400	3

Table 2.13: Résultats sur le jeu de données réel Iris (N = 150, D = 4)

Source : Iris de Fisher (1936), fichier UCI Machine Learning Repository ; calculs de l'auteur.

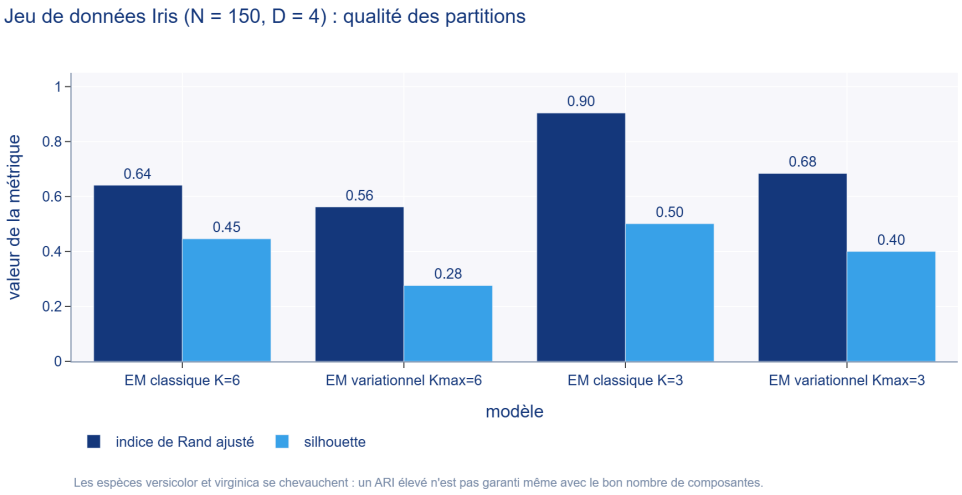


Figure 2.10: Indice de Rand ajusté et silhouette sur le jeu Iris, selon l'algorithme et le nombre de composantes.

Ce résultat, obtenu sur des données réelles, nuance la conclusion univoque de la section 2.3.4. À Kmax = 6, l'EM variationnel élague trois des six composantes mais son indice de Rand ajusté (0,562) est inférieur à celui de l'EM classique (0,641). Les poids finaux donnent la raison de cet écart : une composante de poids 0,585, proche de 2/3, regroupe les espèces versicolor et virginica, et une composante de poids 0,332, proche de 1/3, isole setosa. L'élagage fusionne donc les deux espèces qui se chevauchent, ce qui est cohérent avec le critère d'évidence puisque ces espèces ne forment pas deux nuages gaussiens séparés dans l'espace des quatre mesures, alors que les étiquettes de référence les distinguent. L'algorithme décide sur la base du support statistique observé dans les données, et non du nombre d'étiquettes disponibles : lorsque la structure réelle s'écarte de l'hypothèse d'un mélange de gaussiennes bien séparées, la solution de l'EM

variationnel peut légitimement s'écarter du nombre d'étiquettes de référence, sans que cela constitue une défaillance de l'algorithme.

Avec Kmax = 3, l'EM variationnel obtient un ARI de 0,684, supérieur à celui de Kmax = 6 mais inférieur au 0,904 de l'EM classique à K = 3 exactement spécifié. Sur ce jeu réel, la meilleure stratégie reste donc, si l'on a une bonne raison de croire que K = 3 (ce que l'on sait ici grâce à la vérité terrain, mais qui ne serait pas connu en analyse exploratoire), d'utiliser l'EM classique avec ce K plutôt que de compter sur l'élagage pour le découvrir. Cette nuance, absente de l'étude synthétique, rappelle que la supériorité de l'EM variationnel, démontrée sur données synthétiques bien séparées, doit être relativisée sur des données réelles où les groupes ne sont pas des gaussiennes bien séparées.

2.3.12 Sensibilité aux valeurs aberrantes

Une dernière expérience examine le comportement des deux algorithmes en présence de valeurs aberrantes, situation fréquente en pratique et distincte de la simple surestimation de K. Dix points, tirés uniformément dans le carré $[-10; 15]^2$ (graine 7), ont été ajoutés au jeu de données de la section 2.3.1, puis les deux algorithmes ont été ajustés avec K = 6 sur ce jeu augmenté (N = 310).

Métrique	**EM classique**	**EM variationnel**
ARI (calculé hors valeurs aberrantes)	0,990	0,990
Composantes actives	6	5
Trace maximale des covariances	6,2	98,4
Plus petit poids de mélange	0,0065	0,0108

Table 2.14: Comportement des deux algorithmes en présence de 10 valeurs aberrantes ajoutées (N = 310, K = 6)

Les deux algorithmes retrouvent une partition quasi parfaite sur les points non aberrants (ARI de 0,990), ce qui montre qu'aucun n'est déstabilisé par un petit nombre de valeurs aberrantes dispersées. Les stratégies d'absorption diffèrent toutefois. L'EM classique répartit les valeurs aberrantes sur de petites composantes (poids minimal 0,0065) sans élargir aucune composante (trace maximale de 6,2). L'EM variationnel en absorbe plusieurs dans une composante diffuse, dont la trace de covariance (98,4) est environ seize fois supérieure, et conserve cinq composantes actives plutôt que trois. Ce comportement est compatible avec l'effet régularisant du prior Normal-Wishart, qui favorise l'élargissement d'une composante existante plutôt que la création de composantes quasi ponctuelles, mais cette expérience, fondée sur un seul tirage des valeurs aberrantes, ne

permet pas de l'établir. Elle montre en revanche que la robustesse de l'EM variationnel n'est pas une insensibilité aux données aberrantes : le modèle conserve une composante supplémentaire non strictement nécessaire à la description du signal principal.

2.3.13 Réplication sur des jeux de données indépendants

Les sections précédentes reposent sur un jeu de données unique. Pour chiffrer l'incertitude des conclusions, le protocole de la section 2.3.1 (K = 6 imposé, dix initialisations par ajustement) a été répété sur R = 30 jeux de données indépendants, tirés avec les graines 1000 à 1029. Trois hypothèses, fixées avant l'analyse, sont testées : (H1) l'ARI moyen de l'EM variationnel dépasse celui de l'EM classique, ce que l'on décide si l'intervalle de confiance bootstrap à 95 % de la différence appariée est strictement positif ; (H2) l'EM variationnel retrouve exactement trois composantes actives dans la majorité des jeux, ce que l'on décide si la borne basse de l'intervalle de Wilson à 95 % dépasse 0,5 ; (H3) ces résultats dépendent du choix des priors. Les moyennes sont accompagnées d'intervalles de confiance bootstrap (5 000 rééchantillonnages) et les proportions d'intervalles de Wilson.

Algorithme	**ARI moyen [IC 95 %]**	**Silh. moy.**	**Part de jeux à 3 actives [IC 95 %]**	**Itér. moy.**	**Temps (s)**
EM classique	0,693 [0,677 ; 0,709]	0,456	0 % [0 ; 11 %] (0 sur 30)	9,2	0,21
EM variationnel, priors diffus	0,971 [0,953 ; 0,988]	0,709	73 % [56 ; 86 %] (22 sur 30)	118,8	4,04
EM variationnel, priors empiriques	0,999 [0,998 ; 1,000]	0,746	100 % [89 ; 100 %] (30 sur 30)	25,8	0,98

Table 2.15: Résultats moyens sur 30 jeux de données indépendants (K = 6 imposé)

Comparaison (même jeu de données)	**Différence moyenne d'ARI**	**IC 95 % (bootstrap)**	**Jeux favorables à l'EM variationnel**
EM variationnel, priors diffus, moins EM classique	+0,278	[0,258 ; 0,298]	30 sur 30
EM variationnel, priors empiriques, moins EM classique	+0,307	[0,290 ; 0,322]	30 sur 30

Table 2.16: Différences appariées d'indice de Rand ajusté entre EM variationnel et EM classique

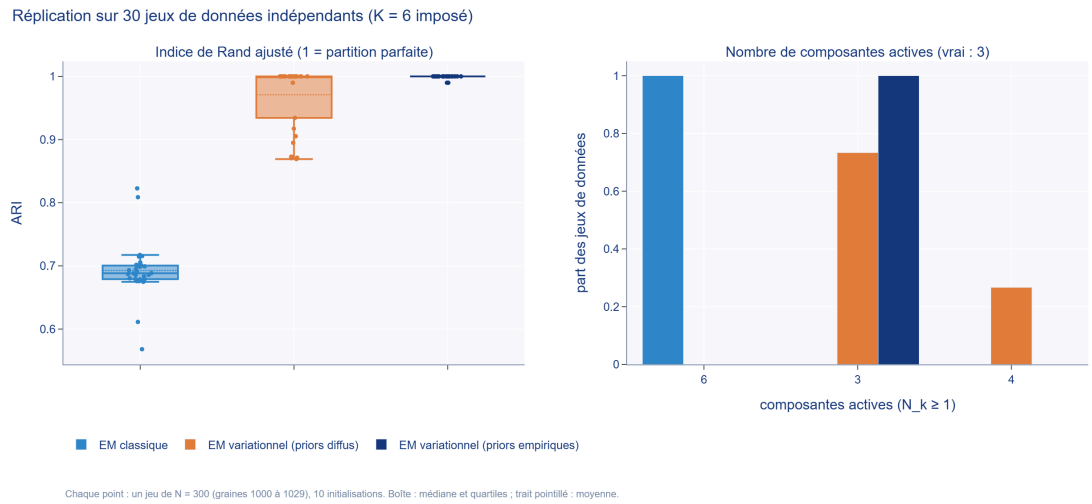


Figure 2.11: Distribution de l'indice de Rand ajusté (à gauche, un point par jeu de données) et du nombre de composantes actives (à droite) sur 30 jeux de données indépendants.

H1 est soutenue pour les deux jeux de priors : l'EM variationnel obtient un ARI supérieur à celui de l'EM classique sur chacun des 30 jeux, avec une différence moyenne de +0,28 (priors diffus) à +0,31 (priors empiriques) et des intervalles de confiance éloignés de zéro. H2 est soutenue pour les deux jeux de priors, mais avec des marges très différentes : 100 % des jeux (borne basse de Wilson 89 %) avec les priors empiriques, 73 % (borne basse 56 %) avec les priors diffus. H3 est confirmée : le choix des priors modifie l'ARI moyen (0,971 contre 0,999), le nombre d'itérations (119 contre 26) et le temps de calcul (4,0 s contre 1,0 s). L'EM classique, lui, ne retrouve jamais trois composantes (0 sur 30) puisqu'il conserve par construction les six composantes imposées. La variabilité entre jeux est faible pour les priors empiriques (écart-type de l'ARI de 0,003) et plus élevée pour les priors diffus (0,050), ce qui rejoint le constat de la section 2.3.9.

2.4 Analyse des résultats et synthèse

L'étude expérimentale conduite dans ce chapitre confirme, sur un exemple concret et reproductible, l'ensemble des différences théoriques et algorithmiques mises en évidence précédemment. Lorsque le nombre de composantes est correctement spécifié, les deux algorithmes produisent des résultats de qualité comparable, l'EM classique bénéficiant alors d'un net avantage en termes de rapidité d'exécution. En revanche, dès lors que le nombre de composantes est surestimé situation extrêmement fréquente en pratique, où le nombre réel de groupes sous-jacents est rarement connu avec certitude , l'EM variationnel démontre un avantage net et reproductible sur trente jeux de données (section 2.3.13) grâce à son mécanisme d'élagage automatique, sous réserve d'un choix de priors adapté à l'échelle des données, produisant une partition à la fois plus parcimonieuse et plus fidèle à la structure réelle des données, sans qu'il soit nécessaire de relancer l'al-

gorithme pour plusieurs valeurs de K ni de recourir à des critères de sélection externes tels que le BIC ou l'AIC.

Le tableau 2.17 propose une synthèse globale, intégrant l'ensemble des dimensions de comparaison abordées dans ce chapitre, et servira de référence pour la discussion des applications pratiques menée au chapitre 3.

Critère	**EM classique**	**EM variationnel**
Simplicité conceptuelle	Élevée	Modérée
Coût computationnel	Faible	Modéré à élevé
Nécessité de fixer K a priori	Oui	Non (Kmax suffit)
Robustesse aux singularités	Faible (sans régularisation)	Élevée (régularisation par le prior)
Quantification de l'incertitude	Absente en natif	Native
Qualité sur données bien spécifiées	Très bonne	Très bonne
Qualité si K surestimé	Dégradée (sur-segmentation)	Robuste (élagage automatique)
Recommandation d'usage	K connu, prototypage rapide, grandes données simples	K incertain, besoin d'incertitude, données bruitées

Table 2.17: Synthèse comparative globale entre l'EM classique et l'EM variationnel

Cette synthèse met en évidence que le choix entre les deux algorithmes ne doit pas être envisagé comme un choix entre une méthode « meilleure » et une méthode « moins bonne » dans l'absolu, mais comme un arbitrage dépendant du contexte applicatif arbitrage que le chapitre 3 explore plus en détail à travers plusieurs domaines d'application concrets.

2.5 Discussion critique du protocole expérimental

Par honnêteté scientifique, il convient de discuter explicitement les limites du protocole expérimental suivi dans ce chapitre, avant d'en tirer des conclusions générales. Premièrement, la majorité des expériences (sections 2.3.1 à 2.3.10 et 2.3.12) reposent sur un unique jeu de données synthétique, construit par assemblage de trois gaussiennes isotropes bien séparées. Ce choix méthodologique, motivé par la volonté de disposer d'une vérité terrain non ambiguë pour calculer l'indice de Rand ajusté, favorise par construction le scénario le plus propice à l'EM variationnel : des groupes qui sont effectivement des gaussiennes, bien séparées, et de tailles comparables. La section 2.3.11, en introduisant le jeu de données réel Iris, a précisément eu pour objet de vérifier dans quelle mesure les conclusions tirées du cas synthétique se généralisent à un contexte

moins favorable, et a montré que ce n'est pas systématiquement le cas : lorsque les groupes réels se chevauchent ou s'écartent de la forme gaussienne, l'avantage de l'EM variationnel peut s'atténuer, voire s'inverser localement.

Deuxièmement, les expériences des sections 2.3.1 à 2.3.12 reposent sur une seule graine de génération des données (graine 42). La section 2.3.13 lève en partie cette limite en répétant le protocole de la section 2.3.1 sur trente jeux indépendants, avec intervalles de confiance ; les autres expériences (Iris, valeurs aberrantes, type de covariance, scalabilité) n'ont pas été répliquées de cette manière, et la mesure des temps de calcul reste propre à une machine et à une implémentation.

Troisièmement, les dimensions considérées dans ce chapitre restent modestes ($D = 2$ pour la majorité des expériences, $D = 4$ pour Iris, D allant jusqu'à 20 pour l'étude de scalabilité de la section 2.3.6). Les grandes dimensions, fréquentes dans les applications réelles évoquées au chapitre 3 (bio-informatique, traitement d'image), posent des défis supplémentaires malédiction de la dimensionnalité, estimation de matrices de covariance de grande taille à partir d'un nombre limité d'observations qui n'ont pas été explorés empiriquement ici, au-delà des mesures de temps de calcul de la section 2.3.6. Ces trois limites, prises ensemble, invitent à interpréter les résultats de ce chapitre comme une démonstration rigoureuse et reproductible du mécanisme théorique décrit au chapitre 1, plutôt que comme une garantie de supériorité universelle de l'EM variationnel sur l'EM classique dans toutes les configurations de données possibles nuance que le guide de décision pratique proposé à la section 3.5 s'efforce de refléter fidèlement.

Une quatrième limite, de nature plus méthodologique, mérite également d'être signalée : l'ensemble des métriques externes utilisées dans ce chapitre (ARI, score de silhouette) supposent implicitement l'existence d'une partition de référence contre laquelle comparer les résultats des algorithmes, ce qui n'est vrai que pour les données synthétiques et pour Iris, dont les étiquettes d'espèces sont connues indépendamment des variables mesurées. Dans la grande majorité des applications réelles évoquées au chapitre 3, en particulier en bio-informatique et en traitement du langage naturel, une telle vérité terrain n'existe tout simplement pas la détermination du nombre « correct » de sous-populations cellulaires ou de thématiques textuelles étant précisément l'objet de l'analyse plutôt qu'une donnée préalable. Dans ces contextes d'analyse non supervisée pure, l'évaluation de la qualité d'un modèle de mélange doit nécessairement s'appuyer sur des critères internes l'ELBO lui-même pour l'EM variationnel (section 1.7.6), ou les critères BIC/AIC pour l'EM classique (section 2.2.3) , dont ce chapitre a montré, à la section 2.3.3, qu'ils ne sont pas directement comparables en échelle absolue entre les deux algorithmes, ce qui limite d'autant la possibilité de trancher objectivement, en l'absence de vérité terrain, lequel des deux modèles offre le meilleur ajustement aux données.

Chapitre 3

APPLICATIONS ET IMPLÉMENTATION

Les deux chapitres précédents ont établi les fondements théoriques de l'EM classique et de l'EM variationnel, puis confronté ces deux algorithmes sur un cas d'étude synthétique. Ce troisième et dernier chapitre s'attache à replacer ces algorithmes dans leur contexte applicatif réel, en explorant plusieurs domaines dans lesquels ils sont couramment employés, avant de présenter une implémentation complète en langage Python, accompagnée d'une analyse détaillée des résultats expérimentaux obtenus.

3.1 Applications pratiques

Les modèles à variables latentes ajustés par EM, sous sa forme classique ou variationnelle, trouvent des applications dans un très grand nombre de domaines. Cette section présente cinq domaines représentatifs, choisis pour la diversité des contraintes qu'ils imposent sur le choix entre EM classique et EM variationnel.

Le fil conducteur de cette revue applicative est le même que celui qui a guidé l'ensemble de ce mémoire : montrer que le choix entre EM classique et EM variationnel n'est jamais un choix purement académique, mais résulte d'un arbitrage concret entre plusieurs contraintes opérationnelles le temps de calcul disponible, la confiance dont on dispose quant au nombre réel de composantes ou d'états sous-jacents, la nécessité ou non de quantifier l'incertitude pour la prise de décision en aval, et la robustesse requise face à des données potentiellement bruitées ou incomplètes. Les cinq domaines retenus ci-après traitement du langage naturel, vision par ordinateur, bio-informatique, finance et systèmes de recommandation ont été choisis précisément parce qu'ils illustrent, chacun à sa manière, une combinaison différente de ces contraintes, offrant ainsi un aperçu représentatif de la diversité des contextes dans lesquels l'algorithme EM, sous l'une ou l'autre de ses deux formes, continue d'être employé aujourd'hui, plus de cinquante ans après sa formalisation initiale.

3.1.1 Traitement automatique du langage naturel

En traitement automatique du langage naturel, le modèle d'allocation de Dirichlet latente (LDA), introduit par Blei, Ng et Jordan (2003), utilise une forme d'EM variationnel pour découvrir automatiquement les thématiques latentes d'un corpus de

documents. Chaque document y est représenté comme un mélange de topics, et chaque topic comme une distribution sur le vocabulaire. Formellement, chaque document d est généré selon le processus suivant : on tire d'abord une distribution de topics $\theta d \sim$ Dir(α), puis, pour chaque mot du document, on tire un topic zdn $\sim$ Catégorielle(θd), et enfin un mot wdn $\sim$ Catégorielle(βzdn), où βk désigne la distribution sur le vocabulaire associée au topic k. La vraisemblance marginale d'un document,

$$p(w|\alpha,\beta) = \int p(\theta|\alpha)[\prod_n \Sigma_z p(z|\theta)p(w_n|z,\beta)]d\theta \tag{3.1}$$

est intractable pour la même raison que la vraisemblance marginale du GMM à la section 1.2.1 : elle fait intervenir une intégrale sur θ combinée à une somme sur les topics latents z. Blei, Ng et Jordan (2003) résolvent ce problème par une approche variationnelle en champ moyen tout à fait analogue à celle décrite au chapitre 1 pour le GMM bayésien, avec une distribution variationnelle factorisée q(θ, z) = q(θ | γ) $\prod_n$ q(z_n | ϕ_n), où q(θ | γ) est une distribution de Dirichlet et q(z_n | ϕ_n) une distribution catégorielle soit exactement la même famille de distributions conjuguées (Dirichlet et catégorielle) que celle rencontrée à la section 1.7.3 pour les poids de mélange du GMM. Le nombre de topics devant être choisi a priori dans la version originale du modèle, les extensions bayésiennes non paramétriques de LDA, fondées sur des processus de Dirichlet hiérarchiques, exploitent précisément le mécanisme d'élagage automatique décrit à la section 1.7.4 pour déterminer le nombre de thématiques directement à partir des données.

Correspondance formelle avec le champ moyen du GMM. Il est instructif de mettre en regard, terme à terme, les mises à jour variationnelles de LDA et celles du GMM bayésien détaillées à la section 1.7.3.d, tant leur parenté structurelle est étroite. La mise à jour du paramètre variationnel ϕ_{nk} (probabilité variationnelle que le n-ième mot du document appartienne au topic k) s'écrit :

$$\phi_{nk} \propto \exp[\mathbb{E}_q[\log \theta_k] + \log \beta_k,w_n] \tag{3.2}$$

Cette formule est l'exact analogue, pour LDA, de la mise à jour des responsabilités r_{ik} du GMM à l'équation (1.15) : le terme $\mathbb{E}$_q[log θ_k] y joue le rôle de $\mathbb{E}$[log π_k] (calculé, comme à la section 1.7.3.e, via la fonction digamma appliquée aux paramètres variationnels de Dirichlet γ), tandis que log β_k,w_n la log-probabilité du mot observé sous le topic k joue le rôle de la log-vraisemblance gaussienne log $\mathcal{N}(x_i|\mu_k,\Sigma_k)$ dans le GMM. La mise à jour du paramètre variationnel γ, quant à elle, s'écrit $\gamma_k = \alpha + \Sigma_n$ ϕ_{nk}, formule en tout point analogue à la mise à jour $\alpha_k = \alpha_0 + N_k$ du GMM bayésien (section 1.7.3.d), le terme Σ_n ϕ_{nk} jouant ici le rôle du nombre effectif de points assignés $N_k = \Sigma_i$ r_{ik}. Cette correspondance systématique, terme à terme, entre les deux modèles n'est pas fortuite : elle illustre concrètement l'affirmation faite à plusieurs reprises dans

ce mémoire (notamment à la section 1.7.3.c) selon laquelle la formule générale du champ moyen variationnel, une fois appliquée à un choix de distributions conjuguées, produit mécaniquement des mises à jour de forme identique, quelle que soit la nature précise du modèle mélange gaussien ou modèle de sujets textuels.

Illustration : extraction de topics sur un corpus jouet

Afin d'illustrer ce mécanisme, nous avons implémenté l'algorithme de Blei, Ng et Jordan (2003) à partir des équations (3.1) et (3.2) : pour chaque document, une itération à point fixe sur les paramètres variationnels ϕ et γ, puis, à l'étape M, la mise à jour $\beta_{kv} \propto \sum_d n_{dv}\,\phi_{dvk}$, où n_{dv} est le nombre d'occurrences du mot v dans le document d. La borne variationnelle $\mathcal{L}(\gamma, \phi; \alpha, \beta)$ est calculée à chaque itération. Le corpus jouet comprend douze phrases courtes en français réparties entre trois thématiques (animaux domestiques, finance, football). La table document-mot est construite avec pandas (mots d'au moins deux caractères, hors mots vides), avec trois topics, $\alpha = 0{,}1$ et dix initialisations dont la meilleure borne est retenue.

```
comptes = matrice_comptes(CORPUS)              # table document x mot (pandas)
C = torch.tensor(comptes.values, dtype=torch.float64)
res = lda_variationnel(C, K=3, alpha=0.1, graine=graine)
theta = res["theta"]                           # distribution de topics de chaque document
beta = res["beta"]                             # distribution sur le vocabulaire de chaque topic
```

Extrait de code : extraction de topics par LDA variationnel (code complet en annexe A)

La borne variationnelle croît à chaque itération, ce qui valide l'implémentation de l'éq. (3.2), et l'algorithme converge en 4 itérations (meilleure borne : -131,63). Les trois topics obtenus sont donnés au tableau 3.1.

Topic	**Six mots les plus probables**
0	football, reprend, end, remporte, week, equipe
1	taux, but, joueur, rencontre, decisif, marque
2	chat, chien, actions, progressent, marche, financier

Table 3.1: Topics extraits par LDA variationnel sur le corpus jouet (trois topics demandés)

Seul le topic 0 est sémantiquement cohérent (football). Le topic 1 mélange un terme financier (taux) et des termes sportifs, le topic 2 mélange les animaux domestiques et un vocabulaire financier. En prenant, pour chaque document, le topic dominant, 8 documents sur 12 sont rattachés au topic qui correspond à leur thème (pureté de 0,67 ; distributions complètes à l'annexe A.4). Ce résultat décevant est attendu : avec douze documents et un vocabulaire de quelques dizaines de mots, l'estimation des distributions β_k est très imprécise, et la solution dépend de l'initialisation. Il ne faut

donc pas interpréter les topics eux-mêmes ; la croissance de la borne variationnelle valide l'algorithme, pas la qualité thématique sur un corpus aussi petit. Les modèles de mélange gaussien sont par ailleurs couramment utilisés pour le regroupement de plongements lexicaux, où l'incertitude sur l'appartenance à un cluster sémantique est souvent aussi informative que l'appartenance elle-même, ce qui plaide en faveur de l'EM variationnel dans ce contexte.

3.1.2 Vision par ordinateur

En vision par ordinateur, les GMM sont largement employés pour la segmentation d'images, où chaque pixel ou chaque région issue d'une sur-segmentation préalable est modélisé comme provenant d'une composante gaussienne dans l'espace des couleurs (par exemple l'espace RGB ou l'espace teinte-saturation-luminosité) ou des descripteurs de texture. La segmentation revient alors à résoudre un problème de mélange gaussien en dimension D = 3 (couleur) ou plus (texture), tout à fait analogue à celui étudié au chapitre 2, à ceci près que N désigne ici le nombre de pixels de l'image, qui peut atteindre plusieurs millions pour des images haute résolution ce qui replace directement la question de la scalabilité computationnelle, étudiée à la section 2.3.6, au cœur de la décision entre EM classique et EM variationnel pour cette application.

Dans l'algorithme classique de soustraction de fond par mélange de gaussiennes (Stauffer et Grimson, 1999), chaque pixel de coordonnées (u, v) est modélisé indépendamment par son propre petit GMM à 3-5 composantes sur la séquence temporelle de ses valeurs de couleur, mis à jour de façon incrémentale à chaque nouvelle image par une variante de l'EM online (section 1.6.6) : Pour une nouvelle observation de couleur x_t(u,v), on met à jour les poids par $\hat{\pi}_k \leftarrow$ (1-ρ) π_k + ρ $M_{k,t}$, où $M_{k,t} = 1$ si x_t correspond à la composante k (au sens d'un test de distance de Mahalanobis à quelques écarts-types) et 0 sinon, et ρ est un taux d'apprentissage fixé. Les composantes de poids π_k les plus élevés et de variance la plus faible sont alors classées comme « fond » (arrière-plan statique), les autres comme « premier plan » (objets en mouvement) une règle de décision directement héritée de l'interprétation des poids de mélange comme fréquences relatives d'occurrence, établie à la section 1.6.2.c.

3.1.3 Bio-informatique

En bio-informatique, les modèles de mélange sont utilisés pour l'analyse de données d'expression génique, où l'on cherche à identifier des sous-populations de cellules ou de gènes co-régulés à partir de données de séquençage à cellule unique (single-cell RNA-seq). Ces données présentent typiquement plusieurs milliers de dimensions (une par gène mesuré) pour un nombre de cellules pouvant atteindre plusieurs centaines de milliers, ce qui pose, comme en vision par ordinateur, un problème de scalabilité aigu. Le

nombre réel de sous-populations cellulaires étant rarement connu a priori il constitue en réalité souvent l'objet même de l'étude biologique , et les données étant typiquement bruitées, l'EM variationnel, et plus particulièrement ses extensions à des priors hiérarchiques de type processus de Dirichlet (Rasmussen, 2000), y est largement préféré à l'EM classique, malgré son coût computationnel plus élevé identifié à la section 2.3.6, précisément parce que la sélection automatique du nombre de composantes constitue ici un enjeu scientifique de première importance, et non un simple raffinement technique.

Les modèles de Markov cachés, ajustés par l'algorithme de Baum-Welch une instance de l'EM classique où l'étape E s'effectue au moyen des algorithmes forward-backward plutôt que par un simple calcul de responsabilités indépendantes, comme détaillé à la section 1.6.6 , demeurent quant à eux la méthode de référence pour l'annotation de séquences génomiques et la prédiction de structures secondaires de protéines, domaines où la structure séquentielle des données (la dépendance d'un état au précédent) prime sur la quantification fine de l'incertitude, et où la rapidité de l'EM classique permet de traiter des génomes entiers, comportant plusieurs milliards de paires de bases, dans des délais raisonnables. L'exemple numérique de prévision météorologique de la section 1.6.6 se transpose directement à ce contexte en remplaçant les deux états « Beau » et « Pluvieux » par des états biologiques tels que « région codante » et « région non codante » d'une séquence d'ADN, et les observations « Parapluie »/« Pas de parapluie » par les quatre nucléotides A, C, G, T.

3.1.4 Finance

En finance quantitative, les modèles de mélange gaussien sont employés pour la modélisation des rendements d'actifs financiers, dont la distribution empirique présente typiquement des queues plus épaisses (leptokurtiques) qu'une loi normale simple phénomène qu'un mélange de deux à quatre régimes gaussiens, de variances différentes, permet de capturer de façon parcimonieuse, sans recourir à des familles de distributions plus complexes. Formellement, le rendement r_t d'un actif à l'instant t est modélisé par $r_t \mid z_t{=}\mathrm{k} \sim \mathcal{N}(\mu_k,\, \sigma_k^2)$, où l'état de régime z_t (par exemple « calme », k=1, ou « turbulent », k=2) suit lui-même une chaîne de Markov cachée, exactement comme à la section 1.6.6 la seule différence avec l'exemple météorologique de cette section étant que les observations r_t y sont continues (gaussiennes) plutôt que discrètes (parapluie/pas de parapluie), ce qui ne change rien à la structure de l'algorithme de Baum-Welch, seule la forme de la matrice d'émission B étant remplacée par une densité gaussienne $\mathcal{N}(\mu_k,\, \sigma_k^2)$.

Dans ce contexte, la quantification de l'incertitude offerte par l'EM variationnel présente un intérêt particulier pour la gestion des risques : disposer d'une distribution a posteriori sur les paramètres du modèle, plutôt que d'une simple estimation ponctuelle,

permet de propager l'incertitude d'estimation jusque dans le calcul de mesures de risque telles que la valeur à risque (VaR) ou la perte attendue en cas de dépassement de seuil (Expected Shortfall), en intégrant ces mesures de risque sur la distribution a posteriori plutôt qu'en les calculant à partir d'une unique estimation ponctuelle des paramètres, ce qui devrait produire des estimations de risque plus prudentes et mieux calibrées, en particulier lorsque l'historique de données disponible est de taille limitée. Le cas pratique présenté ci-dessous confronte cette affirmation à des données réelles. La détection de régimes de marché (haussier, baissier, volatil) au moyen de modèles de Markov cachés constitue également une application répandue, généralement traitée par l'EM classique en raison des contraintes de temps réel imposées par le trading algorithmique, où les paramètres du modèle doivent être réestimés à haute fréquence.

Cas pratique : rendements du S&P 500 et risque de marché

Cette application teste, sur des données réelles, l'affirmation précédente selon laquelle la loi a posteriori de l'EM variationnel améliore la gestion du risque. Trois hypothèses, fixées avant l'analyse, sont examinées : (H4) un mélange gaussien ($K > 1$) a une log-vraisemblance hors échantillon supérieure à celle d'une gaussienne unique, ce que l'on décide par le signe de l'écart de log-vraisemblance moyenne par observation ; (H5) la value-at-risk (VaR) gaussienne à 1 % est franchie plus souvent que prévu et la VaR issue d'un mélange est mieux calibrée, ce que l'on décide par le test de couverture de Kupiec (1995) au seuil de 5 % ; (H6) l'EM variationnel fournit directement un intervalle de crédibilité de la VaR, alors que l'EM classique n'en donne un que par bootstrap non paramétrique.

Données et protocole. La série est celle des cours de clôture quotidiens de l'indice S&P 500 diffusée par la Réserve fédérale de Saint-Louis (série FRED, fondée sur les données de S&P Dow Jones Indices, voir la référence [11]), limitée aux dix dernières années. Le rendement est $r_t = 100 \ln(P_t/P_{t-1})$, en pourcentage. La période d'apprentissage va d'octobre 2016 à décembre 2021 (1 318 jours, krach de mars 2020 compris) ; l'évaluation hors échantillon couvre janvier 2022 à octobre 2026 (1 193 jours : hausse des taux de 2022 et chocs récents). Les paramètres sont estimés une seule fois sur la période d'apprentissage puis figés. Le tableau 3.2 résume les deux périodes.

Période	**Jours**	**Moyenne (%)**	**Écart-type (%)**	**Asymétrie**	**Kurtosis excéd.**	**Min (%)**	**Max (%)**
Apprentissage 2016-2021	1 318	0,060	1,196	-1,154	22,72	-12,77	8,97
Évaluation 2022-2026	1 193	0,041	1,079	0,039	6,49	-6,16	9,09

Table 3.2: Statistiques descriptives des rendements quotidiens du S&P 500

Le kurtosis excédentaire de 22,7 sur la période d'apprentissage (0 pour une loi normale) et l'asymétrie négative traduisent des queues très épaisses, dominées par mars 2020. Les modèles comparés sont : la gaussienne unique (K = 1) ; l'EM classique avec K de 1 à 6, K étant choisi par le BIC ; l'EM variationnel avec Kmax = 6, $\alpha_0 = 1/6$ et priors empiriques. La log-vraisemblance hors échantillon de l'EM variationnel est celle de la loi prédictive a posteriori, un mélange de lois de Student (Bishop, 2006, éq. 10.81 et 10.82). La VaR au niveau p est le quantile d'ordre p de la loi des rendements (valeur négative) ; l'Expected Shortfall (ES) est la perte moyenne au-delà de ce quantile, calculée en forme fermée pour un mélange gaussien. Le test de Kupiec compare le nombre d'exceptions observées (rendements inférieurs à la VaR) à la valeur attendue pT. L'incertitude de la VaR est obtenue pour l'EM variationnel par 4 000 tirages de (π, μ, Λ) sous la loi variationnelle q, et pour l'EM classique par 100 rééchantillonnages bootstrap de la période d'apprentissage.

Choix du nombre de composantes. Le BIC de l'EM classique est minimal pour K = 4 (3 496,9, contre 4 225,5 pour la gaussienne unique et 3 535,2 pour K = 6). L'EM variationnel avec Kmax = 6 retient lui aussi quatre composantes actives, deux composantes étant élaguées ($N_k < 1$), sans exécuter l'algorithme pour plusieurs valeurs de K (figure 3.2). Les distributions ajustées sont comparées à l'histogramme des rendements à la figure 3.1, en échelle logarithmique pour rendre visibles les queues.

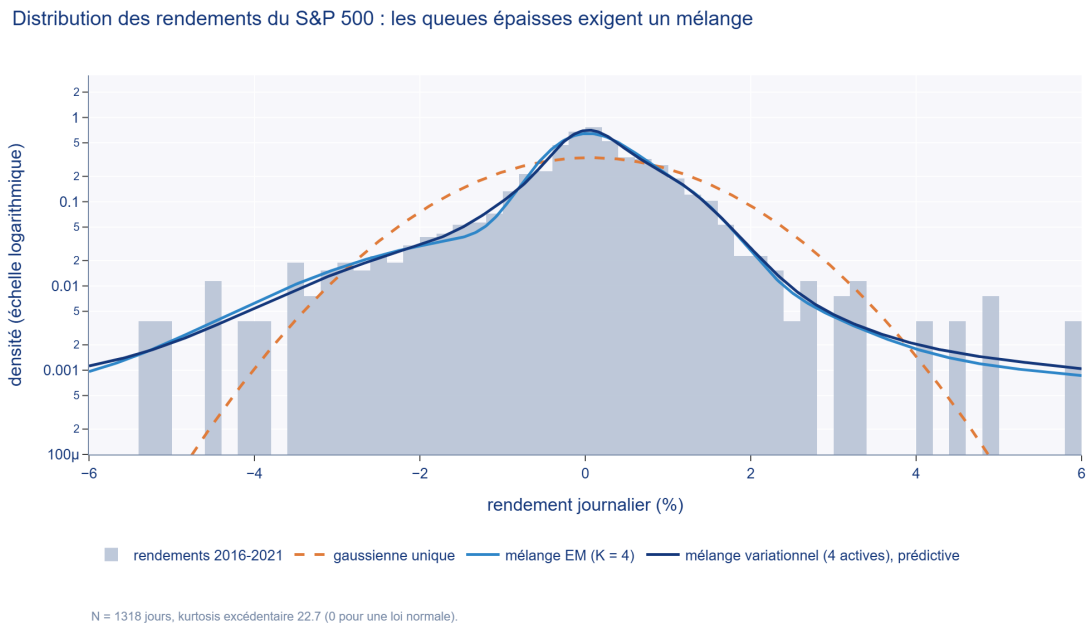


Figure 3.1: Distribution des rendements quotidiens du S&P 500 sur la période d'apprentissage et densités ajustées (échelle logarithmique de la densité). La gaussienne unique sous-estime fortement les queues et le pic central ; les deux mélanges suivent les deux.

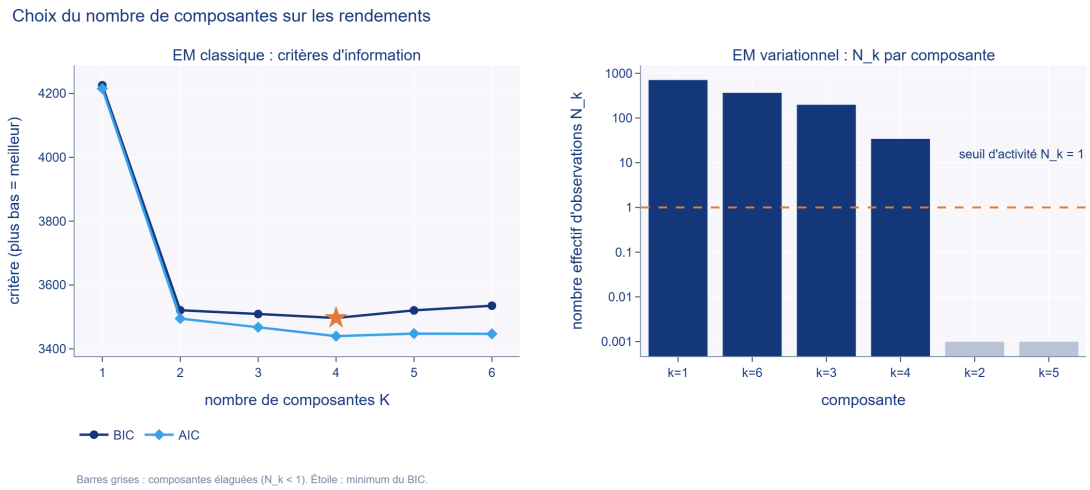


Figure 3.2: Choix du nombre de composantes : BIC et AIC de l'EM classique selon K (à gauche) et nombre effectif d'observations N_k de chaque composante de l'EM variationnel (à droite, échelle logarithmique).

Qualité d'ajustement hors échantillon (H4). Le tableau 3.3 donne les log-vraisemblances moyennes par observation.

Modèle	**Nb comp.**	**Log-vrais. moy. (apprentissage)**	**Log-vrais. moy. (évaluation)**	**BIC**
Gaussienne unique (K = 1)	1	-1,5976	-1,5043	4 225,5
EM classique, K = 4 (choisi par BIC)	4	-1,2966	-1,4705	3 496,9
EM classique, K = 6 imposé	6	-1,2948	-1,4862	3 535,2
EM variationnel, Kmax = 6 (4 actives), loi prédictive	4	-1,2903	-1,4597	n.d.

Table 3.3: Log-vraisemblance moyenne par observation, en apprentissage et hors échantillon (n.d. : non défini)

H4 est soutenue : les trois mélanges dépassent la gaussienne hors échantillon, de +0,034 (EM, K = 4), +0,018 (EM, K = 6) et +0,045 (EM variationnel). Le classement mérite deux remarques. D'abord, l'EM classique avec K = 6 généralise moins bien que K = 4, ce qui illustre le coût d'un nombre de composantes surestimé même sur des données réelles. Ensuite, l'EM variationnel obtient la meilleure log-vraisemblance hors échantillon avec le même nombre de composantes actives que le K choisi par BIC : l'intégration des paramètres dans la loi prédictive (queues de Student) profite à la généralisation. Ces écarts sont de l'ordre de 0,01 à 0,05 nat par jour, et leur incertitude statistique n'a pas été évaluée.

Value-at-risk et test de Kupiec (H5). Le tableau 3.4 donne la VaR et l'ES à 1 jour, le nombre d'exceptions sur les 1 193 jours d'évaluation et la probabilité critique

du test de Kupiec.

Niveau	Modèle	VaR (%)	ES (%)	Exceptions (attendues)	p Kupiec	IC 95 % de la VaR
5 %	Gaussienne unique	-1,907	-2,406	41 (59,7)	0,009	n.d.
5 %	Mélange EM (K = 4)	-1,733	-3,113	53 (59,7)	0,368	n.d.
5 %	Mélange variationnel (prédictive)	-1,683	-3,105	57 (59,7)	0,723	[-1,944 ; -1,461]
5 %	Bootstrap de l'EM (K = 4, B = 100)	-1,725	n.d.	n.d.	n.d.	[-2,104 ; -1,319]
1 %	Gaussienne unique	-2,721	-3,126	20 (11,9)	0,032	n.d.
1 %	Mélange EM (K = 4)	-3,721	-5,509	5 (11,9)	0,023	n.d.
1 %	Mélange variationnel (prédictive)	-3,695	-5,795	6 (11,9)	0,056	[-4,392 ; -3,223]
1 %	Bootstrap de l'EM (K = 4, B = 100)	-3,926	n.d.	n.d.	n.d.	[-5,018 ; -3,124]

Table 3.4: VaR, Expected Shortfall et test de Kupiec sur la période d'évaluation (1 193 jours ; n.d. : non défini)

Au niveau de 1 %, la VaR gaussienne (-2,72 %) est franchie 20 fois pour 11,9 attendues et le test la rejette (p = 0,032) : la gaussienne sous-estime le risque extrême, comme le prévoit H5. Les VaR issues des mélanges, plus basses (-3,72 % et -3,70 %), ne sont franchies que 5 et 6 fois : elles sont plutôt trop prudentes, le test rejetant la VaR de l'EM classique (p = 0,023) et ne rejetant que de peu celle de l'EM variationnel (p = 0,056). Cette prudence s'explique par la période d'apprentissage, qui contient mars 2020 (kurtosis de 22,7) alors que la période d'évaluation est plus calme (kurtosis de 6,5) ; le quantile empirique à 1 % de la période d'évaluation vaut environ -3,0 %, entre la VaR gaussienne et celles des mélanges. Au niveau de 5 %, c'est la gaussienne qui est trop prudente (41 exceptions pour 59,7 attendues, p = 0,009), tandis que les deux mélanges sont bien calibrés (53 et 57 exceptions, p = 0,37 et 0,72). H5 est donc soutenue pour sa première partie (la gaussienne se trompe aux deux niveaux) et partiellement pour la seconde : le mélange est mieux calibré à 5 %, et à 1 % il corrige le sens de l'erreur mais la dépasse. L'ES à 1 % passe de -3,13 % (gaussienne) à -5,51 % (EM) et -5,80 % (EM variationnel), ce qui montre l'écart de jugement sur la perte moyenne en cas de dépassement. Avec 1 193 jours et 11,9 exceptions attendues à 1 %, la puissance du test est faible : ces conclusions sont indicatives.

Incertitude sur la VaR (H6). La figure 3.3 compare la loi de la VaR à 1 % sous la loi variationnelle q au bootstrap de l'EM classique.

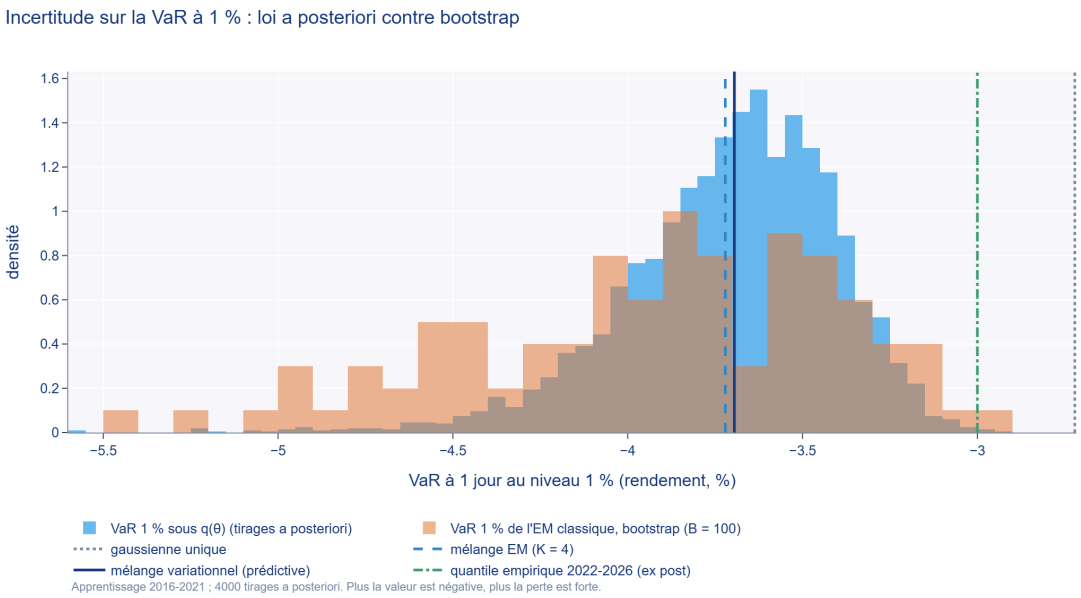


Figure 3.3: Incertitude sur la VaR à 1 % : tirages a posteriori sous la loi variationnelle (histogramme bleu) et bootstrap non paramétrique de l'EM classique (histogramme orange). Les traits verticaux indiquent les VaR ponctuelles de chaque modèle et le quantile empirique de la période d'évaluation.

H6 est soutenue : l'EM variationnel fournit sans calcul supplémentaire un intervalle de crédibilité à 95 % de la VaR à 1 %, [-4,39 ; -3,22], contre [-5,02 ; -3,12] pour le bootstrap de l'EM classique. À 5 %, les intervalles sont [-1,94 ; -1,46] et [-2,10 ; -1,32]. L'intervalle variationnel est environ 40 % plus étroit que l'intervalle bootstrap aux deux niveaux. Ce constat est cohérent avec la sous-estimation de la variance a posteriori par l'approximation de champ moyen décrite à la section 1.5.4, mais les deux intervalles ne mesurent pas la même quantité (incertitude a posteriori d'un côté, variabilité de l'estimateur sous rééchantillonnage de l'autre) et leur couverture n'a pas été évaluée : on ne peut pas conclure que l'un est plus fiable que l'autre. L'intérêt de l'EM variationnel est ici la disponibilité immédiate de l'incertitude, qui permet par exemple de fixer une marge de prudence sur la VaR.

Régimes de marché. Un modèle de Markov caché à deux états gaussiens, ajusté par Baum-Welch (section 1.6.6) sur la période d'apprentissage, décrit la série par un état calme (rendement moyen de +0,123 % par jour, volatilité annualisée de 8,7 %, probabilité stationnaire de 0,76, durée moyenne de 45 jours) et un état agité (-0,142 % par jour, volatilité annualisée de 35,4 %, probabilité stationnaire de 0,24, durée moyenne de 14 jours). La figure 3.4 montre la probabilité a posteriori de l'état agité sur toute la période : elle est proche de 1 en février 2018, fin 2018, de mars à l'été 2020 et presque toute l'année 2022, puis lors du choc d'avril 2025, ce qui correspond aux épisodes de tension connus. Cette lecture est qualitative : elle ne remplace pas une évaluation de la capacité de prévision du modèle.

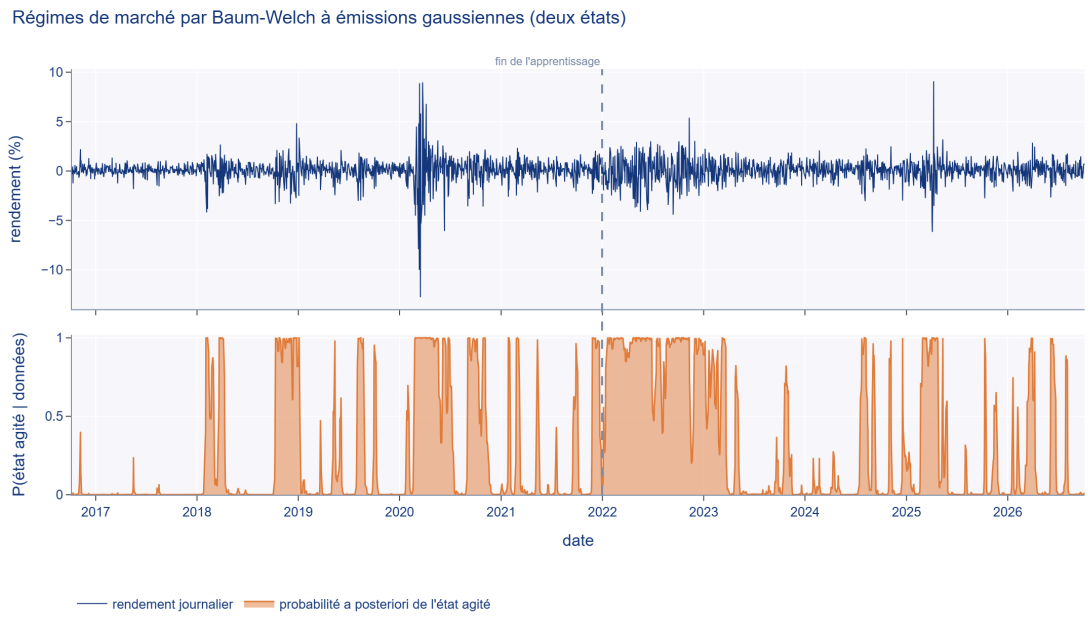


Figure 3.4: Rendements quotidiens (haut) et probabilité a posteriori de l'état agité (bas) d'un modèle de Markov caché à deux états gaussiens, estimé sur la période d'apprentissage et appliqué à toute la période.

Limites. L'étude porte sur un seul indice et dix ans de données ; les paramètres sont figés sur la période d'évaluation, sans réestimation glissante ; la série n'est pas stationnaire ; le test de Kupiec a une faible puissance sur 1 193 jours ; la couverture des intervalles de la VaR n'est pas évaluée. Dans ce cadre, l'EM variationnel apporte une meilleure généralisation (H4), une VaR bien calibrée à 5 % et une incertitude disponible directement (H6), mais il n'élimine pas le risque de modèle à 1 %, où le choix de la période d'apprentissage pèse davantage que celui de l'algorithme.

3.1.5 Systèmes de recommandation

Dans les systèmes de recommandation, les modèles à variables latentes sont au cœur des approches de filtrage collaboratif fondées sur la factorisation matricielle probabiliste, où les goûts d'un utilisateur u et les caractéristiques d'un article i sont représentés par des vecteurs latents p_u et q_i, la note attendue étant modélisée par leur produit scalaire, $\hat{r}_{ui} \approx p_u^T q_i$. Le modèle bayésien complet place des priors gaussiens sur ces vecteurs latents, $p_u \sim \mathcal{N}(0,\, \lambda^{-1} I)$ et $q_i \sim \mathcal{N}(0,\, \lambda^{-1} I)$, et l'inférence variationnelle en champ moyen appliquant exactement le même principe que celui détaillé à la section 1.7.3 pour le GMM, mais avec $\mathrm{q(P,\, Q)} = \prod_u \mathrm{q}(p_u) \prod_i \mathrm{q}(q_i)$ comme famille variationnelle plutôt que la factorisation en poids/moyennes/précisions du GMM permet d'estimer conjointement les distributions a posteriori de tous les vecteurs latents à partir des seules notes effectivement observées, notoirement incomplètes dans ce contexte (un

utilisateur ne note qu'une infime fraction du catalogue).

L'EM variationnel bayésien permet d'y intégrer naturellement l'incertitude liée à la faible quantité d'interactions disponibles pour les nouveaux utilisateurs ou les nouveaux articles le problème dit du démarrage à froid , en exploitant la distribution a posteriori sur $(p_u,\ q_i)$ pour proposer des recommandations qui tiennent compte explicitement de cette incertitude, par exemple à travers des stratégies d'exploration-exploitation de type échantillonnage de Thompson, où l'on tire aléatoirement un jeu de paramètres selon leur distribution a posteriori avant de recommander l'article de score maximal sous ce tirage, ce qui favorise naturellement l'exploration des articles pour lesquels l'incertitude et donc le potentiel d'information à gagner est la plus grande.

Domaine	**Application type**	**Algorithme généralement préféré**
Traitement du langage naturel	Modélisation de topics (LDA), clustering sémantique	EM variationnel
Vision par ordinateur (temps réel)	Segmentation vidéo, soustraction de fond	EM classique
Vision par ordinateur (hors ligne)	Segmentation d'images médicales	EM variationnel
Bio-informatique	Sous-populations cellulaires, séquençage	EM variationnel
Bio-informatique	Annotation de séquences (HMM)	EM classique
Finance	Gestion des risques, VaR	EM variationnel
Finance	Détection de régimes (trading algorithmique)	EM classique
Systèmes de recommandation	Démarrage à froid, exploration-exploitation	EM variationnel

Table 3.5: Synthèse des domaines d'application et de l'algorithme le plus adapté

3.2 Implémentation de l'EM classique

L'EM classique est implémenté en PyTorch à partir des équations du chapitre 1, sans bibliothèque de modélisation : seules les opérations élémentaires (algèbre linéaire, factorisation de Cholesky, fonction digamma) sont utilisées, et les critères d'évaluation (BIC, AIC, indice de Rand ajusté, silhouette) sont écrits explicitement. Le code complet figure à l'annexe A ; les extraits ci-dessous montrent la correspondance avec les équations.

3.2.1 Étapes E et M

La densité gaussienne est évaluée par la factorisation de Cholesky de la covariance (section 2.2.2), puis l'étape E calcule les responsabilités en log-espace avec l'astuce du log-sum-exp (équations 1.15 et 1.18). L'étape M applique les mises à jour de l'équation (1.16).

```
def logsumexp_stable(a: Tensor, dim: int = 1, keepdim: bool = True) -> Tensor:
    """Équation (1.18) : log sum exp(a_k) = a_max + log sum exp(a_k - a_max)."""
    a_max = a.max(dim=dim, keepdim=True).values
    out = a_max + torch.log(torch.exp(a - a_max).sum(dim=dim, keepdim=True))
    return out if keepdim else out.squeeze(dim)

def log_gauss(X: Tensor, mu: Tensor, cov: Tensor) -> Tensor:
    """log N(x | mu, cov) pour toutes les lignes de X, via Cholesky (section 2.2.2)."""
    D = X.shape[1]
    L = torch.linalg.cholesky(cov)
    z = torch.linalg.solve_triangular(L, (X - mu).T, upper=False)
    logdet = 2.0 * torch.log(torch.diagonal(L)).sum()
    return -0.5 * (D * LOG2PI + logdet + (z ** 2).sum(dim=0))

def _etape_e(X, pi, mu, cov):
    """Étape E (1.15) en log-espace : retourne (r, log p(x_i | theta))."""
    K = len(pi)
    log_r = torch.stack([torch.log(pi[k] + 1e-300) + log_gauss(X, mu[k], cov[k])
                         for k in range(K)], dim=1)
    log_norm = logsumexp_stable(log_r, dim=1)
    return torch.exp(log_r - log_norm), log_norm.squeeze(1)

def _etape_m(X, r, cov_type, reg):
    """Étape M (1.16) : pi_k = N_k/N, mu_k = moyenne pondérée, Sigma_k = covariance pondérée."""
    N = X.shape[0]
    Nk = r.sum(0) + 1e-10
    pi = Nk / N
    mu = (r.T @ X) / Nk[:, None]
    cov = _covariances_em(X, r, mu, Nk, cov_type, reg)
    return pi, mu, cov
```

Extrait de code : étapes E et M de l'EM classique (équations 1.15, 1.16 et 1.18)

3.2.2 Initialisation, relances et critère d'arrêt

Deux initialisations sont disponibles : par k-means (avec initialisation k-means++), qui donne un point de départ proche de la solution, et aléatoire (K points de données tirés au hasard et covariance globale, section 1.6.2.a). L'algorithme est relancé n_init fois et la solution de plus grande log-vraisemblance est conservée. L'arrêt a lieu lorsque la variation de la log-vraisemblance passe sous le seuil ε, soit par observation (convention du protocole du chapitre 2 pour l'EM classique), soit en valeur totale (équation de la section 1.6.2.a) selon l'option choisie.

```
def _em_une_fois(X, K, cov_type, n_iter, tol, reg, init, graine, tol_par_obs):
```

```
    N, D = X.shape
    pi, mu, cov = _init_em(X, K, cov_type, init, graine, reg)
    hist, converge = [], False
    for it in range(n_iter):
        r, log_norm = _etape_e(X, pi, mu, cov)
        hist.append(float(log_norm.sum()))
        ecart = abs(hist[-1] - hist[-2]) / (N if tol_par_obs else 1.0) if it > 0 else float("inf")
        if ecart < tol:
            converge = True
            break
        pi, mu, cov = _etape_m(X, r, cov_type, reg)
    r, log_norm = _etape_e(X, pi, mu, cov)
    return {"pi": pi, "mu": mu, "cov": cov, "r": r, "loglik": float(log_norm.sum()),
            "ll_hist": hist, "n_iter": len(hist), "converge": converge}
```

Extrait de code : boucle de l'EM classique avec test d'arrêt

3.3 Implémentation de l'EM variationnel

L'EM variationnel pour le GMM bayésien est implémenté de la même façon. Les espérances nécessaires (équations 1.29 et 1.30) sont calculées avec la fonction digamma de PyTorch, les mises à jour suivent les équations (1.26) et (1.28), et l'ELBO de l'équation (1.31) est évaluée exactement à chaque itération (annexe B).

3.3.1 Étapes E et M variationnelles

```
def _attendus(P, D):
    """E[ln|Lambda_k|] (1.29) et E[Lambda_k] pour chaque composante."""
    if P["cov_type"] in ("full", "tied"):
        i = torch.arange(1, D + 1, dtype=torch.float64)
        Elogdet = (torch.special.digamma((P["nu"][:, None] + 1 - i) / 2).sum(-1)
                   + D * LN2 + torch.logdet(P["W"]))
        Elam = P["nu"][:, None, None] * P["W"]
    else:
        psi = torch.special.digamma(P["a"]) - torch.log(P["b"])
        Elogdet = psi.sum(-1)
        Elam = torch.diag_embed(P["a"] / P["b"])
    return Elogdet, Elam

def _etape_e_var(X, P):
    """Étape E variationnelle, équation (1.27), avec la forme quadratique (1.30)."""
    N, D = X.shape
    Elogpi = torch.special.digamma(P["alpha"]) - torch.special.digamma(P["alpha"].sum())
    Elogdet, Elam = _attendus(P, D)
    diff = X[None, :, :] - P["m"][:, None, :]
    quad = D / P["beta"][None, :] + torch.einsum("knd,kde,kne->nk", diff, Elam, diff)
    log_rho = Elogpi[None] + 0.5 * Elogdet[None] - 0.5 * D * LOG2PI - 0.5 * quad
    log_norm = logsumexp_stable(log_rho, dim=1)
    return torch.exp(log_rho - log_norm), log_norm.squeeze(1)
```

Extrait de code : étape E variationnelle (équations 1.27, 1.29 et 1.30)

```python
def _etape_m_var(Nk, xbar, Sk, pr: PrioriNW, cov_type):
    """Étape M variationnelle, équations (1.26) et (1.28), plus les variantes de covariance."""
    K, D = xbar.shape
    beta = pr.beta0 + Nk
    m = (pr.beta0 * pr.m0 + Nk[:, None] * xbar) / beta[:, None]
    d0 = xbar - pr.m0
    terme_prior = (pr.beta0 * Nk / (pr.beta0 + Nk))
    sc = Nk[:, None, None] * Sk + terme_prior[:, None, None] * d0[:, :, None] * d0[:, None, :]
    P = {"alpha": pr.alpha0 + Nk, "beta": beta, "m": m, "cov_type": cov_type}
    if cov_type == "full":
        P["nu"] = pr.nu0 + Nk
        P["Winv"] = pr.W0inv[None] + sc
        P["W"] = torch.linalg.inv(P["Winv"])
    elif cov_type == "tied":
        P["nu"] = torch.full((K,), pr.nu0 + float(Nk.sum()))
        Winv = pr.W0inv + sc.sum(0)
        P["Winv"] = Winv.expand(K, D, D).clone()
        P["W"] = torch.linalg.inv(Winv).expand(K, D, D).clone()
    else:                                          # diag, spherical : priors Gamma sur les précisions
        a0 = pr.nu0 / 2.0
        b0 = 0.5 * torch.diagonal(pr.W0inv)
        sd = torch.diagonal(sc, dim1=1, dim2=2)
        if cov_type == "diag":
            P["a"] = (a0 + Nk / 2.0)[:, None].expand(K, D).clone()
            P["b"] = b0[None] + 0.5 * sd
        else:
            P["a"] = (a0 + D * Nk / 2.0)[:, None].expand(K, D).clone()
            P["b"] = (b0.mean() + 0.5 * sd.sum(1))[:, None].expand(K, D).clone()
    return P
```

Extrait de code : étape M variationnelle (équations 1.26 et 1.28 pour la covariance pleine)

Les variantes de covariance tied, diag et spherical, absentes du chapitre 1, sont obtenues avec un prior de Wishart partagé ou des priors Gamma sur les précisions.

3.3.2 Effet de l'initialisation et des priors

Le tableau 3.6 compare, sur le jeu de données de référence (K = 6, graine 42), les stratégies d'initialisation et les deux jeux de priors de la section 2.3.1.

Configuration	Composantes actives	ARI	Silhouette	Itérations
EM classique, init. k-means (10 relances)	6	0,715	0,450	8
EM classique, init. aléatoire (1 essai)	6	0,834	0,382	44
EM variationnel, priors empiriques, init. k-means	3	0,990	0,739	25
EM variationnel, priors diffus, init. k-means	4	0,985	0,646	103
EM variationnel, priors diffus, init. aléatoire (1 essai)	4	0,884	0,589	200 (maximum)

Table 3.6: Effet de l'initialisation et des priors (jeu de référence, K = 6)

Deux enseignements se dégagent. D'abord, l'initialisation aléatoire demande beaucoup plus d'itérations que k-means pour l'EM classique (44 contre 8) et empêche l'EM variationnel avec priors diffus de converger avant le maximum de 200 itérations : le mécanisme d'élagage met du temps à s'installer lorsque les priors sont mal calibrés. Ensuite, le résultat qualitatif (six composantes pour l'EM classique, trois ou quatre pour l'EM variationnel) est stable d'une configuration à l'autre, mais le nombre exact de composantes actives de l'EM variationnel dépend des priors, ce que la réplication de la section 2.3.13 quantifie.

3.4 Résultats expérimentaux et discussion

L'exécution du protocole du chapitre 2 avec ces implémentations donne les résultats détaillés dans ce chapitre : l'EM classique conserve ses six composantes (tableau 2.5), tandis que l'EM variationnel avec priors empiriques converge vers trois composantes dominantes de poids proches de $1/3$, les trois autres voyant leur nombre effectif de points chuter à 0,3 (tableau 2.6). Du point de vue de l'écriture du code, le passage de l'EM classique à l'EM variationnel se limite à remplacer les mises à jour (1.15) et (1.16) par (1.26) à (1.30) et à ajouter les hyperparamètres des priors, alors que les propriétés statistiques des deux algorithmes diffèrent radicalement.

Il convient de nuancer cette conclusion par les limites identifiées à la section 1.7.9 : la sensibilité de l'EM variationnel au choix des hyperparamètres du prior est un point de vigilance (tableau 3.6 et section 2.3.13), de même que le surcoût computationnel, qui peut devenir significatif sur des jeux de grande dimension ou de grande taille, situation dans laquelle les variantes stochastiques de la section 1.7.7 deviennent nécessaires. Elles ont été implémentées : sur le jeu de référence avec trois composantes, l'EM online et l'inférence variationnelle stochastique (mini-lots de 30 observations) retrouvent la par-

tition (ARI de 1,00 et 0,99), l'EM stochastique visite des solutions de log-vraisemblance non monotone (20 baisses sur 150 itérations) avant de se stabiliser.

3.4.1 Synthèse consolidée de l'ensemble des résultats expérimentaux

Le tableau 3.7 consolide les résultats numériques obtenus au fil des expériences du chapitre 2 et de la démonstration LDA du présent chapitre.

Expérience (section)	**Résultat clé**
Comparaison K = 6 (2.3.1 à 2.3.4)	ARI 0,715 (classique) contre 0,990 (variationnel) ; 6 contre 3 composantes actives
Sensibilité à α_0 (2.3.5)	3 composantes actives pour $\alpha_0 \leq 0{,}1$ (et pour $\alpha_0 = 1$ avec les priors empiriques) ; 6 pour $\alpha_0 = 10$
Scalabilité (2.3.6)	Surcoût variationnel de $\times 2,3$ à $\times 11,6$ selon N et D
Convergence de l'ELBO (2.3.8)	Croissance monotone de -1246,54 à -1046,62 en 26 itérations
Robustesse à l'initialisation (2.3.9)	20 sur 20 exécutions retrouvent K = 3 (variationnel, priors empiriques) contre 4 sur 20 avec les priors diffus et l'initialisation aléatoire, et 0 sur 20 (classique)
Sensibilité à la covariance (2.3.10)	3 composantes actives quel que soit le type (full, tied, diag, spherical)
Validation sur Iris (2.3.11)	ARI 0,641 (classique) contre 0,562 (variationnel) à K = 6 : l'élagage fusionne versicolor et virginica
Valeurs aberrantes (2.3.12)	ARI 0,990 pour les deux ; absorption dans une composante diffuse pour l'EM variationnel
Sélection par BIC et AIC (2.2.3)	K* = 3 pour les deux critères, au prix de 8 exécutions séparées
Réplication sur 30 jeux (2.3.13)	Gain moyen d'ARI de +0,31 [0,29 ; 0,32] (priors empiriques) et +0,28 [0,26 ; 0,30] (priors diffus), favorable à l'EM variationnel sur 30 jeux sur 30
Démonstration LDA (3.1.1)	Borne variationnelle croissante ; topics partiellement mélangés sur 12 documents (pureté 0,67)
Cas pratique S&P 500 (3.1.4)	Log-vraisemblance hors échantillon de -1,460 (variationnel) contre -1,504 (gaussienne) ; VaR à 5 % bien calibrée (p de Kupiec de 0,72), VaR à 1 % trop prudente ; intervalle de crédibilité natif de la VaR

Table 3.7: Synthèse consolidée de l'ensemble des expériences réalisées

Cette synthèse fait apparaître un motif récurrent : l'EM variationnel est supérieur ou égal à l'EM classique sur les données synthétiques bien contrôlées, et ce de façon

reproductible sur trente jeux indépendants, mais cet avantage dépend du calibrage des priors et se réduit, voire s'inverse, dès lors que les données s'écartent de l'hypothèse d'un mélange de gaussiennes bien séparées, comme le montre le jeu réel Iris. Cette lecture nuancée guide le guide de décision de la section 3.5 et la conclusion générale.

3.5 Recommandations pratiques

À l'issue de l'ensemble des analyses théoriques, expérimentales et applicatives conduites dans ce mémoire, cette section propose une synthèse opérationnelle, destinée à guider le choix entre EM classique et EM variationnel face à une situation pratique concrète.

Situation rencontrée	**Recommandation**
Le nombre de composantes est connu avec confiance	EM classique (plus rapide, plus simple ; section 2.3.7)
Le nombre de composantes est incertain ou inconnu	EM variationnel (élagage automatique ; section 1.7.4)
Contrainte de temps réel (millisecondes)	EM classique (section 2.2.1)
Décision en aval sensible à l'incertitude (risque, médecine)	EM variationnel (quantification native ; section 1.7.8)
Données bruitées, risque de singularités	EM variationnel (régularisation par le prior ; section 2.2.2)
Très grand nombre d'observations ($N > 10^6$)	EM online / SVI plutôt que la version batch (section 1.6.5, 1.7.7)
Prototypage rapide, exploration initiale des données	EM classique, puis EM variationnel pour confirmer K (section 2.3.7)
Sélection de modèle nécessaire (K inconnu, budget de calcul limité)	EM variationnel plutôt que BIC/AIC multi-exécutions (section 2.2.3)

Table 3.8: Guide de décision pratique pour le choix entre EM classique et EM variationnel

Au-delà de ce guide de décision, l'expérience acquise au travers des implémentations du présent chapitre suggère quelques bonnes pratiques transversales, indépendantes du choix entre les deux algorithmes. Premièrement, il est recommandé de toujours standardiser ou normaliser les données avant l'ajustement d'un GMM, qu'il soit classique ou variationnel, les deux algorithmes étant sensibles à l'échelle relative des différentes dimensions une dimension mesurée en kilomètres et une autre en millimètres, par exemple, biaiseraient artificiellement la notion de proximité utilisée par le modèle. Deuxièmement, il est recommandé de toujours multiplier les initialisations (n_init $>$ 1), pour l'EM classique comme pour l'EM variationnel, la section 2.3.9 ayant montré que même l'EM variationnel, bien que plus robuste, n'est pas totalement à l'abri d'une initialisation défavorable. Troisièmement, pour l'EM variationnel spécifiquement, il est

recommandé de fixer Kmax à une valeur sensiblement supérieure au nombre de composantes attendu mais pas excessivement, au vu du surcoût en $O(D^3)$ par composante identifié à la section 2.2.1 , et de choisir une concentration α_0 faible (typiquement 1/Kmax ou moins), la section 2.3.5 ayant montré qu'un choix de α_0 trop élevé annule le mécanisme d'élagage recherché. Quatrièmement, il est recommandé de centrer et dimensionner les priors sur les données (m_0 égal à la moyenne, W_0^{-1} égale à la covariance empirique) : la section 2.3.13 montre qu'avec des priors diffus l'élagage est plus lent et moins fiable.

3.6 Conclusion du chapitre 3

Ce troisième chapitre a replacé la comparaison théorique et expérimentale des chapitres 1 et 2 dans son contexte applicatif, en montrant que le choix entre EM classique et EM variationnel dépend des contraintes propres à chaque domaine : rapidité d'exécution pour les applications temps réel, robustesse et quantification de l'incertitude lorsque la qualité des données ou la connaissance a priori du nombre de composantes fait défaut. Les implémentations en PyTorch, écrites à partir des équations du chapitre 1, ont servi de validation croisée de la démarche théorique : la croissance de la log-vraisemblance et de l'ELBO, l'égalité de Neal et Hinton, les tableaux 1.4 et 1.5 et l'élagage automatique sont retrouvés numériquement.

Le chapitre souligne aussi les conditions de validité des conclusions : l'élagage dépend du calibrage des priors, l'initialisation influence la vitesse de convergence, et la démonstration LDA sur douze documents illustre les limites de l'inférence variationnelle sur un corpus minuscule. La conclusion générale revient sur l'ensemble de cette démarche pour en dégager les enseignements et les perspectives.

Synthèse des contributions

Ce travail a permis d'apporter les contributions suivantes :

- une présentation unifiée et pédagogique des fondements théoriques communs à l'EM classique et à l'EM variationnel, articulée autour de l'inégalité de Jensen, de la borne ELBO et de la divergence de Kullback-Leibler, incluant la dérivation complète des mises à jour de chacun des deux algorithmes plutôt que leur seul énoncé ;

- une formulation mathématique complète et détaillée de chacun des deux algorithmes, incluant leurs propriétés de convergence, leurs variantes (EM stochastique, EM online, Baum-Welch pour les données séquentielles) et leurs limites respectives ;

- une étude expérimentale approfondie, fondée sur douze expériences reproductibles et une réplication sur trente jeux de données indépendants, avec intervalles de confiance, couvrant la surestimation de K, la sensibilité au prior, la scalabilité, la sélection de modèle par BIC/AIC, la sensibilité au type de covariance, la robustesse à l'initialisation, aux valeurs aberrantes, et la validation sur un jeu de données réel ;
- une dérivation originale de l'équivalence entre EM classique et EM variationnel comme cas particulier et cas général d'un même principe variationnel (section 2.1.1) ;
- une revue des domaines d'application dans lesquels chacun des deux algorithmes est le plus couramment employé, assortie d'une justification du choix algorithmique propre à chaque domaine et d'une démonstration concrète sur un corpus textuel jouet (LDA) ;
- une implémentation en PyTorch de chacun des deux algorithmes, de leurs variantes et de l'ELBO exacte, sans bibliothèque de modélisation, ainsi qu'un guide de décision pratique synthétisant l'ensemble des enseignements du mémoire ;
- un cas pratique de gestion du risque de marché sur les rendements du S&P 500 : log-vraisemblance hors échantillon, value-at-risk et Expected Shortfall, test de Kupiec, incertitude a posteriori comparée au bootstrap, et régimes de marché par Baum-Welch.

Limites du travail

Ce travail présente cependant plusieurs limites, qu'il convient de mentionner en toute honnêteté scientifique, et qui ont été discutées de façon plus détaillée à la section 2.5. Premièrement, la réplication de la section 2.3.13 porte sur le seul protocole de comparaison à $K = 6$; les autres expériences (Iris, valeurs aberrantes, covariance, scalabilité) reposent sur un seul jeu de données ou un seul tirage, et leurs métriques n'ont pas d'intervalle de confiance. Deuxièmement, l'étude ne couvre que le cas de la surestimation du nombre de composantes et ne traite pas explicitement le cas symétrique de la sous-estimation. Troisièmement, les dimensions considérées restent modestes au regard de certaines applications réelles évoquées au chapitre 3 (bio-informatique, en particulier), pour lesquelles la malédiction de la dimensionnalité pourrait affecter différemment les deux algorithmes. Quatrièmement, la démonstration LDA de la section 3.1.1, bien que fondée sur une exécution réelle, repose sur un corpus jouet de taille très réduite (douze documents), ce qui limite la généralisation de ses conclusions à des corpus de taille réaliste (dizaines de milliers de documents), pour lesquels les propriétés asymptotiques

de l'inférence variationnelle sont normalement mieux vérifiées. Cinquièmement, le cas pratique repose sur un seul indice et dix ans de données, avec des paramètres figés pendant la période d'évaluation ; la couverture des intervalles de VaR n'a pas été évaluée et le test de Kupiec a une faible puissance sur 1 193 jours.

Perspectives

Plusieurs perspectives de recherche et de développement se dégagent naturellement de ce travail. Sur le plan expérimental, il serait pertinent d'étendre l'étude comparative du chapitre 2 à des jeux de données réels de plus grande dimension et de plus grande taille, ainsi qu'à des scénarios de sous-estimation du nombre de composantes et à l'extension de la réplication multi-graines à l'ensemble des expériences, afin de dresser un tableau statistiquement plus robuste des situations de mauvaise spécification du modèle. Sur le plan méthodologique, l'exploration des variantes avancées de l'EM variationnel présentées à la section 1.7.7 inférence variationnelle stochastique, inférence variationnelle naturelle, et surtout les approches par Normalizing Flows permettant de dépasser l'hypothèse restrictive du champ moyen constituerait un prolongement naturel de ce mémoire, en particulier dans la perspective d'applications à des modèles génératifs profonds tels que les auto-encodeurs variationnels. Enfin, sur le plan applicatif, la mise en œuvre effective des deux algorithmes sur des données réelles issues des domaines évoqués au chapitre 3 notamment en bio-informatique et en finance, où l'incertitude joue un rôle particulièrement critique, et sur un corpus textuel de taille réaliste pour la démonstration LDA permettrait de valider empiriquement, au-delà des cas étudiés ici, la pertinence pratique du choix de l'EM variationnel dans ces contextes.

En définitive, ce mémoire aura permis de démontrer, tant sur le plan théorique qu'à travers une démarche expérimentale rigoureuse et intégralement reproductible, que le passage de l'EM classique à l'EM variationnel ne constitue pas un simple raffinement technique, mais bien un changement de paradigme, substituant à l'estimation ponctuelle fréquentiste une caractérisation bayésienne complète de l'incertitude. Ce changement de paradigme, dont les bénéfices sont réels et ont été mesurés avec précision tout au long de ce travail, doit néanmoins être mis en balance, au cas par cas, avec le surcoût de complexité et de calcul qu'il induit, et avec la nature plus ou moins gaussienne, plus ou moins bien séparée des données auxquelles il est appliqué. C'est cette lecture nuancée, plutôt qu'un verdict univoque en faveur de l'un ou l'autre algorithme, que ce mémoire s'est efforcé d'établir, dans l'esprit de rigueur scientifique et de curiosité intellectuelle qui doit animer, plus largement, toute démarche de recherche en science des données.

Bibliographie

Les références sont présentées selon le format IEEE, numérotées dans l'ordre alphabétique des auteurs.

[1] H. Attias, « A variational Bayesian framework for graphical models », in Advances in Neural Information Processing Systems 12 (NIPS 1999), MIT Press, 2000, p. 209-215.

[2] L. E. Baum, T. Petrie, G. Soules et N. Weiss, « A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains », The Annals of Mathematical Statistics, vol. 41, n° 1, p. 164-171, 1970.

[3] C. M. Bishop, Pattern Recognition and Machine Learning. New York : Springer, 2006.

[4] D. M. Blei, A. Kucukelbir et J. D. McAuliffe, « Variational inference: A review for statisticians », Journal of the American Statistical Association, vol. 112, n° 518, p. 859-877, 2017.

[5] D. M. Blei, A. Y. Ng et M. I. Jordan, « Latent Dirichlet Allocation », Journal of Machine Learning Research, vol. 3, p. 993-1022, 2003.

[6] R. A. Boyles, « On the convergence of the EM algorithm », Journal of the Royal Statistical Society: Series B, vol. 45, n° 1, p. 47-50, 1983.

[7] O. Cappé et E. Moulines, « On-line expectation-maximization algorithm for latent data models », Journal of the Royal Statistical Society: Series B, vol. 71, n° 3, p. 593-613, 2009.

[8] G. Celeux et J. Diebolt, « The SEM algorithm: A probabilistic teacher algorithm derived from the EM algorithm for the mixture problem », Computational Statistics Quarterly, vol. 2, n° 1, p. 73-82, 1985.

[9] A. Corduneanu et C. M. Bishop, « Variational Bayesian model selection for mixture distributions », in Artificial Intelligence and Statistics 2001 (AISTATS), Morgan Kaufmann, 2001, p. 27-34.

[10] A. P. Dempster, N. M. Laird et D. B. Rubin, « Maximum likelihood from incomplete data via the EM algorithm », Journal of the Royal Statistical Society: Series B, vol. 39, n° 1, p. 1-38, 1977.

[11] Federal Reserve Bank of St. Louis, « S&P 500 [SP500] », série FRED fondée sur les données de S&P Dow Jones Indices LLC, https://fred.stlouisfed.org/series/SP500, consultée en octobre 2026.

[12] R. A. Fisher, « The use of multiple measurements in taxonomic problems », Annals of Eugenics, vol. 7, n° 2, p. 179-188, 1936.

[13] H. O. Hartley, « Maximum likelihood estimation from incomplete data »,

Biometrics, vol. 14, n° 2, p. 174-194, 1958.

[14] G. E. Hinton et D. van Camp, « Keeping the neural networks simple by minimizing the description length of the weights », in Proceedings of the 6th Annual Conference on Computational Learning Theory (COLT), 1993, p. 5-13.

[15] M. D. Hoffman, D. M. Blei, C. Wang et J. Paisley, « Stochastic variational inference », Journal of Machine Learning Research, vol. 14, n° 1, p. 1303-1347, 2013.

[16] M. I. Jordan, Z. Ghahramani, T. S. Jaakkola et L. K. Saul, « An introduction to variational methods for graphical models », Machine Learning, vol. 37, n° 2, p. 183-233, 1999.

[17] D. P. Kingma et M. Welling, « Auto-encoding variational Bayes », in Proceedings of the 2nd International Conference on Learning Representations (ICLR), 2014.

[18] P. H. Kupiec, « Techniques for verifying the accuracy of risk measurement models », The Journal of Derivatives, vol. 3, n° 2, p. 73-84, 1995.

[19] T. A. Louis, « Finding the observed information matrix when using the EM algorithm », Journal of the Royal Statistical Society: Series B, vol. 44, n° 2, p. 226-233, 1982.

[20] G. J. McLachlan et T. Krishnan, The EM Algorithm and Extensions, 2e éd. Hoboken : Wiley, 2008.

[21] R. M. Neal et G. E. Hinton, « A view of the EM algorithm that justifies incremental, sparse, and other variants », in Learning in Graphical Models, M. I. Jordan, Éd. Dordrecht : Springer, 1998, p. 355-368.

[22] T. Orchard et M. A. Woodbury, « A missing information principle: Theory and applications », in Proceedings of the 6th Berkeley Symposium on Mathematical Statistics and Probability, vol. 1, 1972, p. 697-715.

[23] A. Paszke et al., « PyTorch: An imperative style, high-performance deep learning library », in Advances in Neural Information Processing Systems 32 (NeurIPS 2019), 2019, p. 8024-8035.

[24] C. Peterson et J. R. Anderson, « A mean field theory learning algorithm for neural networks », Complex Systems, vol. 1, n° 5, p. 995-1019, 1987.

[25] C. E. Rasmussen, « The infinite Gaussian mixture model », in Advances in Neural Information Processing Systems 12 (NIPS 1999), MIT Press, 2000, p. 554-560.

[26] D. J. Rezende et S. Mohamed, « Variational inference with normalizing flows », in Proceedings of the 32nd International Conference on Machine Learning (ICML), 2015, p. 1530-1538.

[27] M. Sato et S. Ishii, « On-line EM algorithm for the normalized Gaussian network », Neural Computation, vol. 12, n° 2, p. 407-432, 2000.

[28] C. Stauffer et W. E. L. Grimson, « Adaptive background mixture models for real-time tracking », in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), vol. 2, 1999, p. 246-252.

[29] C. F. J. Wu, « On the convergence properties of the EM algorithm », The Annals of Statistics, vol. 11, n° 1, p. 95-103, 1983.

Annexes

Annexe A : Code source

Les résultats de ce mémoire sont produits par quatre fichiers Python (PyTorch, Pandas, Plotly) : `algorithmes.py` (EM classique, EM variationnel, ELBO, variantes, Baum-Welch, LDA, métriques), `risque.py` (outils de gestion du risque pour les mélanges univariés), `main.py` (expériences du chapitre 2 et validations du chapitre 1) et `cas_pratique.py` (application aux rendements boursiers). Le tableau suivant indique, pour chaque fonction de la bibliothèque, la section et l'équation correspondantes.

Fonction	**Section**	**Équations**
em_classique	1.6	(1.15), (1.16), (1.18)
em_stochastique, em_online	1.6.5	Celeux et Diebolt, Cappé et Moulines
forward_backward, baum_welch	1.6.6	forward et backward, tableau 1.4
em_variationnel	1.7	(1.26) à (1.30)
elbo_gmm_bayesien	1.7.6	(1.31)
svi_gmm	1.7.7	Hoffman et al.
lda_variationnel	3.1.1	(3.1), (3.2)
bic, aic	2.2.3	(2.4), (2.5)

Table 3.9: Correspondance entre fonctions du code, sections et équations

A.1 Bibliothèque d'algorithmes (algorithmes.py)

```
"""
================================================================================
EM CLASSIQUE ET EM VARIATIONNEL POUR LES MODÈLES DE MÉLANGE (PyTorch pur)
Kana Tiodong Icare Brichel, Master 2 SDIA, ENSPD, Université de Douala
================================================================================

Implémentation sans scikit-learn, sans SciPy et sans NumPy. Seules les opérations
élémentaires de PyTorch sont utilisées (algèbre linéaire, digamma, lgamma) ; les
modèles, les critères et les métriques sont écrits à la main.

Correspondance entre fonctions, sections et équations du texte :

    Section 1.6   EM classique pour le GMM        em_classique()        (1.15) (1.16) (1.18)
    Section 1.6.5 EM stochastique (SEM)           em_stochastique()
    Section 1.6.5 EM online                       em_online()
    Section 1.6.6 Baum-Welch (HMM)                forward_backward(), baum_welch()
    Section 1.7   EM variationnel pour le GMM     em_variationnel()     (1.26) (1.27) (1.28) (1.29) (1.30)
    Section 1.7.6 ELBO                            elbo_gmm_bayesien()   (1.31)
    Section 1.7.7 Inférence variationnelle stoch. svi_gmm()
    Section 2.2.3 BIC / AIC                       bic(), aic()          (2.4) (2.5)
    Section 3.1.1 LDA variationnel                lda_variationnel()    (3.2)
    Section 2.3.4 ARI, silhouette                 indice_rand_ajuste(), silhouette()
```

```
Critère d'arrêt : on s'arrête quand |critère(t) - critère(t-1)| < tol, le critère
étant la log-vraisemblance TOTALE (EM, option par observation disponible) ou
l'ELBO TOTAL (EM variationnel).
"""

from __future__ import annotations

import math
import time
from dataclasses import dataclass

import torch
from torch import Tensor

torch.set_default_dtype(torch.float64)

LOG2PI = math.log(2.0 * math.pi)
LN2 = math.log(2.0)
LNPI = math.log(math.pi)
TYPES_COVARIANCE = ("full", "tied", "diag", "spherical")


# ============================================================================
# OUTILS GÉNÉRAUX
# ============================================================================

def generateur(graine: int) -> torch.Generator:
    g = torch.Generator()
    g.manual_seed(int(graine))
    return g


def logsumexp_stable(a: Tensor, dim: int = 1, keepdim: bool = True) -> Tensor:
    """Équation (1.18) : log sum exp(a_k) = a_max + log sum exp(a_k - a_max)."""
    a_max = a.max(dim=dim, keepdim=True).values
    out = a_max + torch.log(torch.exp(a - a_max).sum(dim=dim, keepdim=True))
    return out if keepdim else out.squeeze(dim)


def generer_blobs(n_par_groupe, centres, ecart_type, graine=42):
    """Mélange de gaussiennes isotropes (écart-type `ecart_type`), groupes de taille égale."""
    g = generateur(graine)
    centres = torch.as_tensor(centres, dtype=torch.float64)
    C, D = centres.shape
    y = torch.arange(C).repeat_interleave(n_par_groupe)
    X = centres[y] + ecart_type * torch.randn(len(y), D, generator=g)
    perm = torch.randperm(len(y), generator=g)
    return X[perm], y[perm]


def jeu_reference(graine=42):
    """Jeu de données du chapitre 2 : N = 300, centres (0,0), (5,5), (0,5), écart-type 0,7."""
    return generer_blobs(100, [[0.0, 0.0], [5.0, 5.0], [0.0, 5.0]], 0.7, graine)


def kmeans(X: Tensor, K: int, graine: int = 0, n_iter: int = 100):
    """K-means (Lloyd) avec initialisation k-means++. Retourne (étiquettes, centres)."""
    g = generateur(graine)
    N, D = X.shape
    centres = X[torch.randint(N, (1,), generator=g)].clone()
    for _ in range(1, K):
        d2 = torch.cdist(X, centres).pow(2).min(dim=1).values
        p = d2 / d2.sum() if d2.sum() > 0 else torch.full((N,), 1.0 / N)
        centres = torch.cat([centres, X[torch.multinomial(p, 1, generator=g)]], dim=0)
    for _ in range(n_iter):
        etiq = torch.cdist(X, centres).argmin(dim=1)
        nouveaux = centres.clone()
        for k in range(K):
            masque = etiq == k
            if masque.any():
                nouveaux[k] = X[masque].mean(dim=0)
        if torch.allclose(nouveaux, centres):
            break
        centres = nouveaux
    return torch.cdist(X, centres).argmin(dim=1), centres


def nb_parametres(K: int, D: int, cov_type: str) -> int:
    """Nombre de paramètres libres p(K) intervenant dans le BIC et l'AIC."""
```

```
    cov = {"full": K * D * (D + 1) // 2, "tied": D * (D + 1) // 2,
           "diag": K * D, "spherical": K}[cov_type]
    return cov + K * D + (K - 1)


def bic(loglik: float, p: int, N: int) -> float:
    """Équation (2.4)."""
    return -2.0 * loglik + p * math.log(N)


def aic(loglik: float, p: int) -> float:
    """Équation (2.5)."""
    return -2.0 * loglik + 2.0 * p


def log_gauss(X: Tensor, mu: Tensor, cov: Tensor) -> Tensor:
    """log N(x | mu, cov) pour toutes les lignes de X, via Cholesky (section 2.2.2)."""
    D = X.shape[1]
    L = torch.linalg.cholesky(cov)
    z = torch.linalg.solve_triangular(L, (X - mu).T, upper=False)
    logdet = 2.0 * torch.log(torch.diagonal(L)).sum()
    return -0.5 * (D * LOG2PI + logdet + (z ** 2).sum(dim=0))


# ============================================================================
# MÉTRIQUES DE PARTITION (écrites à la main)
# ============================================================================

def indice_rand_ajuste(y_vrai: Tensor, y_pred: Tensor) -> float:
    """ARI de Hubert et Arabie, à partir de la table de contingence."""
    _, a = torch.unique(y_vrai, return_inverse=True)
    _, b = torch.unique(y_pred, return_inverse=True)
    n = len(a)
    cont = torch.zeros(int(a.max()) + 1, int(b.max()) + 1)
    cont.index_put_((a, b), torch.ones(n), accumulate=True)

    def comb2(x):
        return x * (x - 1) / 2.0

    somme_ij = comb2(cont).sum()
    somme_a = comb2(cont.sum(1)).sum()
    somme_b = comb2(cont.sum(0)).sum()
    attendu = somme_a * somme_b / comb2(torch.tensor(float(n)))
    maxi = 0.5 * (somme_a + somme_b)
    if maxi == attendu:
        return 1.0
    return float((somme_ij - attendu) / (maxi - attendu))


def silhouette(X: Tensor, etiq: Tensor) -> float:
    """Score de silhouette moyen (distance euclidienne)."""
    _, e = torch.unique(etiq, return_inverse=True)
    C = int(e.max()) + 1
    if C < 2:
        return 0.0
    dist = torch.cdist(X, X)
    onehot = torch.nn.functional.one_hot(e, C).to(X.dtype)
    tailles = onehot.sum(0)
    somme = dist @ onehot                       # (N, C) : somme des distances à chaque groupe
    N = len(e)
    idx = torch.arange(N)
    taille_propre = tailles[e]
    a = somme[idx, e] / (taille_propre - 1).clamp(min=1)
    moy = somme / tailles.clamp(min=1)
    moy[idx, e] = float("inf")
    b = moy.min(dim=1).values
    s = (b - a) / torch.maximum(a, b)
    s = torch.where(taille_propre > 1, s, torch.zeros_like(s))
    return float(s.mean())


# ============================================================================
# EM CLASSIQUE POUR LE GMM (sections 1.6.2 et 1.6.3)
# ============================================================================

def _covariances_em(X, r, mu, Nk, cov_type, reg):
    """Mise à jour des covariances (1.16) selon la contrainte de forme choisie."""
    K, D = mu.shape
    diff = X[None, :, :] - mu[:, None, :]
    S = torch.einsum("nk,knd,kne->kde", r, diff, diff)
```

```
    I = torch.eye(D)
    if cov_type == "full":
        cov = S / Nk[:, None, None]
    elif cov_type == "tied":
        cov = (S.sum(0) / Nk.sum()).expand(K, D, D).clone()
    elif cov_type == "diag":
        cov = torch.diag_embed(torch.diagonal(S, dim1=1, dim2=2) / Nk[:, None])
    elif cov_type == "spherical":
        v = torch.diagonal(S, dim1=1, dim2=2).sum(-1) / (D * Nk)
        cov = v[:, None, None] * I
    else:
        raise ValueError(cov_type)
    return cov + reg * I


def _etape_m(X, r, cov_type, reg):
    """Étape M (1.16) : pi_k = N_k/N, mu_k = moyenne pondérée, Sigma_k = covariance pondérée."""
    N = X.shape[0]
    Nk = r.sum(0) + 1e-10
    pi = Nk / N
    mu = (r.T @ X) / Nk[:, None]
    cov = _covariances_em(X, r, mu, Nk, cov_type, reg)
    return pi, mu, cov


def _etape_e(X, pi, mu, cov):
    """Étape E (1.15) en log-espace : retourne (r, log p(x_i | theta))."""
    K = len(pi)
    log_r = torch.stack([torch.log(pi[k] + 1e-300) + log_gauss(X, mu[k], cov[k])
                         for k in range(K)], dim=1)
    log_norm = logsumexp_stable(log_r, dim=1)
    return torch.exp(log_r - log_norm), log_norm.squeeze(1)


def _init_em(X, K, cov_type, init, graine, reg):
    N, D = X.shape
    if init == "kmeans":
        etiq, _ = kmeans(X, K, graine)
        r0 = torch.nn.functional.one_hot(etiq, K).to(X.dtype)
        return _etape_m(X, r0, cov_type, reg)
    g = generateur(graine)                                  # initialisation aléatoire (1.6.2.a)
    idx = torch.randperm(N, generator=g)[:K]
    mu = X[idx].clone()
    cov0 = torch.cov(X.T).reshape(D, D) + reg * torch.eye(D)
    return torch.full((K,), 1.0 / K), mu, cov0.expand(K, D, D).clone()


def _em_une_fois(X, K, cov_type, n_iter, tol, reg, init, graine, tol_par_obs):
    N, D = X.shape
    pi, mu, cov = _init_em(X, K, cov_type, init, graine, reg)
    hist, converge = [], False
    for it in range(n_iter):
        r, log_norm = _etape_e(X, pi, mu, cov)
        hist.append(float(log_norm.sum()))
        ecart = abs(hist[-1] - hist[-2]) / (N if tol_par_obs else 1.0) if it > 0 else float("inf")
        if ecart < tol:
            converge = True
            break
        pi, mu, cov = _etape_m(X, r, cov_type, reg)
    r, log_norm = _etape_e(X, pi, mu, cov)
    return {"pi": pi, "mu": mu, "cov": cov, "r": r, "loglik": float(log_norm.sum()),
            "ll_hist": hist, "n_iter": len(hist), "converge": converge}


def em_classique(X, K, cov_type="full", n_iter=200, tol=1e-3, reg=1e-6,
                 init="kmeans", graine=0, n_init=1, tol_par_obs=True):
    """
    EM classique pour un mélange de K gaussiennes.

    init : "kmeans" (par défaut, comme la pratique courante) ou "aleatoire"
           (K points tirés au hasard, covariance globale, section 1.6.2.a).
    n_init : nombre de relances, on garde la plus grande log-vraisemblance.
    tol_par_obs : True  -> arrêt si |delta log-vraisemblance| / N < tol (convention de
                           GaussianMixture, utilisée par le protocole du chapitre 2) ;
                  False -> arrêt sur la variation totale (section 1.6.2.a).
    """
    t0 = time.perf_counter()
    meilleur = None
    for s in range(n_init):
        res = _em_une_fois(X, K, cov_type, n_iter, tol, reg, init, graine + s, tol_par_obs)
```

```
        if meilleur is None or res["loglik"] > meilleur["loglik"]:
            meilleur = res
    N, D = X.shape
    p = nb_parametres(K, D, cov_type)
    meilleur.update({
        "Nk": meilleur["r"].sum(0), "labels": meilleur["r"].argmax(1),
        "n_params": p, "bic": bic(meilleur["loglik"], p, N),
        "aic": aic(meilleur["loglik"], p), "temps": time.perf_counter() - t0,
        "cov_type": cov_type})
    return meilleur


def energie_libre_em(X, r, pi, mu, cov):
    """
    F(q, theta) = E_q[log p(X,Z|theta)] - E_q[log q(Z)] (section 1.2.2).
    Après une étape E, q(Z) = p(Z|X,theta) et F = log-vraisemblance (égalité de Jensen).
    """
    K = len(pi)
    log_joint = torch.stack([torch.log(pi[k]) + log_gauss(X, mu[k], cov[k]) for k in range(K)], 1)
    return float((r * log_joint).sum() - torch.xlogy(r, r).sum())


# ============================================================================
# VARIANTES DE L'EM CLASSIQUE (section 1.6.5)
# ============================================================================

def em_stochastique(X, K, n_iter=200, reg=1e-6, graine=0, burn=50):
    """
    EM stochastique (Celeux et Diebolt, 1985) : l'étape E devient un tirage
    z_i ~ Categorielle(r_i), l'étape M est faite sur ces affectations dures.
    La log-vraisemblance n'est plus monotone. On renvoie la moyenne des
    paramètres après `burn` itérations (les composantes vides sont réinitialisées).
    """
    g = generateur(graine)
    N, D = X.shape
    pi, mu, cov = _init_em(X, K, "full", "kmeans", graine, reg)
    hist, accum = [], []
    for it in range(n_iter):
        r, log_norm = _etape_e(X, pi, mu, cov)
        hist.append(float(log_norm.sum()))
        z = torch.multinomial(r, 1, generator=g).squeeze(1)
        dur = torch.nn.functional.one_hot(z, K).to(X.dtype)
        if (dur.sum(0) < D + 1).any():                          # composante quasi vide : on la ré-ancre
            vides = (dur.sum(0) < D + 1).nonzero().flatten()
            for k in vides:
                mu[k] = X[torch.randint(N, (1,), generator=g)].squeeze(0)
                cov[k] = torch.cov(X.T).reshape(D, D) + reg * torch.eye(D)
            continue
        pi, mu, cov = _etape_m(X, dur, "full", reg)
        if it >= burn:
            accum.append((pi.clone(), mu.clone(), cov.clone()))
    if accum:
        pi = torch.stack([a[0] for a in accum]).mean(0)
        mu = torch.stack([a[1] for a in accum]).mean(0)
        cov = torch.stack([a[2] for a in accum]).mean(0)
    r, log_norm = _etape_e(X, pi, mu, cov)
    return {"pi": pi, "mu": mu, "cov": cov, "r": r, "loglik": float(log_norm.sum()),
            "ll_hist": hist, "labels": r.argmax(1)}


def em_online(X, K, taille_lot=50, n_epoques=5, kappa=0.6, t0=10.0, reg=1e-6, graine=0):
    """
    EM online par statistiques suffisantes (Sato et Ishii 2000, Cappé et Moulines 2009).
    s <- (1 - eta_t) s + eta_t * s_lot avec eta_t = (t + t0)^(-kappa), kappa dans (0.5, 1]
    (conditions de type Robbins-Monro de la section 1.6.5).
    """
    g = generateur(graine)
    N, D = X.shape
    sous = X[torch.randperm(N, generator=g)[:min(N, 500)]]
    pi, mu, cov = _init_em(sous, K, "full", "kmeans", graine, reg)
    s0 = pi.clone()
    s1 = pi[:, None] * mu
    s2 = pi[:, None, None] * (cov + mu[:, :, None] * mu[:, None, :])
    hist, t = [], 0
    for _ in range(n_epoques):
        perm = torch.randperm(N, generator=g)
        for debut in range(0, N, taille_lot):
            lot = X[perm[debut:debut + taille_lot]]
            r, _ = _etape_e(lot, pi, mu, cov)
            eta = (t + t0) ** (-kappa)
```

```
            s0 = (1 - eta) * s0 + eta * r.mean(0)
            s1 = (1 - eta) * s1 + eta * (r.T @ lot) / len(lot)
            s2 = (1 - eta) * s2 + eta * torch.einsum("nk,nd,ne->kde", r, lot, lot) / len(lot)
            pi = s0 / s0.sum()
            mu = s1 / s0[:, None]
            cov = s2 / s0[:, None, None] - mu[:, :, None] * mu[:, None, :] + reg * torch.eye(D)
            t += 1
        _, ln = _etape_e(X, pi, mu, cov)
        hist.append(float(ln.sum()))
    r, ln = _etape_e(X, pi, mu, cov)
    return {"pi": pi, "mu": mu, "cov": cov, "r": r, "loglik": float(ln.sum()),
            "ll_hist": hist, "labels": r.argmax(1)}


# ============================================================================
# EM VARIATIONNEL POUR LE GMM BAYÉSIEN (section 1.7)
# ============================================================================

@dataclass
class PrioriNW:
    """Hyperparamètres : Dirichlet(alpha0) sur pi, Normal-Wishart(m0, beta0, W0, nu0) sur (mu, Lambda)."""
    alpha0: float
    m0: Tensor
    beta0: float
    W0inv: Tensor
    nu0: float


def priori_diffus(X, alpha0=None, K=None, beta0=1e-3):
    """Priors diffus : m0 = moyenne, beta0 = 1e-3, W0 = I, nu0 = D."""
    D = X.shape[1]
    if alpha0 is None:
        alpha0 = 1.0 / K
    return PrioriNW(alpha0, X.mean(0), beta0, torch.eye(D), float(D))


def priori_empirique(X, alpha0=None, K=None):
    """Priors empiriques : beta0 = 1, nu0 = D, W0^-1 = covariance empirique, m0 = moyenne."""
    D = X.shape[1]
    if alpha0 is None:
        alpha0 = 1.0 / K
    return PrioriNW(alpha0, X.mean(0), 1.0, torch.cov(X.T).reshape(D, D), float(D))


def _ln_B(W, nu, D):
    """log de la constante de normalisation de la Wishart, ln B(W, nu)."""
    i = torch.arange(1, D + 1, dtype=torch.float64)
    lg = torch.lgamma((nu[..., None] + 1 - i) / 2).sum(-1)
    return -0.5 * nu * torch.logdet(W) - (0.5 * nu * D * LN2 + D * (D - 1) / 4 * LNPI + lg)


def _stats(X, r):
    Nk = r.sum(0) + 1e-10
    xbar = (r.T @ X) / Nk[:, None]
    diff = X[None, :, :] - xbar[:, None, :]
    Sk = torch.einsum("nk,knd,kne->kde", r, diff, diff) / Nk[:, None, None]
    return Nk, xbar, Sk


def _etape_m_var(Nk, xbar, Sk, pr: PrioriNW, cov_type):
    """Étape M variationnelle, équations (1.26) et (1.28), plus les variantes de covariance."""
    K, D = xbar.shape
    beta = pr.beta0 + Nk
    m = (pr.beta0 * pr.m0 + Nk[:, None] * xbar) / beta[:, None]
    d0 = xbar - pr.m0
    terme_prior = (pr.beta0 * Nk / (pr.beta0 + Nk))
    sc = Nk[:, None, None] * Sk + terme_prior[:, None, None] * d0[:, :, None] * d0[:, None, :]
    P = {"alpha": pr.alpha0 + Nk, "beta": beta, "m": m, "cov_type": cov_type}
    if cov_type == "full":
        P["nu"] = pr.nu0 + Nk
        P["Winv"] = pr.W0inv[None] + sc
        P["W"] = torch.linalg.inv(P["Winv"])
    elif cov_type == "tied":
        P["nu"] = torch.full((K,), pr.nu0 + float(Nk.sum()))
        Winv = pr.W0inv + sc.sum(0)
        P["Winv"] = Winv.expand(K, D, D).clone()
        P["W"] = torch.linalg.inv(Winv).expand(K, D, D).clone()
    else:                                       # diag, spherical : priors Gamma sur les précisions
        a0 = pr.nu0 / 2.0
        b0 = 0.5 * torch.diagonal(pr.W0inv)
```

```
        sd = torch.diagonal(sc, dim1=1, dim2=2)
        if cov_type == "diag":
            P["a"] = (a0 + Nk / 2.0)[:, None].expand(K, D).clone()
            P["b"] = b0[None] + 0.5 * sd
        else:
            P["a"] = (a0 + D * Nk / 2.0)[:, None].expand(K, D).clone()
            P["b"] = (b0.mean() + 0.5 * sd.sum(1))[:, None].expand(K, D).clone()
    return P


def _attendus(P, D):
    """E[ln|Lambda_k|] (1.29) et E[Lambda_k] pour chaque composante."""
    if P["cov_type"] in ("full", "tied"):
        i = torch.arange(1, D + 1, dtype=torch.float64)
        Elogdet = (torch.special.digamma((P["nu"][:, None] + 1 - i) / 2).sum(-1)
                   + D * LN2 + torch.logdet(P["W"]))
        Elam = P["nu"][:, None, None] * P["W"]
    else:
        psi = torch.special.digamma(P["a"]) - torch.log(P["b"])
        Elogdet = psi.sum(-1)
        Elam = torch.diag_embed(P["a"] / P["b"])
    return Elogdet, Elam


def _etape_e_var(X, P):
    """Étape E variationnelle, équation (1.27), avec la forme quadratique (1.30)."""
    N, D = X.shape
    Elogpi = torch.special.digamma(P["alpha"]) - torch.special.digamma(P["alpha"].sum())
    Elogdet, Elam = _attendus(P, D)
    diff = X[None, :, :] - P["m"][:, None, :]
    quad = D / P["beta"][None, :] + torch.einsum("knd,kde,kne->nk", diff, Elam, diff)
    log_rho = Elogpi[None] + 0.5 * Elogdet[None] - 0.5 * D * LOG2PI - 0.5 * quad
    log_norm = logsumexp_stable(log_rho, dim=1)
    return torch.exp(log_rho - log_norm), log_norm.squeeze(1)


def elbo_gmm_bayesien(r, Nk, xbar, Sk, P, pr: PrioriNW):
    """
    ELBO du GMM bayésien (équation 1.31, détail dans Bishop 2006, éq. 10.70 à 10.77).
    Disponible pour la covariance pleine.
    """
    K, D = xbar.shape
    alpha, beta, m, nu, W = P["alpha"], P["beta"], P["m"], P["nu"], P["W"]
    Elogpi = torch.special.digamma(alpha) - torch.special.digamma(alpha.sum())
    Elogdet, _ = _attendus(P, D)
    dm = xbar - m
    t_trace = torch.einsum("kde,ked->k", Sk, W)
    t_quad = torch.einsum("kd,kde,ke->k", dm, W, dm)
    e_x = 0.5 * (Nk * (Elogdet - D / beta - nu * t_trace - nu * t_quad - D * LOG2PI)).sum()
    e_z = (Nk * Elogpi).sum()
    a0 = pr.alpha0
    ln_c0 = math.lgamma(K * a0) - K * math.lgamma(a0)
    e_pi = ln_c0 + (a0 - 1.0) * Elogpi.sum()
    W0 = torch.linalg.inv(pr.W0inv)
    d0 = m - pr.m0
    quad_m = torch.einsum("kd,kde,ke->k", d0, W, d0)
    nu0_t = torch.tensor(pr.nu0)
    lnB0 = _ln_B(W0, nu0_t, D)
    e_muLam = (0.5 * (D * math.log(pr.beta0 / (2 * math.pi)) + Elogdet - D * pr.beta0 / beta
                      - pr.beta0 * nu * quad_m).sum()
               + K * lnB0 + 0.5 * (pr.nu0 - D - 1) * Elogdet.sum()
               - 0.5 * (nu * torch.einsum("de,ked->k", pr.W0inv, W)).sum())
    q_z = torch.xlogy(r, r).sum()
    ln_c = torch.lgamma(alpha.sum()) - torch.lgamma(alpha).sum()
    q_pi = ((alpha - 1.0) * Elogpi).sum() + ln_c
    H = -_ln_B(W, nu, D) - 0.5 * (nu - D - 1) * Elogdet + 0.5 * nu * D
    q_muLam = (0.5 * Elogdet + 0.5 * D * torch.log(beta / (2 * math.pi)) - 0.5 * D - H).sum()
    return float(e_x + e_z + e_pi + e_muLam - q_z - q_pi - q_muLam)


def _init_params_var(X, K, pr, cov_type, init, graine):
    """Retourne (P, r0). `prior` : paramètres de 1.7.3.d ; `kmeans` : r0 durs, puis étape M."""
    N, D = X.shape
    if init == "kmeans":
        etiq, _ = kmeans(X, K, graine)
        r0 = torch.nn.functional.one_hot(etiq, K).to(X.dtype)
        Nk, xbar, Sk = _stats(X, r0)
        return _etape_m_var(Nk, xbar, Sk, pr, cov_type), r0
    g = generateur(graine)
    idx = torch.randperm(N, generator=g)[:K]
```

```
    Nk0 = torch.full((K,), N / K)
    xbar0 = X[idx].clone()
    Sk0 = torch.eye(D).expand(K, D, D).clone()
    P = _etape_m_var(Nk0, xbar0, Sk0, pr, cov_type)
    P["m"] = xbar0                                  # m_k = point de données tiré au hasard
    if cov_type in ("full", "tied"):
        P["Winv"] = pr.W0inv.expand(K, D, D).clone()    # W_k = W_0 (section 1.7.3.d)
        P["W"] = torch.linalg.inv(pr.W0inv).expand(K, D, D).clone()
    return P, None


def _em_var_une_fois(X, K, pr, cov_type, n_iter, tol, init, graine):
    N, D = X.shape
    P, r = _init_params_var(X, K, pr, cov_type, init, graine)
    hist, converge = [], False
    for it in range(n_iter):
        r, log_norm = _etape_e_var(X, P)
        Nk, xbar, Sk = _stats(X, r)
        P = _etape_m_var(Nk, xbar, Sk, pr, cov_type)
        crit = (elbo_gmm_bayesien(r, Nk, xbar, Sk, P, pr) if cov_type == "full"
                else float(log_norm.sum()))
        hist.append(crit)
        if it > 0 and abs(hist[-1] - hist[-2]) < tol:
            converge = True
            break
    r, _ = _etape_e_var(X, P)
    return {"P": P, "r": r, "crit": hist[-1], "hist": hist, "n_iter": len(hist), "converge": converge}


def em_variationnel(X, Kmax, priori: PrioriNW | None = None, alpha0=None, cov_type="full",
                    n_iter=200, tol=1e-3, init="kmeans", graine=0, n_init=1):
    """
    EM variationnel (champ moyen) pour un GMM bayésien.

    Priors par défaut : priors empiriques (beta0 = 1, W0^-1 = covariance empirique). Les priors diffus (beta0 = 1e-3, W0 = I)
    s'obtiennent avec priori=priori_diffus(X, alpha0).

    Retourne un dictionnaire avec les poids E[pi_k], les centres m_k, les covariances
    (E[Lambda_k])^-1, les responsabilités, N_k, l'historique de l'ELBO (covariance pleine)
    et la liste des composantes actives (N_k >= 1, section 1.7.4).
    """
    t0 = time.perf_counter()
    pr = priori if priori is not None else priori_empirique(X, alpha0, Kmax)
    if alpha0 is not None and priori is not None:
        pr = PrioriNW(alpha0, pr.m0, pr.beta0, pr.W0inv, pr.nu0)
    meilleur = None
    for s in range(n_init):
        res = _em_var_une_fois(X, Kmax, pr, cov_type, n_iter, tol, init, graine + s)
        if meilleur is None or res["crit"] > meilleur["crit"]:
            meilleur = res
    P = meilleur["P"]
    D = X.shape[1]
    _, Elam = _attendus(P, D)
    Nk = meilleur["r"].sum(0)
    alpha = P["alpha"]
    meilleur.update({
        "pi": alpha / alpha.sum(), "alpha": alpha, "m": P["m"], "Nk": Nk,
        "cov": torch.linalg.inv(Elam), "labels": meilleur["r"].argmax(1),
        "actives": (Nk >= 1.0).nonzero().flatten(), "n_actives": int((Nk >= 1.0).sum()),
        "elbo_hist": meilleur["hist"] if cov_type == "full" else None,
        "elbo": meilleur["crit"] if cov_type == "full" else None,
        "temps": time.perf_counter() - t0, "priori": pr, "cov_type": cov_type})
    return meilleur


def svi_gmm(X, Kmax, priori: PrioriNW, taille_lot=50, n_epoques=10, tau=1.0, kappa=0.7, graine=0):
    """
    Inférence variationnelle stochastique (Hoffman et al., 2013), covariance pleine.
    Les paramètres naturels globaux sont mélangés avec le pas rho_t = (t + tau)^(-kappa) :
    lambda <- (1 - rho) lambda + rho lambda_hat, où lambda_hat est calculé sur un
    mini-lot dont les statistiques sont multipliées par N/B.
    """
    g = generateur(graine)
    N, D = X.shape
    P, _ = _init_params_var(X, Kmax, priori, "full", "prior", graine)
    t = 0
    for _ in range(n_epoques):
        perm = torch.randperm(N, generator=g)
        for debut in range(0, N, taille_lot):
            lot = X[perm[debut:debut + taille_lot]]
```

```
            r, _ = _etape_e_var(lot, P)
            Nk, xbar, Sk = _stats(lot, r)
            echelle = N / len(lot)
            cible = _etape_m_var(Nk * echelle, xbar, Sk, priori, "full")
            rho = (t + tau) ** (-kappa)
            alpha = (1 - rho) * P["alpha"] + rho * cible["alpha"]
            beta = (1 - rho) * P["beta"] + rho * cible["beta"]
            nu = (1 - rho) * P["nu"] + rho * cible["nu"]
            bm = (1 - rho) * P["beta"][:, None] * P["m"] + rho * cible["beta"][:, None] * cible["m"]
            # paramètre naturel : W^-1 + beta m m^T
            nat = (1 - rho) * (P["Winv"] + P["beta"][:, None, None] * P["m"][:, :, None] * P["m"][:, None, :]) \
                + rho * (cible["Winv"] + cible["beta"][:, None, None] * cible["m"][:, :, None] * cible["m"][:, None, :])
            m = bm / beta[:, None]
            Winv = nat - beta[:, None, None] * m[:, :, None] * m[:, None, :]
            P = {"alpha": alpha, "beta": beta, "m": m, "nu": nu, "Winv": Winv,
                 "W": torch.linalg.inv(Winv), "cov_type": "full"}
            t += 1
    r, _ = _etape_e_var(X, P)
    _, Elam = _attendus(P, D)
    Nk = r.sum(0)
    return {"pi": P["alpha"] / P["alpha"].sum(), "m": P["m"], "cov": torch.linalg.inv(Elam),
            "Nk": Nk, "r": r, "labels": r.argmax(1), "n_actives": int((Nk >= 1).sum()), "P": P}


# ============================================================================
# MODÈLE DE MARKOV CACHÉ : FORWARD-BACKWARD ET BAUM-WELCH (section 1.6.6)
# ============================================================================

def forward_backward(A, B, pi0, obs):
    """
    Forward-backward NON normalisé (tableau 1.4).
    A[i,j] = P(z_{t+1}=j | z_t=i), B[i,v] = P(o_t=v | z_t=i). obs : tenseur d'entiers.
    alpha_t(i) = [sum_j alpha_{t-1}(j) A_ji] B_i(o_t), beta_t(i) = sum_j A_ij B_j(o_{t+1}) beta_{t+1}(j).
    """
    T, S = len(obs), A.shape[0]
    alpha = torch.zeros(T, S)
    beta = torch.ones(T, S)
    alpha[0] = pi0 * B[:, obs[0]]
    for t in range(1, T):
        alpha[t] = (alpha[t - 1] @ A) * B[:, obs[t]]
    for t in range(T - 2, -1, -1):
        beta[t] = A @ (B[:, obs[t + 1]] * beta[t + 1])
    proba = alpha[-1].sum()
    gamma = alpha * beta / proba
    xi = torch.zeros(T - 1, S, S)
    for t in range(T - 1):
        xi[t] = alpha[t][:, None] * A * (B[:, obs[t + 1]] * beta[t + 1])[None, :] / proba
    return {"alpha": alpha, "beta": beta, "gamma": gamma, "xi": xi, "proba": float(proba)}


def _fb_normalise(A, logB, pi0):
    """Forward-backward avec normalisation de Rabiner (stable pour les longues séquences)."""
    T, S = logB.shape
    Bm = torch.exp(logB - logB.max(1, keepdim=True).values)
    decal = logB.max(1).values
    alpha = torch.zeros(T, S)
    c = torch.zeros(T)
    a = pi0 * Bm[0]
    c[0] = a.sum()
    alpha[0] = a / c[0]
    for t in range(1, T):
        a = (alpha[t - 1] @ A) * Bm[t]
        c[t] = a.sum()
        alpha[t] = a / c[t]
    beta = torch.ones(T, S)
    for t in range(T - 2, -1, -1):
        beta[t] = (A @ (Bm[t + 1] * beta[t + 1])) / c[t + 1]
    gamma = alpha * beta
    gamma = gamma / gamma.sum(1, keepdim=True)
    xi = (alpha[:-1, :, None] * A[None] * (Bm[1:] * beta[1:])[:, None, :]) / c[1:, None, None]
    loglik = float(torch.log(c).sum() + decal.sum())
    return gamma, xi, loglik


def baum_welch(sequences, S, emission="discrete", V=None, n_iter=100, tol=1e-6, graine=0):
    """
    Baum-Welch : EM classique où l'étape E est forward-backward.
    emission = "discrete" (V symboles) ou "gauss" (observations réelles scalaires).
    Étape M : A_ij = sum xi / sum gamma, B_i(v) = sum_{o_t = v} gamma / sum gamma (discret),
              mu_i, sigma_i^2 = moyennes pondérées par gamma (gaussien).
```

```
    """
    g = generateur(graine)
    A = torch.rand(S, S, generator=g) + 1.0
    A = A / A.sum(1, keepdim=True)
    pi0 = torch.ones(S) / S
    if emission == "discrete":
        B = torch.rand(S, V, generator=g) + 1.0
        B = B / B.sum(1, keepdim=True)
    else:
        tout = torch.cat([s for s in sequences])
        q = torch.sort(tout).values
        mu = q[(torch.arange(S) * 2 + 1) * len(q) // (2 * S)].clone()
        var = torch.full((S,), float(tout.var()))
    hist = []
    for it in range(n_iter):
        gam_l, xi_l, ll = [], [], 0.0
        for obs in sequences:
            if emission == "discrete":
                logB = torch.log(B[:, obs].T)
            else:
                logB = -0.5 * (LOG2PI + torch.log(var)[None] + (obs[:, None] - mu[None]) ** 2 / var[None])
            gamma, xi, l = _fb_normalise(A, logB, pi0)
            gam_l.append(gamma); xi_l.append(xi); ll += l
        hist.append(ll)
        if it > 0 and abs(hist[-1] - hist[-2]) < tol:
            break
        pi0 = torch.stack([g_[0] for g_ in gam_l]).mean(0)
        num = sum(x.sum(0) for x in xi_l)
        den = sum(g_[:-1].sum(0) for g_ in gam_l)
        A = num / den[:, None]
        tot = sum(g_.sum(0) for g_ in gam_l)
        if emission == "discrete":
            Bn = torch.zeros(S, V)
            for obs, g_ in zip(sequences, gam_l):
                Bn.index_add_(1, obs, g_.T)
            B = Bn / tot[:, None]
        else:
            mu = sum((g_ * o[:, None]).sum(0) for o, g_ in zip(sequences, gam_l)) / tot
            var = sum((g_ * (o[:, None] - mu[None]) ** 2).sum(0)
                      for o, g_ in zip(sequences, gam_l)) / tot + 1e-9
    out = {"A": A, "pi0": pi0, "loglik": hist[-1], "ll_hist": hist, "n_iter": len(hist)}
    out.update({"B": B} if emission == "discrete" else {"mu": mu, "var": var})
    return out


def simuler_hmm(A, B, pi0, T, graine=0):
    g = generateur(graine)
    z = torch.multinomial(pi0, 1, generator=g).item()
    obs = []
    for _ in range(T):
        obs.append(torch.multinomial(B[z], 1, generator=g).item())
        z = torch.multinomial(A[z], 1, generator=g).item()
    return torch.tensor(obs)


# ============================================================================
# LDA VARIATIONNEL (section 3.1.1, équations 3.1 et 3.2)
# ============================================================================

def lda_variationnel(comptes: Tensor, K: int, alpha=0.1, n_iter=200, n_iter_doc=50,
                     tol=1e-6, graine=0, lissage=1e-3):
    """
    LDA de Blei, Ng et Jordan (2003) par EM variationnel.

    comptes : (D, V) nombres d'occurrences de chaque mot dans chaque document.
    Étape E (par document) : phi_{vk} proportionnel à beta_{kv} exp(E[log theta_k])   (éq. 3.2)
                             gamma_k = alpha + sum_v comptes_v phi_{vk}
    Étape M : beta_{kv} proportionnel à sum_d comptes_{dv} phi_{dvk}.
    Retourne aussi la borne variationnelle L(gamma, phi ; alpha, beta) à chaque itération.
    """
    g = generateur(graine)
    D, V = comptes.shape
    beta = torch.rand(K, V, generator=g) + 1.0
    beta = beta / beta.sum(1, keepdim=True)
    Nd = comptes.sum(1)
    gamma = alpha + Nd[:, None] / K
    digam = torch.special.digamma
    bornes = []
    for it in range(n_iter):
        logbeta = torch.log(beta)
```

```
        for _ in range(n_iter_doc):
            Elog = digam(gamma) - digam(gamma.sum(1, keepdim=True))                # (D,K)
            logphi = Elog[:, None, :] + logbeta.T[None, :, :]                      # (D,V,K)
            phi = torch.softmax(logphi, dim=2)
            gamma_new = alpha + (comptes[:, :, None] * phi).sum(1)
            delta = (gamma_new - gamma).abs().max()
            gamma = gamma_new
            if delta < 1e-8:
                break
        Elog = digam(gamma) - digam(gamma.sum(1, keepdim=True))
        phi = torch.softmax(Elog[:, None, :] + logbeta.T[None, :, :], dim=2)
        c = comptes[:, :, None]
        L = (math.lgamma(K * alpha) - K * math.lgamma(alpha)
             + ((alpha - 1) * Elog).sum()
             + (c * phi * Elog[:, None, :]).sum()
             + (c * phi * logbeta.T[None]).sum()
             - (torch.lgamma(gamma.sum(1)) - torch.lgamma(gamma).sum(1)).sum()
             - ((gamma - 1) * Elog).sum()
             - (c * torch.xlogy(phi, phi)).sum())
        bornes.append(float(L))
        beta = (c * phi).sum(0).T + lissage
        beta = beta / beta.sum(1, keepdim=True)
        if it > 0 and abs(bornes[-1] - bornes[-2]) < tol * abs(bornes[-2]):
            break
    theta = gamma / gamma.sum(1, keepdim=True)
    return {"beta": beta, "gamma": gamma, "theta": theta, "bornes": bornes, "n_iter": len(bornes)}
```

A.2 Outils de gestion du risque (risque.py)

```
"""
================================================================================
OUTILS DE GESTION DU RISQUE POUR LES MÉLANGES GAUSSIENS UNIVARIÉS (PyTorch pur)
Kana Tiodong Icare Brichel, Master 2 SDIA, ENSPD, Université de Douala
================================================================================

    cdf_melange_gauss()                fonction de répartition d'un mélange gaussien
    quantile_melange_gauss()           quantile (value-at-risk) par dichotomie
    es_melange_gauss()                 perte moyenne au-delà du quantile (Expected Shortfall)
    echantillonner_posterieur_gmm_1d()  tirages de (pi, mu, sigma) sous q(pi) q(mu, Lambda)  (1.23)
    log_predictive_variationnel()      log densité prédictive a posteriori, mélange de lois de Student
                                       (Bishop 2006, éq. 10.81 et 10.82)
    kupiec()                           test de couverture inconditionnelle de Kupiec (1995)
"""

from __future__ import annotations

import math

import torch
from torch import Tensor

from algorithmes import generateur

SQRT2 = math.sqrt(2.0)
SQRT2PI = math.sqrt(2.0 * math.pi)


def _en_2d(*tenseurs):
    return [t if t.dim() == 2 else t[None] for t in tenseurs]


def cdf_melange_gauss(x: Tensor, pi: Tensor, mu: Tensor, sigma: Tensor) -> Tensor:
    """F(x) = somme_k pi_k Phi((x - mu_k) / sigma_k). x : (S,) ; pi, mu, sigma : (S, K) ou (K,)."""
    pi, mu, sigma = _en_2d(pi, mu, sigma)
    z = (x.reshape(-1, 1) - mu) / sigma
    return (pi * 0.5 * (1.0 + torch.erf(z / SQRT2))).sum(-1)


def quantile_melange_gauss(p: float, pi: Tensor, mu: Tensor, sigma: Tensor, n_iter: int = 100) -> Tensor:
    """Quantile d'ordre p de chaque mélange (une ligne par jeu de paramètres), par dichotomie."""
    pi, mu, sigma = _en_2d(pi, mu, sigma)
    bas = (mu - 12.0 * sigma).min(1).values
    haut = (mu + 12.0 * sigma).max(1).values
    for _ in range(n_iter):
        milieu = 0.5 * (bas + haut)
```

```
        trop_bas = cdf_melange_gauss(milieu, pi, mu, sigma) < p
        bas = torch.where(trop_bas, milieu, bas)
        haut = torch.where(trop_bas, haut, milieu)
    return 0.5 * (bas + haut)


def es_melange_gauss(p: float, q: Tensor, pi: Tensor, mu: Tensor, sigma: Tensor) -> Tensor:
    """
    ES_p = E[X | X <= q_p] = (1 / p) somme_k pi_k [mu_k Phi(z_k) - sigma_k phi(z_k)], z_k = (q - mu_k) / sigma_k.
    Valeur négative (perte exprimée comme un rendement).
    """
    pi, mu, sigma = _en_2d(pi, mu, sigma)
    z = (q.reshape(-1, 1) - mu) / sigma
    Phi = 0.5 * (1.0 + torch.erf(z / SQRT2))
    phi = torch.exp(-0.5 * z ** 2) / SQRT2PI
    return (pi * (mu * Phi - sigma * phi)).sum(-1) / p


def echantillonner_posterieur_gmm_1d(P: dict, n: int, graine: int = 0):
    """
    Tire n jeux (pi, mu, sigma) de la loi variationnelle q (D = 1) :
        pi ~ Dirichlet(alpha), Lambda_k ~ Wishart(W_k, nu_k) = Gamma(forme nu_k / 2, taux 1 / (2 W_k)),
        mu_k | Lambda_k ~ Normale(m_k, 1 / (beta_k Lambda_k)), sigma_k = Lambda_k^(-1/2).
    P est le dictionnaire de paramètres variationnels renvoyé par em_variationnel() (clé "P").
    """
    g = generateur(graine)
    K = P["alpha"].shape[0]
    gam = torch._standard_gamma(P["alpha"].expand(n, K).contiguous(), generator=g)
    pi = gam / gam.sum(1, keepdim=True)
    forme = (P["nu"] / 2.0).expand(n, K).contiguous()
    W = P["W"].reshape(K)
    lam = torch._standard_gamma(forme, generator=g) * 2.0 * W[None, :]
    eps = torch.randn(n, K, generator=g)
    mu = P["m"].reshape(K)[None, :] + eps / torch.sqrt(P["beta"][None, :] * lam)
    return pi, mu, 1.0 / torch.sqrt(lam)


def log_predictive_variationnel(X: Tensor, P: dict) -> Tensor:
    """
    log p(x | X_train) = log somme_k E[pi_k] St(x | m_k, L_k, nu_k + 1 - D),
    L_k = ((nu_k + 1 - D) beta_k / (1 + beta_k)) W_k  (Bishop 2006, éq. 10.81 et 10.82).
    """
    N, D = X.shape
    Epi = P["alpha"] / P["alpha"].sum()
    dof = P["nu"] + 1.0 - D
    L = (dof * P["beta"] / (1.0 + P["beta"]))[:, None, None] * P["W"]
    diff = X[None, :, :] - P["m"][:, None, :]
    maha = torch.einsum("knd,kde,kne->kn", diff, L, diff)
    cste = (torch.lgamma((dof + D) / 2.0) - torch.lgamma(dof / 2.0) + 0.5 * torch.logdet(L)
            - 0.5 * D * torch.log(dof * math.pi))
    log_st = cste[:, None] - 0.5 * (dof + D)[:, None] * torch.log1p(maha / dof[:, None])
    return torch.logsumexp(torch.log(Epi)[:, None] + log_st, dim=0)


def log_densite_gauss_1d(x: Tensor, pi: Tensor, mu: Tensor, sigma: Tensor) -> Tensor:
    """log de la densité d'un mélange gaussien univarié, évaluée en chaque x."""
    z = (x.reshape(-1, 1) - mu[None, :]) / sigma[None, :]
    lg = torch.log(pi)[None, :] - torch.log(sigma)[None, :] - 0.5 * z ** 2 - math.log(SQRT2PI)
    return torch.logsumexp(lg, dim=1)


def kupiec(exceptions: int, T: int, p: float):
    """
    Test de Kupiec : sous H0 le taux d'exceptions vaut p ; LR = -2 ln [L(p) / L(x/T)] suit un khi-deux à
    1 degré de liberté. La probabilité critique vaut erfc(sqrt(LR / 2)).
    Retourne (statistique LR, probabilité critique).
    """
    x = exceptions
    ph = x / T

    def ll(q):
        terme = (T - x) * math.log(1.0 - q)
        return terme + (x * math.log(q) if x > 0 else 0.0)

    lr = max(-2.0 * (ll(p) - ll(ph)), 0.0)
    return lr, math.erfc(math.sqrt(lr / 2.0))
```

A.3 Expériences et cas pratique

Les fichiers `main.py` et `cas_pratique.py` orchestrent les expériences : génération des données, appels aux fonctions ci-dessus, calcul des métriques, tableaux (Pandas) et figures (Plotly). Ils s'exécutent par `python main.py` et `python cas_pratique.py`, et le carnet `code.ipynb` les enchaîne avec le protocole.

A.4 Corpus et résultats détaillés de la démonstration LDA

Le tableau suivant donne le corpus complet des douze documents (accents omis) et la distribution de topics obtenue pour chacun (colonnes T0, T1, T2), le topic dominant et le thème de référence (0 : animaux, 1 : finance, 2 : football).

#	Document	T0	T1	T2	Dominant	Référence
0	le chat mange des croquettes et dort sur le canape	0,96	0,02	0,02	0	0
1	le chien joue dans le jardin avec la balle	0,02	0,02	0,95	2	0
2	le chat et le chien dorment ensemble le soir	0,02	0,02	0,96	2	0
3	la bourse chute apres l annonce des taux d interet	0,02	0,02	0,96	2	1
4	les actions technologiques progressent sur le marche financier	0,02	0,02	0,96	2	1
5	la banque centrale releve son taux directeur cette semaine	0,02	0,97	0,02	1	1
6	l equipe de football remporte le match a l exterieur	0,96	0,02	0,02	0	2
7	le joueur marque un but decisif en fin de rencontre	0,02	0,97	0,02	1	2
8	le championnat de football reprend ce week end	0,96	0,02	0,02	0	2
9	le chat observe les oiseaux depuis la fenetre du salon	0,02	0,02	0,96	2	0
10	les investisseurs surveillent la reaction des marches financiers	0,02	0,96	0,02	1	1
11	l entraineur felicite ses joueurs apres la victoire	0,95	0,02	0,02	0	2

Table 3.10: Distribution topique complète des douze documents du corpus jouet

Un document sur trois n'est pas rattaché au topic de son thème de référence : sur un corpus aussi petit, le résultat dépend de l'initialisation (section 3.1.1).

Annexe B : Détail du calcul de l'ELBO pour le GMM bayésien

Cette annexe donne l'implémentation, en PyTorch, du calcul exact de l'ELBO de l'équation (1.31) (Bishop, 2006, éq. 10.70 à 10.77). Les sept termes sont : $\mathbb{E}[\log p(X|Z,\mu,\Lambda)]$, $\mathbb{E}[\log p(Z|\pi)]$, $\mathbb{E}[\log p(\pi)]$ et $\mathbb{E}[\log p(\mu,\Lambda)]$ (partie vraisemblance et priors), puis $-\mathbb{E}[\log q(Z)]$, $-\mathbb{E}[\log q(\pi)]$ et $-\mathbb{E}[\log q(\mu,\Lambda)]$ (entropies des facteurs variationnels). La fonction `_ln_B` calcule la constante de normalisation de la loi de Wishart ; `_attendus` calcule $\mathbb{E}[\log|\Lambda_k|]$ (équation 1.29).

```
def _ln_B(W, nu, D):
    """log de la constante de normalisation de la Wishart, ln B(W, nu)."""
```

```
    i = torch.arange(1, D + 1, dtype=torch.float64)
    lg = torch.lgamma((nu[..., None] + 1 - i) / 2).sum(-1)
    return -0.5 * nu * torch.logdet(W) - (0.5 * nu * D * LN2 + D * (D - 1) / 4 * LNPI + lg)

def _attendus(P, D):
    """E[ln|Lambda_k|] (1.29) et E[Lambda_k] pour chaque composante."""
    if P["cov_type"] in ("full", "tied"):
        i = torch.arange(1, D + 1, dtype=torch.float64)
        Elogdet = (torch.special.digamma((P["nu"][:, None] + 1 - i) / 2).sum(-1)
                   + D * LN2 + torch.logdet(P["W"]))
        Elam = P["nu"][:, None, None] * P["W"]
    else:
        psi = torch.special.digamma(P["a"]) - torch.log(P["b"])
        Elogdet = psi.sum(-1)
        Elam = torch.diag_embed(P["a"] / P["b"])
    return Elogdet, Elam

def elbo_gmm_bayesien(r, Nk, xbar, Sk, P, pr: PrioriNW):
    """
    ELBO du GMM bayésien (équation 1.31, détail dans Bishop 2006, éq. 10.70 à 10.77).
    Disponible pour la covariance pleine.
    """
    K, D = xbar.shape
    alpha, beta, m, nu, W = P["alpha"], P["beta"], P["m"], P["nu"], P["W"]
    Elogpi = torch.special.digamma(alpha) - torch.special.digamma(alpha.sum())
    Elogdet, _ = _attendus(P, D)
    dm = xbar - m
    t_trace = torch.einsum("kde,ked->k", Sk, W)
    t_quad = torch.einsum("kd,kde,ke->k", dm, W, dm)
    e_x = 0.5 * (Nk * (Elogdet - D / beta - nu * t_trace - nu * t_quad - D * LOG2PI)).sum()
    e_z = (Nk * Elogpi).sum()
    a0 = pr.alpha0
    ln_c0 = math.lgamma(K * a0) - K * math.lgamma(a0)
    e_pi = ln_c0 + (a0 - 1.0) * Elogpi.sum()
    W0 = torch.linalg.inv(pr.W0inv)
    d0 = m - pr.m0
    quad_m = torch.einsum("kd,kde,ke->k", d0, W, d0)
    nu0_t = torch.tensor(pr.nu0)
    lnB0 = _ln_B(W0, nu0_t, D)
    e_muLam = (0.5 * (D * math.log(pr.beta0 / (2 * math.pi)) + Elogdet - D * pr.beta0 / beta
                      - pr.beta0 * nu * quad_m).sum()
               + K * lnB0 + 0.5 * (pr.nu0 - D - 1) * Elogdet.sum()
               - 0.5 * (nu * torch.einsum("de,ked->k", pr.W0inv, W)).sum())
    q_z = torch.xlogy(r, r).sum()
    ln_c = torch.lgamma(alpha.sum()) - torch.lgamma(alpha).sum()
    q_pi = ((alpha - 1.0) * Elogpi).sum() + ln_c
    H = -_ln_B(W, nu, D) - 0.5 * (nu - D - 1) * Elogdet + 0.5 * nu * D
    q_muLam = (0.5 * Elogdet + 0.5 * D * torch.log(beta / (2 * math.pi)) - 0.5 * D - H).sum()
    return float(e_x + e_z + e_pi + e_muLam - q_z - q_pi - q_muLam)
```

Extrait de code : calcul de l'ELBO du GMM bayésien (équation 1.31)

Annexe C : Table de correspondance des symboles mathématiques

Le tableau ci-dessous rappelle la signification des principaux symboles mathématiques employés dans ce mémoire, à des fins de référence rapide.

Symbole	**Signification**
X = $\{x_1$,. . . ,x_N$\}$	Observations (données)
Z = $\{z_1$,. . . ,z_N$\}$	Variables latentes
θ	Paramètres du modèle
K	Nombre de composantes du mélange
π_k	Poids de la composante k
μ_k, Σ_k	Moyenne et covariance de la composante k
r_{ik}	Responsabilité de la composante k pour l'observation i
$\ell(\theta)$	Log-vraisemblance incomplète
Q($\theta|\theta^t$)	Espérance de la log-vraisemblance complète
ELBO(q,θ)	Borne inférieure de l'évidence
KL[$q\|p$]	Divergence de Kullback-Leibler
q(θ,Z)	Distribution variationnelle
α_0, m_0, β_0, W_0, ν_0	Hyperparamètres des priors (Dirichlet, Normal-Wishart)
$\psi(\cdot)$	Fonction digamma
N_k	Nombre effectif de points assignés à la composante k

Annexe D : Environnement logiciel et reproductibilité

Toutes les expériences ont été exécutées sur une machine unique, avec l'environnement suivant, enregistré automatiquement dans le fichier `environnement.json` :

Composant	**Version utilisée**
Python	3.10.0
PyTorch (CPU, précision double)	2.4.1
Pandas	2.1.4
Plotly	6.8.0
Système	Windows 10, 4 fils de calcul

Les graines suivantes sont fixées explicitement : 42 pour le jeu de référence, 1000 à 1029 pour la réplication de la section 2.3.13, 7 pour les valeurs aberrantes, 0 à 19

pour la robustesse à l'initialisation (section 2.3.9). Chaque ajustement utilise la graine du jeu de données augmentée de l'indice de relance. Avec ces graines et la précision double, deux exécutions successives donnent des résultats identiques ; des différences minimes peuvent apparaître avec une autre version de PyTorch ou un autre processeur, en raison de l'ordre des opérations en virgule flottante.

Le protocole est entièrement scripté : le fichier `main.py` réalise les expériences, écrit les tableaux (fichiers `.csv`) et les figures (`.html` interactifs et `.png`), et le carnet `code.ipynb` présente le protocole, les hypothèses et les résultats dans l'ordre du texte.

Annexe E : Rappels mathématiques utilisés dans les dérivations

Cette dernière annexe rassemble, à des fins de référence, les principales définitions et identités de calcul matriciel et de statistique bayésienne mobilisées dans les dérivations du chapitre 1 (sections 1.6.2.d et 1.7.3.c) et de la section 2.1.1, afin que le mémoire reste autonome et ne nécessite pas de recherche bibliographique complémentaire pour en suivre le détail des calculs.

G.1 Identités de calcul matriciel

Dérivée du logarithme d'un déterminant. Pour une matrice symétrique définie positive A, on a $\partial/\partial A \log|\mathrm{A}| = (A^{-1})^T = A^{-1}$ (la dernière égalité utilisant la symétrie de A). Cette identité est utilisée à la section 1.6.2.d pour dériver la mise à jour de la covariance Σ_k.

Dérivée d'une forme quadratique. Pour un vecteur x et une matrice symétrique A, on a $\partial/\partial A\ (x^T A x) = x x^T$. Combinée à l'identité précédente, cette formule permet d'annuler le gradient de la fonction Q par rapport à la matrice de précision Σ_k^{-1}, comme détaillé à la section 1.6.2.d.

Trace et forme quadratique. Pour tout vecteur x et toute matrice A, la forme quadratique $x^T A x$ peut se réécrire comme une trace : $x^T A x = \mathrm{tr}(A x x^T A^0) = \mathrm{tr}(\mathrm{A}\ x x^T)$, ce qui permet de faire commuter certains calculs d'espérance avec l'opérateur trace une manipulation fréquente, bien que non utilisée explicitement dans les dérivations du chapitre 1, mais standard dans la littérature de l'inférence variationnelle pour les modèles gaussiens.

G.2 La distribution de Wishart

La distribution de Wishart $\mathcal{W}(\mathrm{W}, \nu)$, utilisée à la section 1.7.3 comme composante du prior Normal-Wishart sur la matrice de précision Λ_k, généralise la distribution du χ^2 à

des matrices. Sa densité, pour une matrice symétrique définie positive Λ de dimension $D \times D$, est proportionnelle à :

$$\mathcal{W}(\Lambda|W,\nu) \propto |\Lambda|^{(\nu-D-1)/2} \exp[-\tfrac{1}{2}\operatorname{tr}(W^{-1}\Lambda)] \tag{G.1}$$

où W est la matrice d'échelle ($D \times D$, symétrique définie positive) et $\nu >$ D - 1 le nombre de degrés de liberté. L'espérance de cette distribution vaut $\mathbb{E}[\Lambda] = \nu W$, et l'espérance de son logarithme de déterminant, utilisée à l'équation de la section 1.7.3.e, s'exprime au moyen de la fonction digamma multivariée : $\mathbb{E}[\log|\Lambda|] = \Sigma_{j=1}^{D}\, \psi[(\nu$+1-j$)/2]$ + D log 2 + log|W|, formule qui se déduit de la fonction génératrice des moments de la distribution de Wishart.

G.3 La distribution de Dirichlet

La distribution de Dirichlet Dir($\alpha_1, \ldots, \alpha K$), utilisée à la section 1.4.2 comme prior sur les poids de mélange, est la généralisation multivariée de la distribution Bêta au simplexe de dimension K-1. Sa densité s'écrit p($\pi \mid \alpha$) $\propto \prod_k \pi_k\hat{}\{\alpha_k-1\}$, sous la contrainte $\Sigma_k\, \pi_k = 1$, $\pi_k \geq 0$. Son espérance vaut :

$$\mathbb{E}[\pi_k] = \alpha_k / \Sigma_j \alpha_j \tag{G.2}$$

et l'espérance de son logarithme utilisée à l'équation de la section 1.7.3.e s'exprime, comme pour la Wishart, au moyen de la fonction digamma : $\mathbb{E}[\log\, \pi_k] = \psi(\alpha_k)$ - $\psi(\Sigma_j\, \alpha_j)$. Cette formule intervient dans l'étape E variationnelle (équation 1.27) ; l'exemple de la section 1.4.4, avec $\alpha = (8, 5, 2)$, n'en utilise que l'espérance $\mathbb{E}[\pi_k] = \alpha_k\, /\, \Sigma_j\, \alpha_j$.

G.4 La fonction digamma

La fonction digamma ψ(x) = d/dx log Γ(x), dérivée logarithmique de la fonction Gamma, apparaît systématiquement dans les espérances de logarithmes de variables aléatoires des familles Dirichlet, Wishart et Gamma, en raison de la forme exponentielle de la famille exponentielle sous-jacente à ces distributions. Elle ne possède pas de forme fermée élémentaire, mais admet le développement asymptotique ψ(x) $\approx$ log(x) - 1/(2x) pour x grand, et est implémentée numériquement dans scipy.special.digamma, utilisée dans les deux implémentations from scratch du chapitre 3 (sections 3.2.1 et 3.3.1).